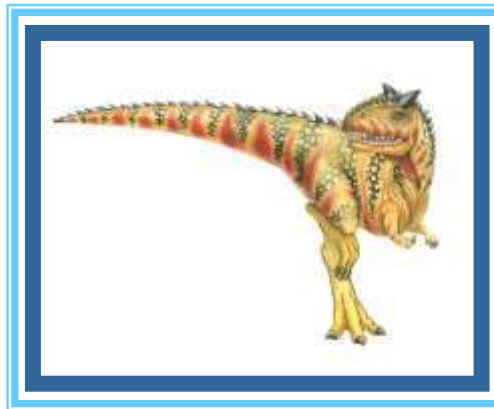


Chapter 1: Introduction





Chapter 1: Introduction

- What Operating Systems Do
- Computer-System Organization
- Computer-System Architecture
- Operating-System Structure
- Operating-System Operations
- Process Management
- Memory Management
- Storage Management
- Protection and Security
- Kernel Data Structures
- Computing Environments
- Open-Source Operating Systems





Objectives

- To describe the basic organization of computer systems
- To provide a grand tour of the major components of operating systems
- To give an overview of the many types of computing environments
- To explore several open-source operating systems





What is an Operating System?

- A program that acts as an intermediary between a user of a computer and the computer hardware
- Operating system goals:
 - Execute user programs and make solving user problems easier
 - Make the computer system convenient to use
 - Use the computer hardware in an efficient manner





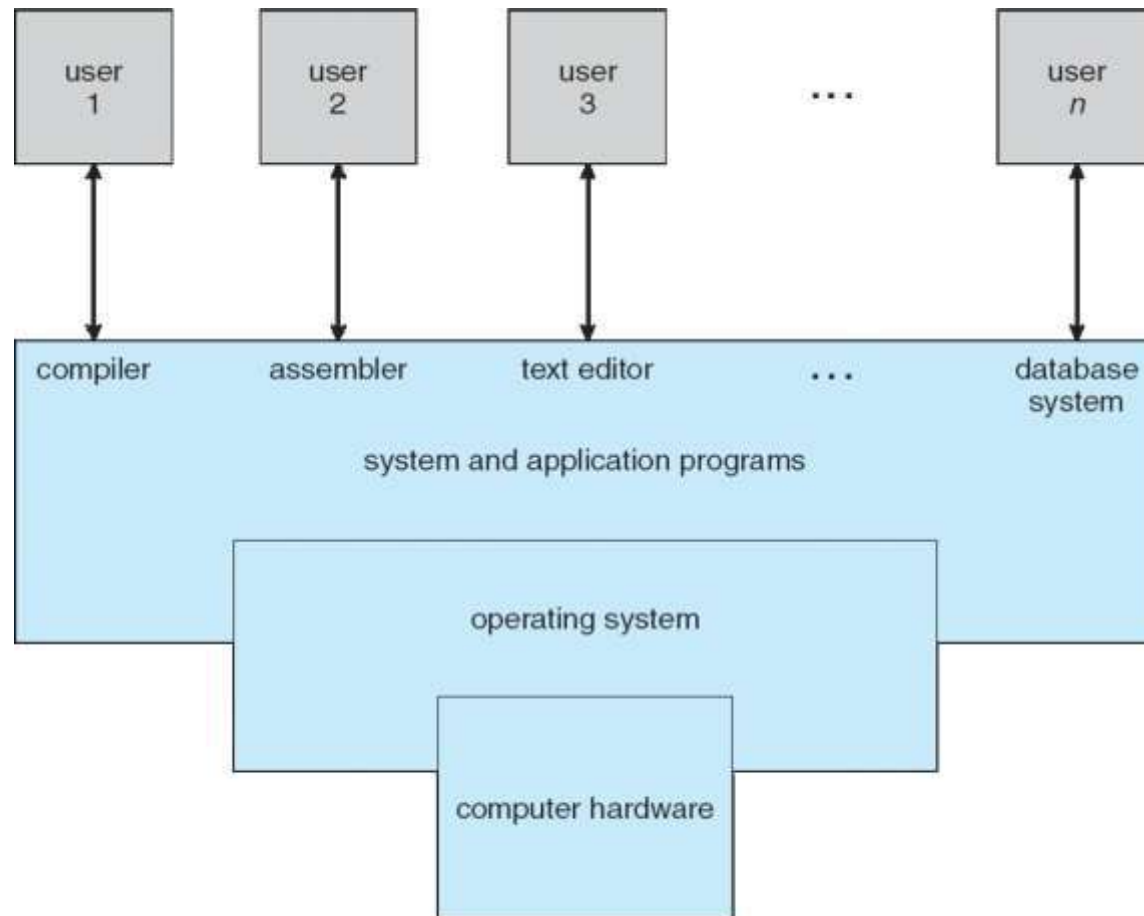
Computer System Structure

- Computer system can be divided into four components:
 - Hardware – provides basic computing resources
 - ▶ CPU, memory, I/O devices
 - Operating system
 - ▶ Controls and coordinates use of hardware among various applications and users
 - Application programs – define the ways in which the system resources are used to solve the computing problems of the users
 - ▶ Word processors, compilers, web browsers, database systems, video games
 - Users
 - ▶ People, machines, other computers





Four Components of a Computer System





What Operating Systems Do

- Depends on the point of view
- Users want convenience, **ease of use** and **good performance**
 - Don't care about **resource utilization**
- But shared computer such as **mainframe** or **minicomputer** must keep all users happy
- Users of dedicated systems such as **workstations** have dedicated resources but frequently use shared resources from **servers**
- Handheld computers are resource poor, optimized for usability and battery life
- Some computers have little or no user interface, such as embedded computers in devices and automobiles





Operating System Definition

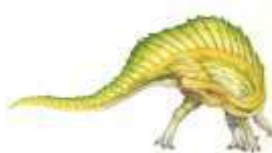
- OS is a **resource allocator**
 - Manages all resources
 - Decides between conflicting requests for efficient and fair resource use
- OS is a **control program**
 - Controls execution of programs to prevent errors and improper use of the computer





Operating System Definition (Cont.)

- No universally accepted definition
- “Everything a vendor ships when you order an operating system” is a good approximation
 - But varies wildly
- “The one program running at all times on the computer” is the **kernel**.
- Everything else is either
 - a system program (ships with the operating system) , or
 - an application program.





Computer Startup

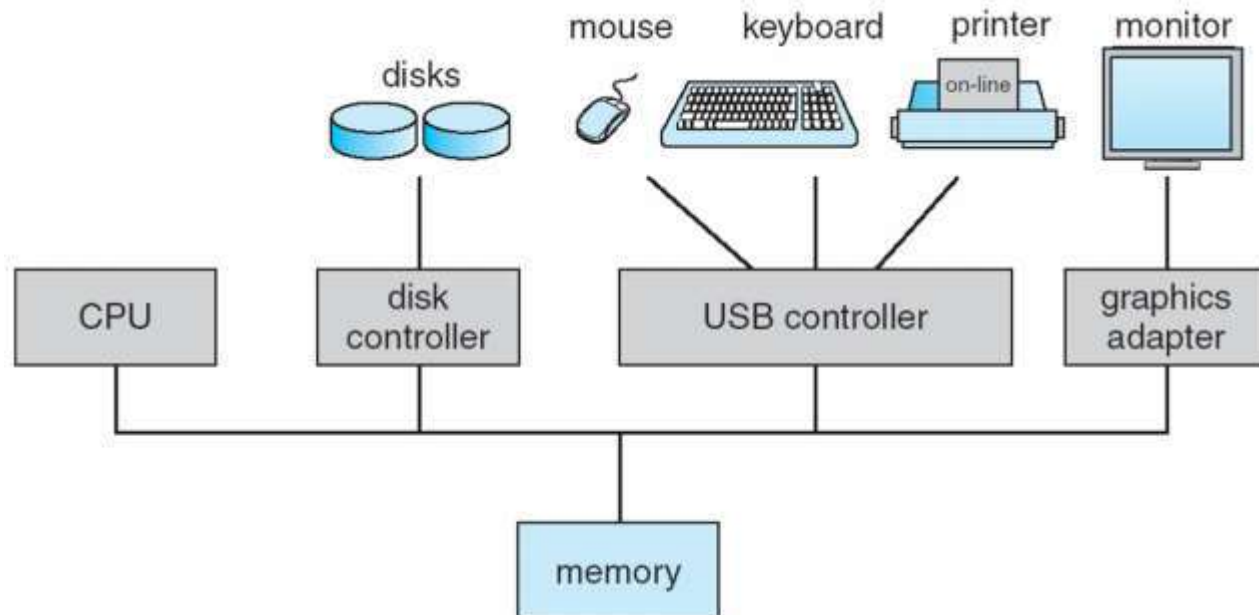
- **bootstrap program** is loaded at power-up or reboot
 - Typically stored in ROM or EPROM, generally known as **firmware**
 - Initializes all aspects of system
 - Loads operating system kernel and starts execution





Computer System Organization

- Computer-system operation
 - One or more CPUs, device controllers connect through common bus providing access to shared memory
 - Concurrent execution of CPUs and devices competing for memory cycles





Computer-System Operation

- I/O devices and the CPU can execute concurrently
- Each device controller is in charge of a particular device type
- Each device controller has a local buffer
- CPU moves data from/to main memory to/from local buffers
- I/O is from the device to local buffer of controller
- Device controller informs CPU that it has finished its operation by causing an **interrupt**





Common Functions of Interrupts

- Interrupt transfers control to the interrupt service routine generally, through the **interrupt vector**, which contains the addresses of all the service routines
- Interrupt architecture must save the address of the interrupted instruction
- A **trap** or **exception** is a software-generated interrupt caused either by an error or a user request
- An operating system is **interrupt driven**





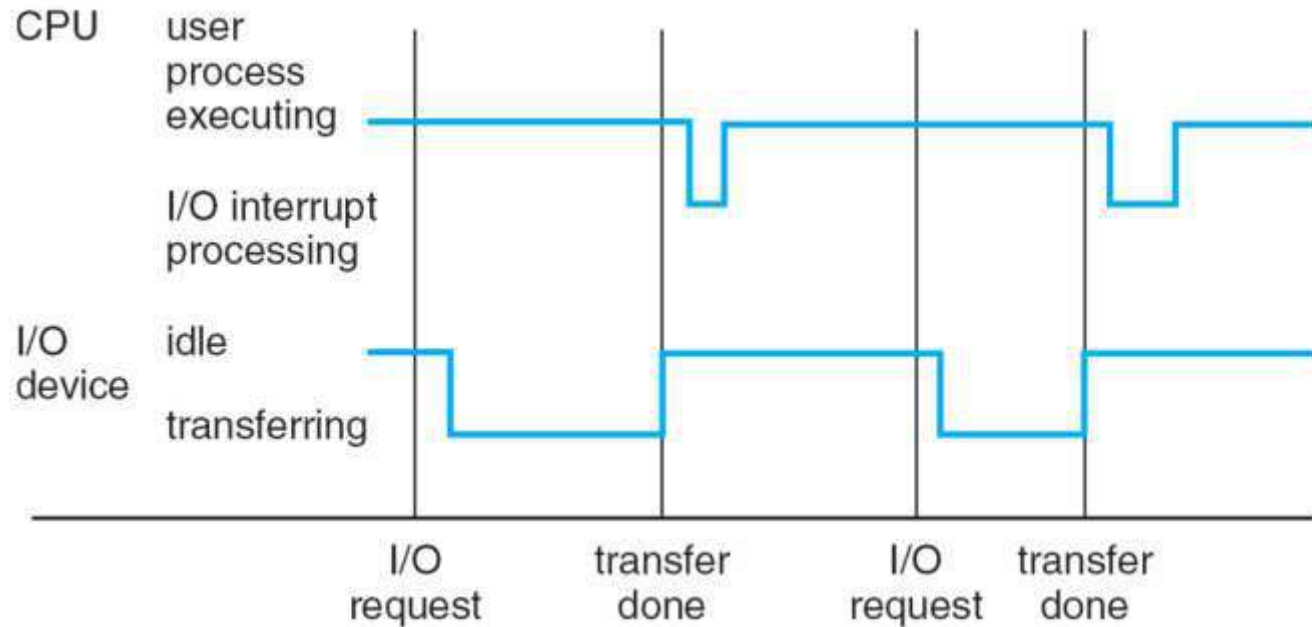
Interrupt Handling

- The operating system preserves the state of the CPU by storing registers and the program counter
- Determines which type of interrupt has occurred:
 - **polling**
 - **vectored** interrupt system
- Separate segments of code determine what action should be taken for each type of interrupt





Interrupt Timeline





I/O Structure

- After I/O starts, control returns to user program only upon I/O completion
 - Wait instruction idles the CPU until the next interrupt
 - Wait loop (contention for memory access)
 - At most one I/O request is outstanding at a time, no simultaneous I/O processing
- After I/O starts, control returns to user program without waiting for I/O completion
 - **System call** – request to the OS to allow user to wait for I/O completion
 - **Device-status table** contains entry for each I/O device indicating its type, address, and state
 - OS indexes into I/O device table to determine device status and to modify table entry to include interrupt





Storage Definitions and Notation Review

The basic unit of computer storage is the **bit**. A bit can contain one of two values, 0 and 1. All other storage in a computer is based on collections of bits. Given enough bits, it is amazing how many things a computer can represent: numbers, letters, images, movies, sounds, documents, and programs, to name a few. A **byte** is 8 bits, and on most computers it is the smallest convenient chunk of storage. For example, most computers don't have an instruction to move a bit but do have one to move a byte. A less common term is **word**, which is a given computer architecture's native unit of data. A word is made up of one or more bytes. For example, a computer that has 64-bit registers and 64-bit memory addressing typically has 64-bit (8-byte) words. A computer executes many operations in its native word size rather than a byte at a time.

Computer storage, along with most computer throughput, is generally measured and manipulated in bytes and collections of bytes.

A **kilobyte**, or **KB**, is 1,024 bytes

a **megabyte**, or **MB**, is $1,024^2$ bytes

a **gigabyte**, or **GB**, is $1,024^3$ bytes

a **terabyte**, or **TB**, is $1,024^4$ bytes

a **petabyte**, or **PB**, is $1,024^5$ bytes

Computer manufacturers often round off these numbers and say that a megabyte is 1 million bytes and a gigabyte is 1 billion bytes. Networking measurements are an exception to this general rule; they are given in bits (because networks move data a bit at a time).





Storage Structure

- Main memory – only large storage media that the CPU can access directly
 - **Random access**
 - Typically **volatile**
- Secondary storage – extension of main memory that provides large **nonvolatile** storage capacity
- Hard disks – rigid metal or glass platters covered with magnetic recording material
 - Disk surface is logically divided into **tracks**, which are subdivided into **sectors**
 - The **disk controller** determines the logical interaction between the device and the computer
- **Solid-state disks** – faster than hard disks, nonvolatile
 - Various technologies
 - Becoming more popular





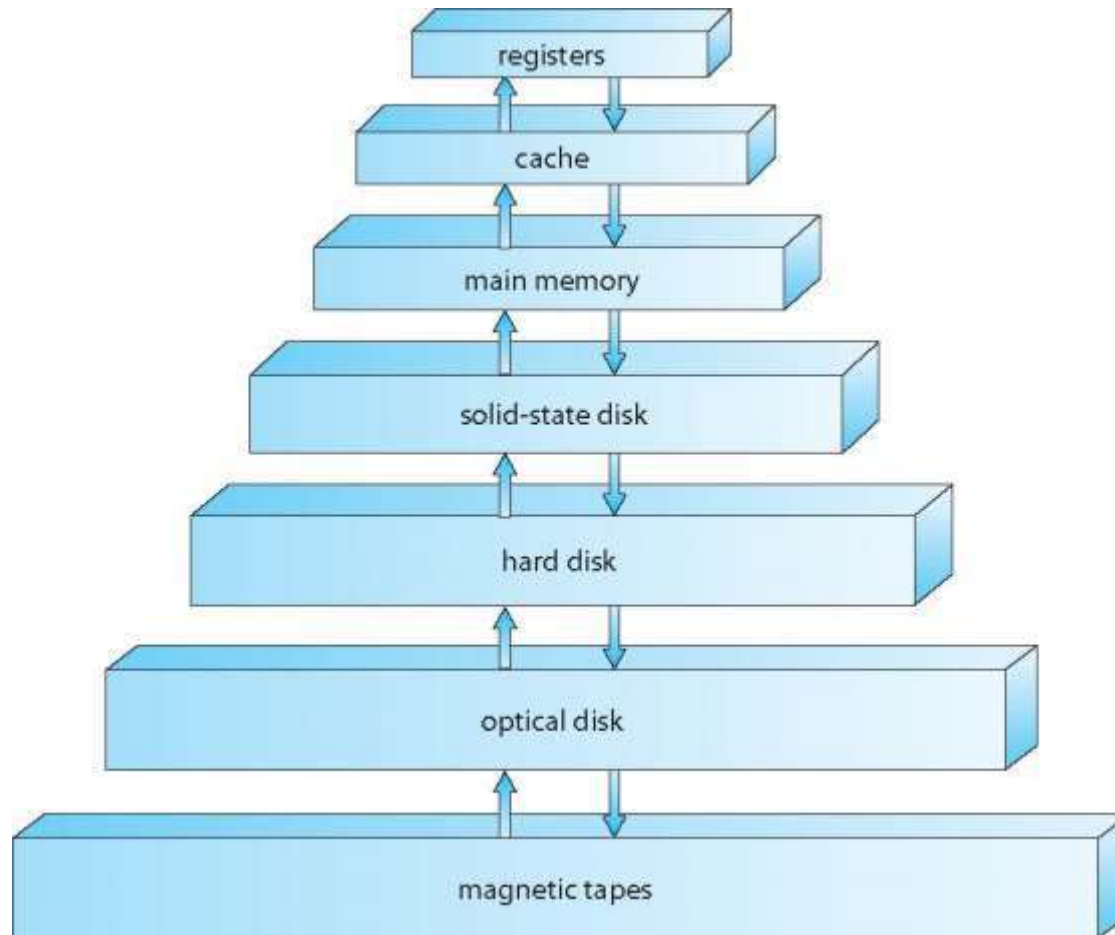
Storage Hierarchy

- Storage systems organized in hierarchy
 - Speed
 - Cost
 - Volatility
- **Caching** – copying information into faster storage system; main memory can be viewed as a cache for secondary storage
- **Device Driver** for each device controller to manage I/O
 - Provides uniform interface between controller and kernel





Storage-Device Hierarchy





Caching

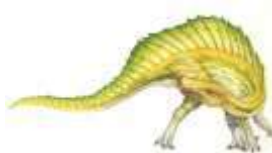
- Important principle, performed at many levels in a computer (in hardware, operating system, software)
- Information in use copied from slower to faster storage temporarily
- Faster storage (cache) checked first to determine if information is there
 - If it is, information used directly from the cache (fast)
 - If not, data copied to cache and used there
- Cache smaller than storage being cached
 - Cache management important design problem
 - Cache size and replacement policy





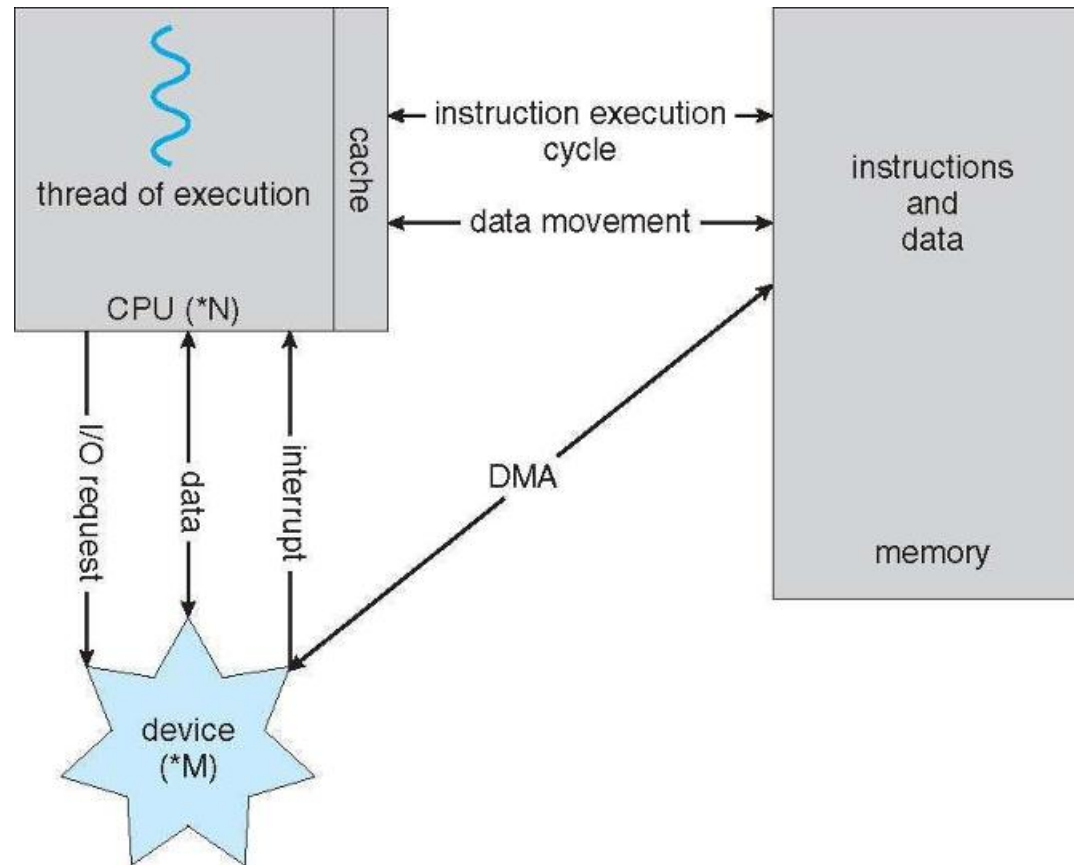
Direct Memory Access Structure

- Used for high-speed I/O devices able to transmit information at close to memory speeds
- Device controller transfers blocks of data from buffer storage directly to main memory without CPU intervention
- Only one interrupt is generated per block, rather than the one interrupt per byte





How a Modern Computer Works



A von Neumann architecture





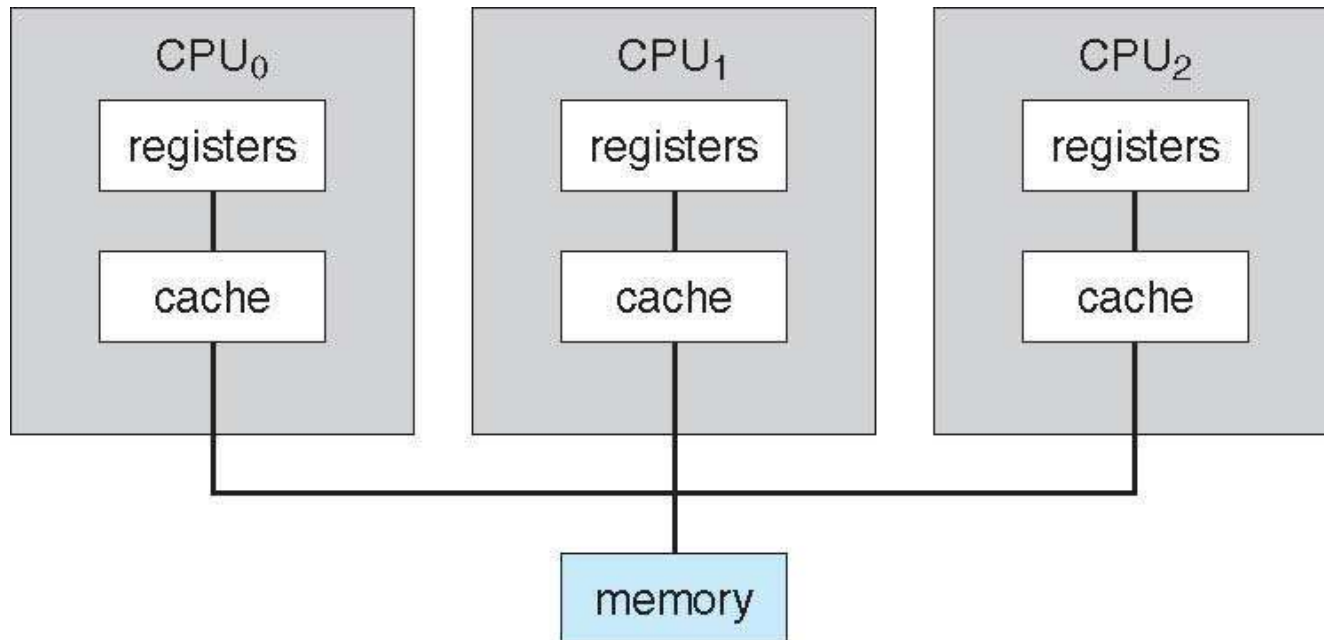
Computer-System Architecture

- Most systems use a single general-purpose processor
 - Most systems have special-purpose processors as well
- **Multiprocessors** systems growing in use and importance
 - Also known as **parallel systems**, **tightly-coupled systems**
 - Advantages include:
 1. **Increased throughput**
 2. **Economy of scale**
 3. **Increased reliability** – graceful degradation or fault tolerance
 - Two types:
 1. **Asymmetric Multiprocessing** – each processor is assigned a specific task.
 2. **Symmetric Multiprocessing** – each processor performs all tasks





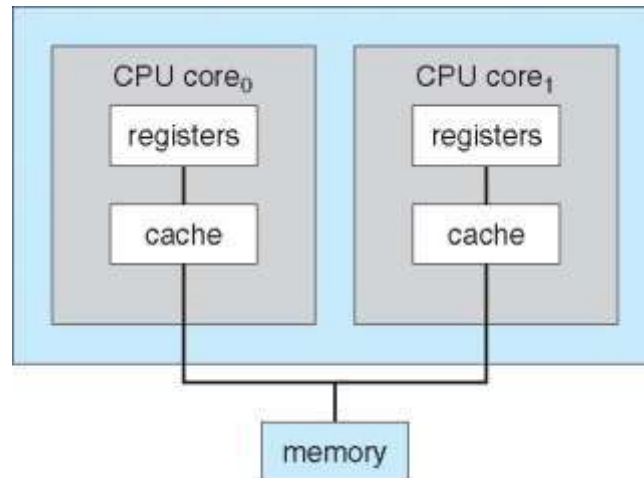
Symmetric Multiprocessing Architecture





A Dual-Core Design

- Multi-chip and **multicore**
- Systems containing all chips
 - Chassis containing multiple separate systems





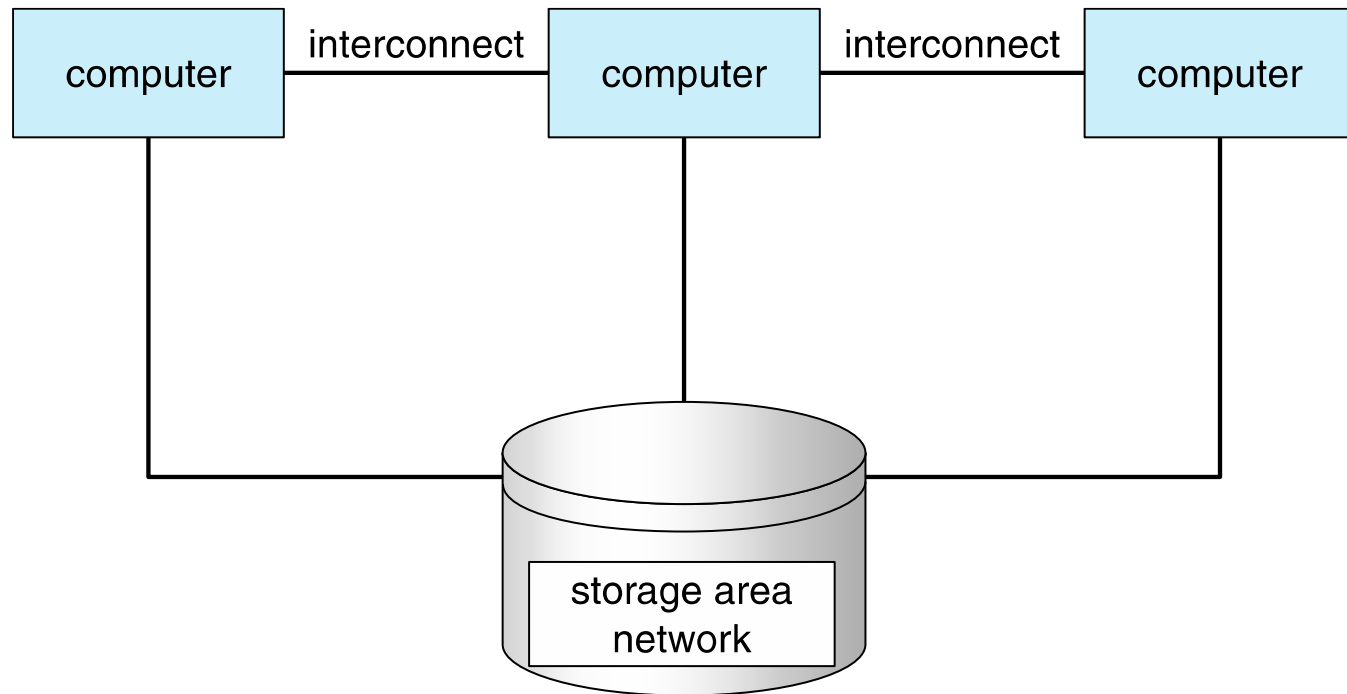
Clustered Systems

- Like multiprocessor systems, but multiple systems working together
 - Usually sharing storage via a **storage-area network (SAN)**
 - Provides a **high-availability** service which survives failures
 - ▶ **Asymmetric clustering** has one machine in hot-standby mode
 - ▶ **Symmetric clustering** has multiple nodes running applications, monitoring each other
 - Some clusters are for **high-performance computing (HPC)**
 - ▶ Applications must be written to use **parallelization**
 - Some have **distributed lock manager (DLM)** to avoid conflicting operations





Clustered Systems





Operating System Structure

- **Multiprogramming (Batch system)** needed for efficiency
 - Single user cannot keep CPU and I/O devices busy at all times
 - Multiprogramming organizes jobs (code and data) so CPU always has one to execute
 - A subset of total jobs in system is kept in memory
 - One job selected and run via **job scheduling**
 - When it has to wait (for I/O for example), OS switches to another job
- **Timesharing (multitasking)** is logical extension in which CPU switches jobs so frequently that users can interact with each job while it is running, creating **interactive** computing
 - **Response time** should be < 1 second
 - Each user has at least one program executing in memory $\Rightarrow$ **process**
 - If several jobs ready to run at the same time $\Rightarrow$ **CPU scheduling**
 - If processes don't fit in memory, **swapping** moves them in and out to run
 - **Virtual memory** allows execution of processes not completely in memory





Memory Layout for Multiprogrammed System





Operating-System Operations

- **Interrupt driven** (hardware and software)
 - Hardware interrupt by one of the devices
 - Software interrupt (**exception** or **trap**):
 - ▶ Software error (e.g., division by zero)
 - ▶ Request for operating system service
 - ▶ Other process problems include infinite loop, processes modifying each other or the operating system





Operating-System Operations (cont.)

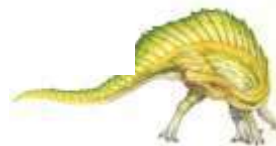
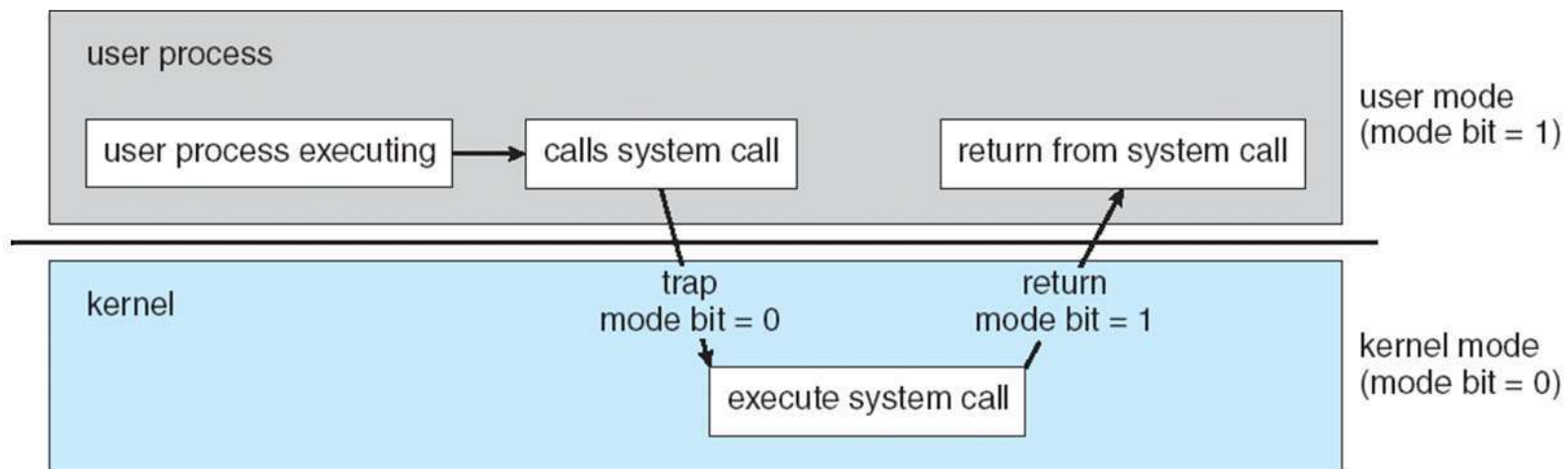
- **Dual-mode** operation allows OS to protect itself and other system components
 - **User mode** and **kernel mode**
 - **Mode bit** provided by hardware
 - ▶ Provides ability to distinguish when system is running user code or kernel code
 - ▶ Some instructions designated as **privileged**, only executable in kernel mode
 - ▶ System call changes mode to kernel, return from call resets it to user
- Increasingly CPUs support multi-mode operations
 - i.e. **virtual machine manager (VMM)** mode for guest **VMs**





Transition from User to Kernel Mode

- Timer to prevent infinite loop / process hogging resources
 - Timer is set to interrupt the computer after some time period
 - Keep a counter that is decremented by the physical clock.
 - Operating system set the counter (privileged instruction)
 - When counter zero generate an interrupt
 - Set up before scheduling process to regain control or terminate program that exceeds allotted time





Process Management

- A process is a program in execution. It is a unit of work within the system. Program is a ***passive entity***, process is an ***active entity***.
- Process needs resources to accomplish its task
 - CPU, memory, I/O, files
 - Initialization data
- Process termination requires reclaim of any reusable resources
- Single-threaded process has one **program counter** specifying location of next instruction to execute
 - Process executes instructions sequentially, one at a time, until completion
- Multi-threaded process has one program counter per thread
- Typically system has many processes, some user, some operating system running concurrently on one or more CPUs
 - Concurrency by multiplexing the CPUs among the processes / threads

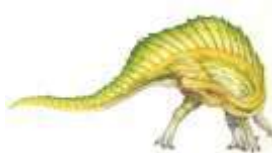




Process Management Activities

The operating system is responsible for the following activities in connection with process management:

- Creating and deleting both user and system processes
- Suspending and resuming processes
- Providing mechanisms for process synchronization
- Providing mechanisms for process communication
- Providing mechanisms for deadlock handling





Memory Management

- To execute a program all (or part) of the instructions must be in memory
- All (or part) of the data that is needed by the program must be in memory.
- Memory management determines what is in memory and when
 - Optimizing CPU utilization and computer response to users
- Memory management activities
 - Keeping track of which parts of memory are currently being used and by whom
 - Deciding which processes (or parts thereof) and data to move into and out of memory
 - Allocating and deallocating memory space as needed





Storage Management

- OS provides uniform, logical view of information storage
 - Abstracts physical properties to logical storage unit - **file**
 - Each medium is controlled by device (i.e., disk drive, tape drive)
 - ▶ Varying properties include access speed, capacity, data-transfer rate, access method (sequential or random)
- File-System management
 - Files usually organized into directories
 - Access control on most systems to determine who can access what
 - OS activities include
 - ▶ Creating and deleting files and directories
 - ▶ Primitives to manipulate files and directories
 - ▶ Mapping files onto secondary storage
 - ▶ Backup files onto stable (non-volatile) storage media





Mass-Storage Management

- Usually disks used to store data that does not fit in main memory or data that must be kept for a “long” period of time
- Proper management is of central importance
- Entire speed of computer operation hinges on disk subsystem and its algorithms
- OS activities
 - Free-space management
 - Storage allocation
 - Disk scheduling
- Some storage need not be fast
 - Tertiary storage includes optical storage, magnetic tape
 - Still must be managed – by OS or applications
 - Varies between WORM (write-once, read-many-times) and RW (read-write)





Performance of Various Levels of Storage

Level	1	2	3	4	5
Name	registers	cache	main memory	solid state disk	magnetic disk
Typical size	< 1 KB	< 16MB	< 64GB	< 1 TB	< 10 TB
Implementation technology	custom memory with multiple ports CMOS	on-chip or off-chip CMOS SRAM	CMOS SRAM	flash memory	magnetic disk
Access time (ns)	0.25 - 0.5	0.5 - 25	80 - 250	25,000 - 50,000	5,000,000
Bandwidth (MB/sec)	20,000 - 100,000	5,000 - 10,000	1,000 - 5,000	500	20 - 150
Managed by	compiler	hardware	operating system	operating system	operating system
Backed by	cache	main memory	disk	disk	disk or tape

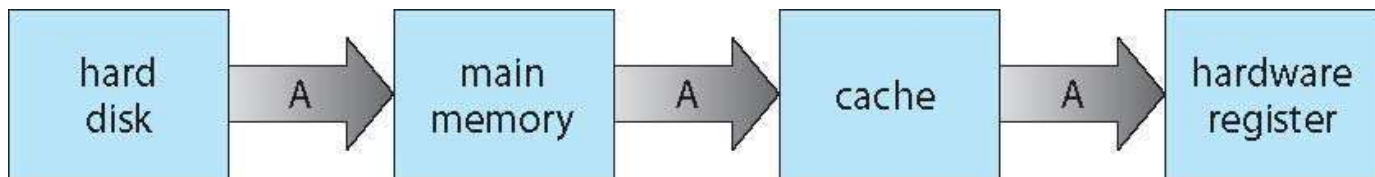
Movement between levels of storage hierarchy can be explicit or implicit





Migration of data “A” from Disk to Register

- Multitasking environments must be careful to use most recent value, no matter where it is stored in the storage hierarchy



- Multiprocessor environment must provide **cache coherency** in hardware such that all CPUs have the most recent value in their cache
- Distributed environment situation even more complex
 - Several copies of a datum can exist
 - Various solutions covered in Chapter 17





I/O Subsystem

- One purpose of OS is to hide peculiarities of hardware devices from the user
- I/O subsystem responsible for
 - Memory management of I/O including buffering (storing data temporarily while it is being transferred), caching (storing parts of data in faster storage for performance), spooling (the overlapping of output of one job with input of other jobs)
 - General device-driver interface
 - Drivers for specific hardware devices





Protection and Security

- **Protection** – any mechanism for controlling access of processes or users to resources defined by the OS
- **Security** – defense of the system against internal and external attacks
 - Huge range, including denial-of-service, worms, viruses, identity theft, theft of service
- Systems generally first distinguish among users, to determine who can do what
 - User identities (**user IDs**, security IDs) include name and associated number, one per user
 - User ID then associated with all files, processes of that user to determine access control
 - Group identifier (**group ID**) allows set of users to be defined and controls managed, then also associated with each process, file
 - **Privilege escalation** allows user to change to effective ID with more rights

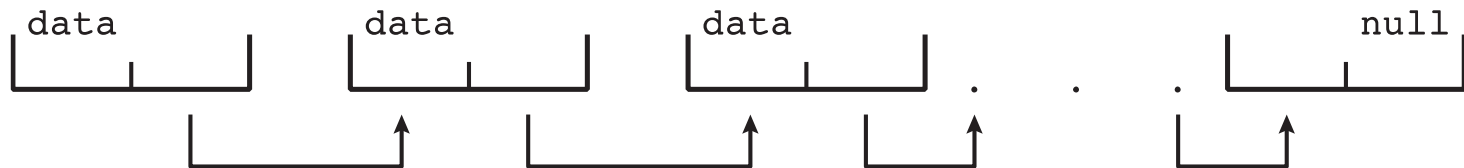




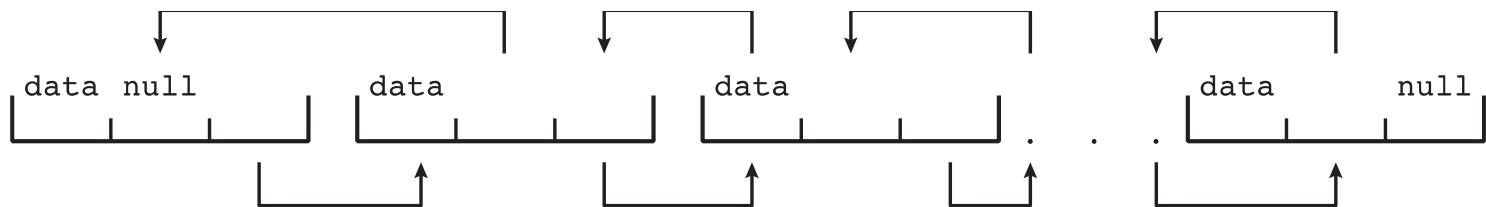
Kernel Data Structures

n Many similar to standard programming data structures

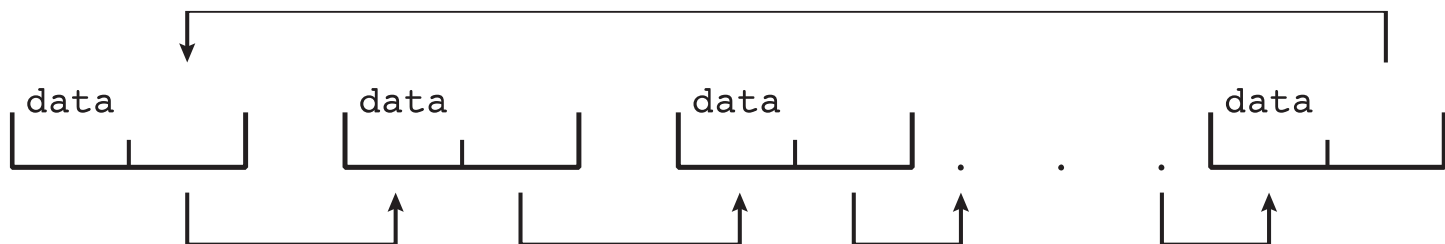
n ***Singly linked list***



n ***Doubly linked list***



n ***Circular linked list***



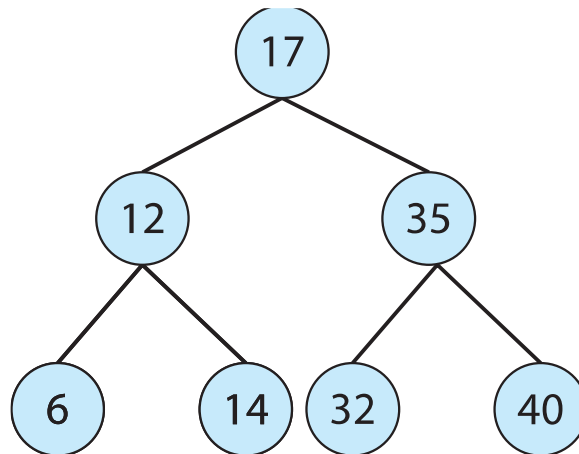


Kernel Data Structures

■ Binary search tree

left $\leq$ right

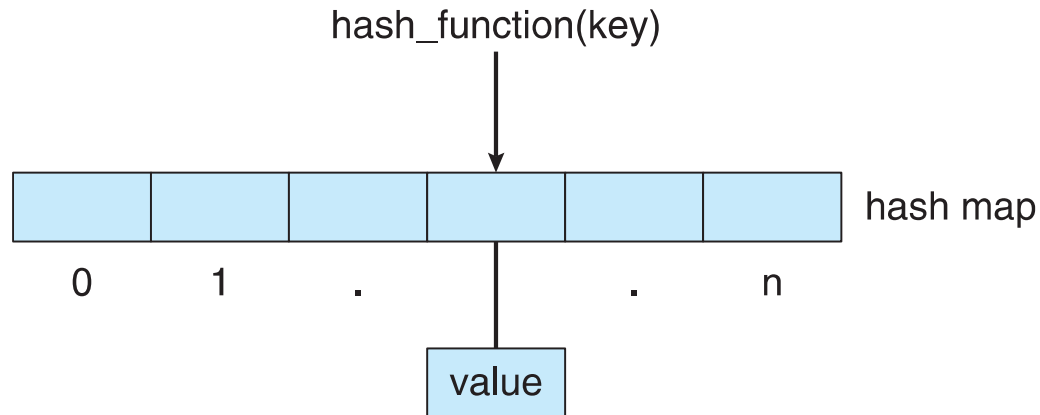
- Search performance is $O(n)$
- **Balanced binary search tree** is $O(\lg n)$





Kernel Data Structures

- **Hash function** can create a **hash map**



- **Bitmap** – string of n binary digits representing the status of n items
- Linux data structures defined in
include files `<linux/list.h>`, `<linux/kfifo.h>`,
`<linux/rbtree.h>`





Computing Environments - Traditional

- Stand-alone general purpose machines
- But blurred as most systems interconnect with others (i.e., the Internet)
- **Portals** provide web access to internal systems
- **Network computers** (**thin clients**) are like Web terminals
- Mobile computers interconnect via **wireless networks**
- Networking becoming ubiquitous – even home systems use **firewalls** to protect home computers from Internet attacks





Computing Environments - Mobile

- Handheld smartphones, tablets, etc
- What is the functional difference between them and a “traditional” laptop?
- Extra feature – more OS features (GPS, gyroscope)
- Allows new types of apps like ***augmented reality***
- Use IEEE 802.11 wireless, or cellular data networks for connectivity
- Leaders are **Apple iOS** and **Google Android**





Computing Environments – Distributed

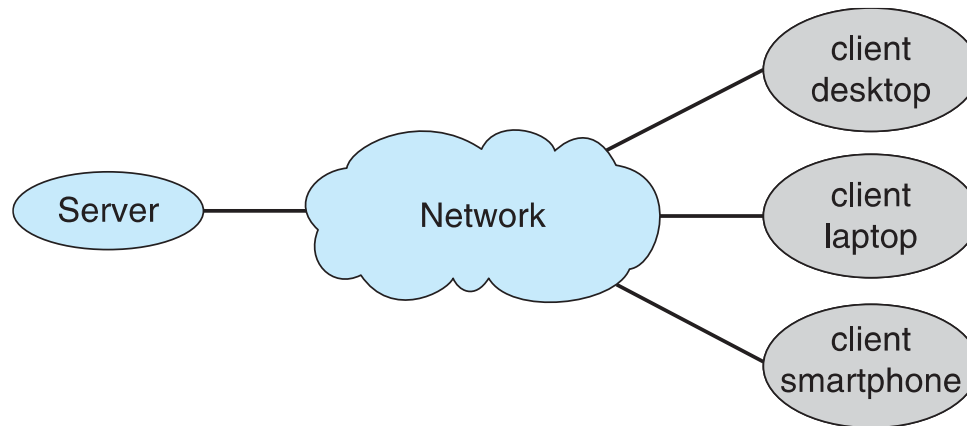
- Distributed computing
 - Collection of separate, possibly heterogeneous, systems networked together
 - ▶ **Network** is a communications path, **TCP/IP** most common
 - **Local Area Network (LAN)**
 - **Wide Area Network (WAN)**
 - **Metropolitan Area Network (MAN)**
 - **Personal Area Network (PAN)**
 - **Network Operating System** provides features between systems across network
 - ▶ Communication scheme allows systems to exchange messages
 - ▶ Illusion of a single system





Computing Environments – Client-Server

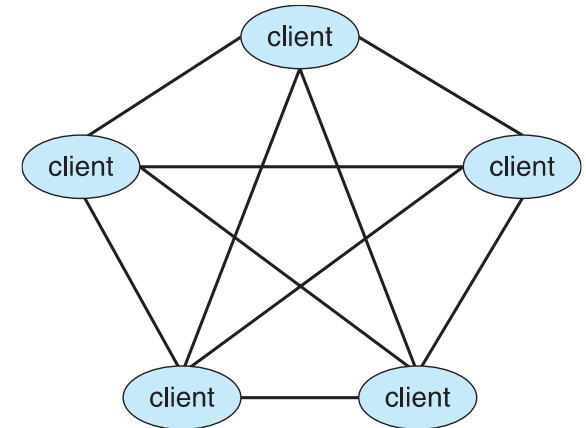
- Client-Server Computing
 - Dumb terminals supplanted by smart PCs
 - Many systems now **servers**, responding to requests generated by **clients**
 - ▶ **Compute-server system** provides an interface to client to request services (i.e., database)
 - ▶ **File-server system** provides interface for clients to store and retrieve files





Computing Environments - Peer-to-Peer

- Another model of distributed system
- P2P does not distinguish clients and servers
 - Instead all nodes are considered peers
 - May each act as client, server or both
 - Node must join P2P network
 - ▶ Registers its service with central lookup service on network, or
 - ▶ Broadcast request for service and respond to requests for service via ***discovery protocol***
 - Examples include Napster and Gnutella, **Voice over IP (VoIP)** such as Skype





Computing Environments - Virtualization

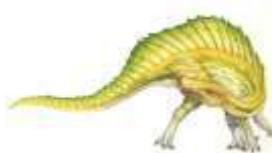
- Allows operating systems to run applications within other OSES
 - Vast and growing industry
- **Emulation** used when source CPU type different from target type (i.e. PowerPC to Intel x86)
 - Generally slowest method
 - When computer language not compiled to native code – **Interpretation**
- **Virtualization** – OS natively compiled for CPU, running **guest** OSES also natively compiled
 - Consider VMware running WinXP guests, each running applications, all on native WinXP **host** OS
 - **VMM** (virtual machine Manager) provides virtualization services





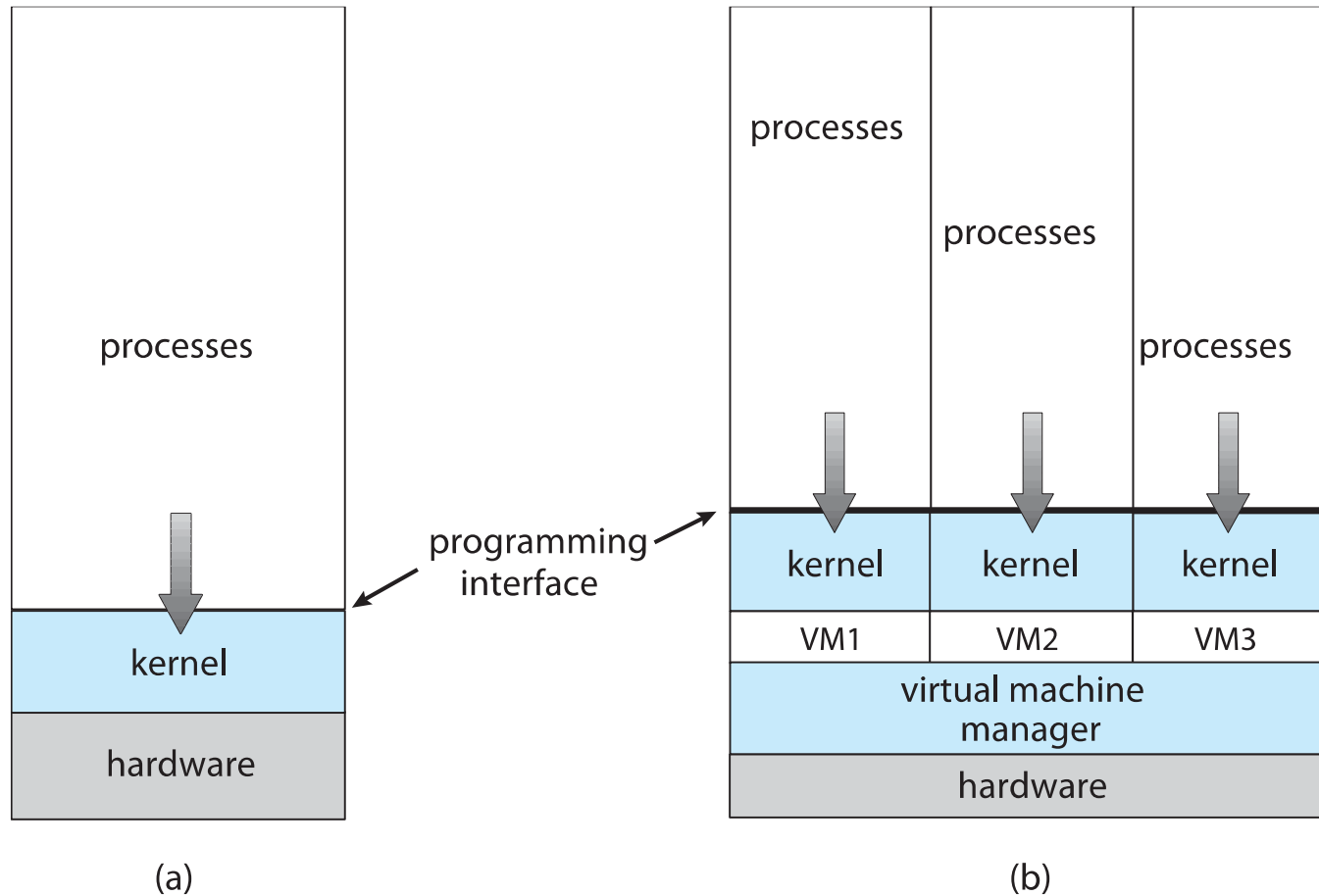
Computing Environments - Virtualization

- Use cases involve laptops and desktops running multiple OSES for exploration or compatibility
 - Apple laptop running Mac OS X host, Windows as a guest
 - Developing apps for multiple OSES without having multiple systems
 - QA testing applications without having multiple systems
 - Executing and managing compute environments within data centers
- VMM can run natively, in which case they are also the host
 - There is no general purpose host then (VMware ESX and Citrix XenServer)





Computing Environments - Virtualization





Computing Environments – Cloud Computing

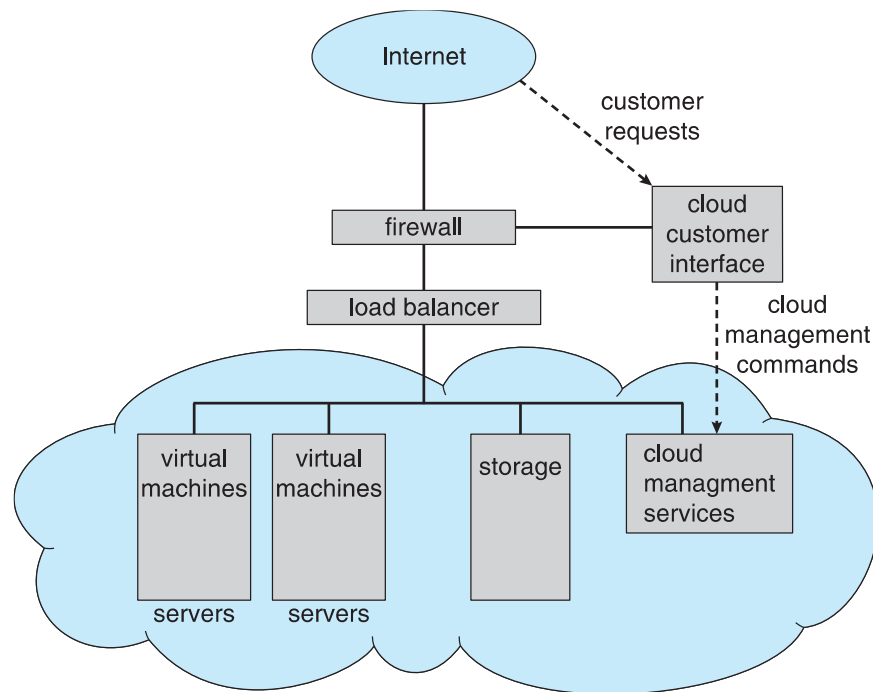
- Delivers computing, storage, even apps as a service across a network
- Logical extension of virtualization because it uses virtualization as the base for its functionality.
 - Amazon **EC2** has thousands of servers, millions of virtual machines, petabytes of storage available across the Internet, pay based on usage
- Many types
 - **Public cloud** – available via Internet to anyone willing to pay
 - **Private cloud** – run by a company for the company's own use
 - **Hybrid cloud** – includes both public and private cloud components
 - Software as a Service (**SaaS**) – one or more applications available via the Internet (i.e., word processor)
 - Platform as a Service (**PaaS**) – software stack ready for application use via the Internet (i.e., a database server)
 - Infrastructure as a Service (**IaaS**) – servers or storage available over Internet (i.e., storage available for backup use)





Computing Environments – Cloud Computing

- Cloud computing environments composed of traditional OSEs, plus VMMs, plus cloud management tools
 - Internet connectivity requires security like firewalls
 - Load balancers spread traffic across multiple applications





Computing Environments – Real-Time Embedded Systems

- Real-time embedded systems most prevalent form of computers
 - Vary considerable, special purpose, limited purpose OS, **real-time OS**
 - Use expanding
- Many other special computing environments as well
 - Some have OSES, some perform tasks without an OS
- Real-time OS has well-defined fixed time constraints
 - Processing ***must*** be done within constraint
 - Correct operation only if constraints met



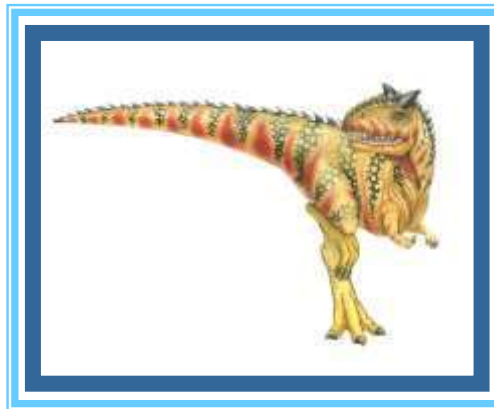


Open-Source Operating Systems

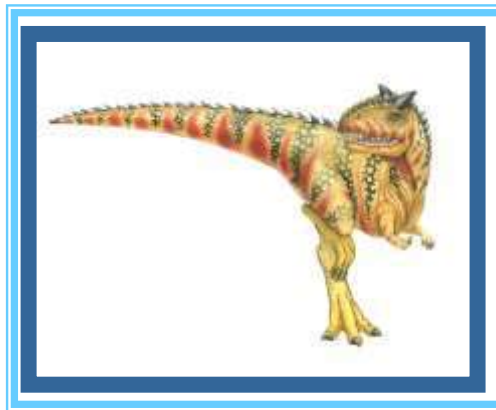
- Operating systems made available in source-code format rather than just binary **closed-source**
- Counter to the **copy protection** and **Digital Rights Management (DRM)** movement
- Started by **Free Software Foundation (FSF)**, which has “copyleft” **GNU Public License (GPL)**
- Examples include **GNU/Linux** and **BSD UNIX** (including core of **Mac OS X**), and many more
- Can use VMM like VMware Player (Free on Windows), Virtualbox (open source and free on many platforms - <http://www.virtualbox.com>)
 - Use to run guest operating systems for exploration



End of Chapter 1



Chapter 2: Operating-System Structures





Chapter 2: Operating-System Structures

- Operating System Services
- User Operating System Interface
- System Calls
- Types of System Calls
- System Programs
- Operating System Design and Implementation
- Operating System Structure
- Virtual Machines
- Operating System Debugging
- Operating System Generation
- System Boot





Objectives

- To describe the services an operating system provides to users, processes, and other systems
- To discuss the various ways of structuring an operating system
- To explain how operating systems are installed and customized and how they boot





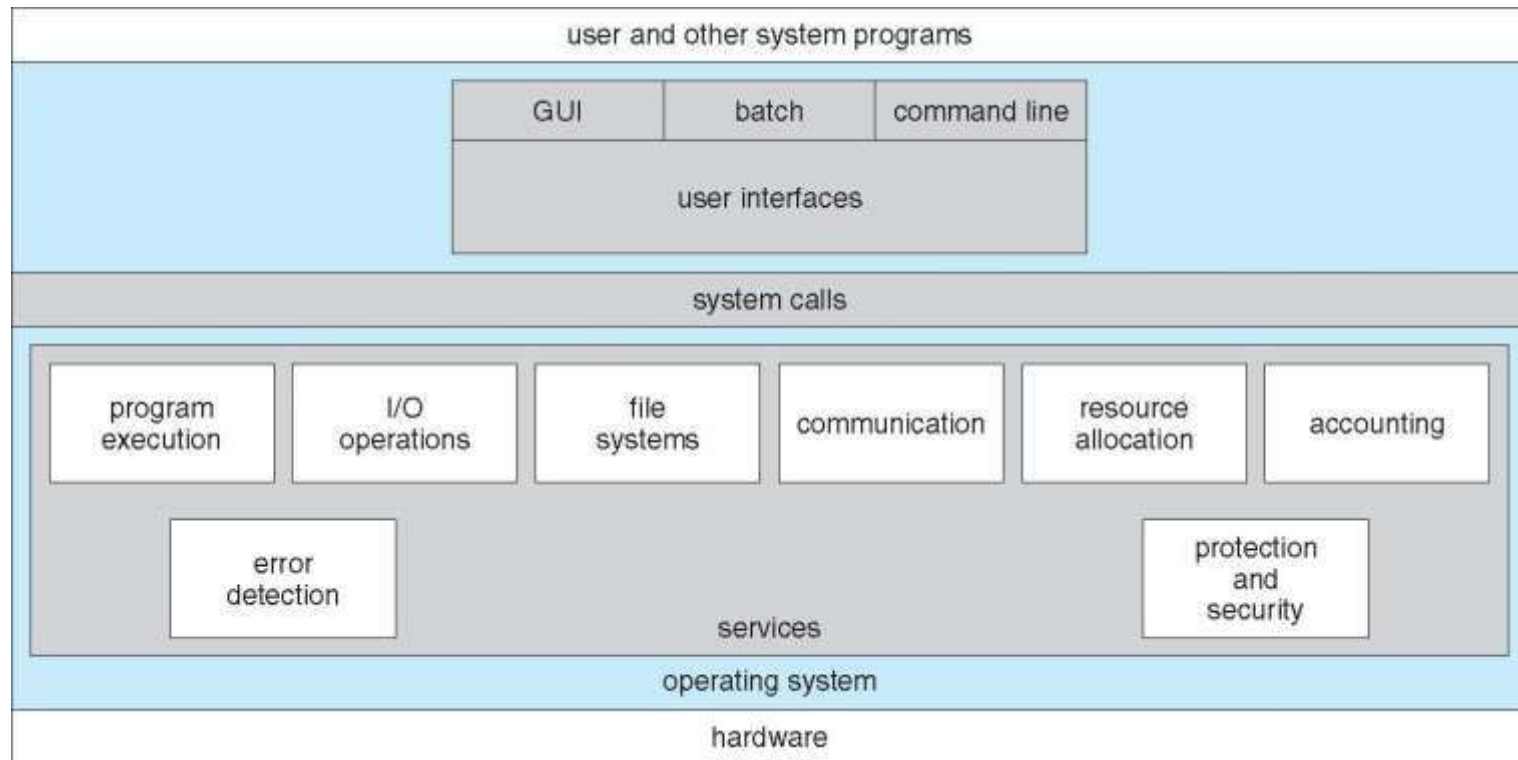
Operating System Services

- One set of operating-system services provides functions that are helpful to the user:
 - User interface - Almost all operating systems have a user interface (UI)
 - ▶ Varies between **Command-Line (CLI)**, **Graphics User Interface (GUI)**, **Batch**
 - Program execution - The system must be able to load a program into memory and to run that program, end execution, either normally or abnormally (indicating error)
 - I/O operations - A running program may require I/O, which may involve a file or an I/O device
 - File-system manipulation - The file system is of particular interest. Obviously, programs need to read and write files and directories, create and delete them, search them, list file Information, permission management.





A View of Operating System Services





Operating System Services (Cont)

- One set of operating-system services provides functions that are helpful to the user (Cont):
 - Communications – Processes may exchange information, on the same computer or between computers over a network
 - ▶ Communications may be via shared memory or through message passing (packets moved by the OS)
 - Error detection – OS needs to be constantly aware of possible errors
 - ▶ May occur in the CPU and memory hardware, in I/O devices, in user program
 - ▶ For each type of error, OS should take the appropriate action to ensure correct and consistent computing
 - ▶ Debugging facilities can greatly enhance the user's and programmer's abilities to efficiently use the system





Operating System Services (Cont)

- Another set of OS functions exists for ensuring the efficient operation of the system itself via resource sharing
 - **Resource allocation** - When multiple users or multiple jobs running concurrently, resources must be allocated to each of them
 - ▶ Many types of resources - Some (such as CPU cycles, main memory, and file storage) may have special allocation code, others (such as I/O devices) may have general request and release code
 - **Accounting** - To keep track of which users use how much and what kinds of computer resources
 - **Protection and security** - The owners of information stored in a multiuser or networked computer system may want to control use of that information, concurrent processes should not interfere with each other
 - ▶ **Protection** involves ensuring that all access to system resources is controlled
 - ▶ **Security** of the system from outsiders requires user authentication, extends to defending external I/O devices from invalid access attempts
 - ▶ If a system is to be protected and secure, precautions must be instituted throughout it. A chain is only as strong as its weakest link.





User Operating System Interface - CLI

Command Line Interface (CLI) or **command interpreter** allows direct command entry

- ▶ Sometimes implemented in kernel, sometimes by systems program
- ▶ Sometimes multiple flavors implemented – **shells**
- ▶ Primarily fetches a command from user and executes it
 - Sometimes commands built-in, sometimes just names of programs
 - » If the latter, adding new features doesn't require shell modification





User Operating System Interface - GUI

- User-friendly **desktop** metaphor interface
 - Usually mouse, keyboard, and monitor
 - **Icons** represent files, programs, actions, etc
 - Various mouse buttons over objects in the interface cause various actions (provide information, options, execute function, open directory (known as a **folder**))
 - Invented at Xerox PARC
- Many systems now include both CLI and GUI interfaces
 - Microsoft Windows is GUI with CLI “command” shell
 - Apple Mac OS X as “Aqua” GUI interface with UNIX kernel underneath and shells available
 - Solaris is CLI with optional GUI interfaces (Java Desktop, KDE)





Bourne Shell Command Interpreter

```
Terminal
File Edit View Terminal Tabs Help
fd0      0.0    0.0    0.0    0.0  0.0  0.0    0.0  0  0
sd0      0.0    0.2    0.0    0.2  0.0  0.0    0.4  0  0
sd1      0.0    0.0    0.0    0.0  0.0  0.0    0.0  0  0
          extended device statistics
device   r/s    w/s    kr/s    kw/s wait actv  svc_t  %w  %b
fd0      0.0    0.0    0.0    0.0  0.0  0.0    0.0  0  0
sd0      0.6    0.0   38.4    0.0  0.0  0.0    8.2  0  0
sd1      0.0    0.0    0.0    0.0  0.0  0.0    0.0  0  0
(root@pbg-nv64-vn)-(11/pts)-(00:53 15-Jun-2007)-(global)
- (/var/tmp/system-contents/scripts)# swap -sh
total: 1.1G allocated + 190M reserved = 1.3G used, 1.6G available
(root@pbg-nv64-vn)-(12/pts)-(00:53 15-Jun-2007)-(global)
- (/var/tmp/system-contents/scripts)# uptime
12:53am up 9 min(s), 3 users, load average: 33.29, 67.68, 36.81
(root@pbg-nv64-vn)-(13/pts)-(00:53 15-Jun-2007)-(global)
- (/var/tmp/system-contents/scripts)# w
4:07pm up 17 day(s), 15:24, 3 users, load average: 0.09, 0.11, 8.66
User      tty          login@ idle   JCPU   PCPU   what
root      console      15Jun07 18days    1      /usr/bin/ssh-agent -- /usr/bi
n/d
root      pts/3        15Jun07    18      4   w
root      pts/4        15Jun07 18days    w
(root@pbg-nv64-vn)-(14/pts)-(16:07 02-Jul-2007)-(global)
- (/var/tmp/system-contents/scripts)#
```





The Mac OS X GUI





System Calls

- Programming interface to the services provided by the OS
- Typically written in a high-level language (C or C++)
- Mostly accessed by programs via a high-level [Application Program Interface \(API\)](#) rather than direct system call use
- Three most common APIs are Win32 API for Windows, POSIX API for POSIX-based systems (including virtually all versions of UNIX, Linux, and Mac OS X), and Java API for the Java virtual machine (JVM)
- Why use APIs rather than system calls?

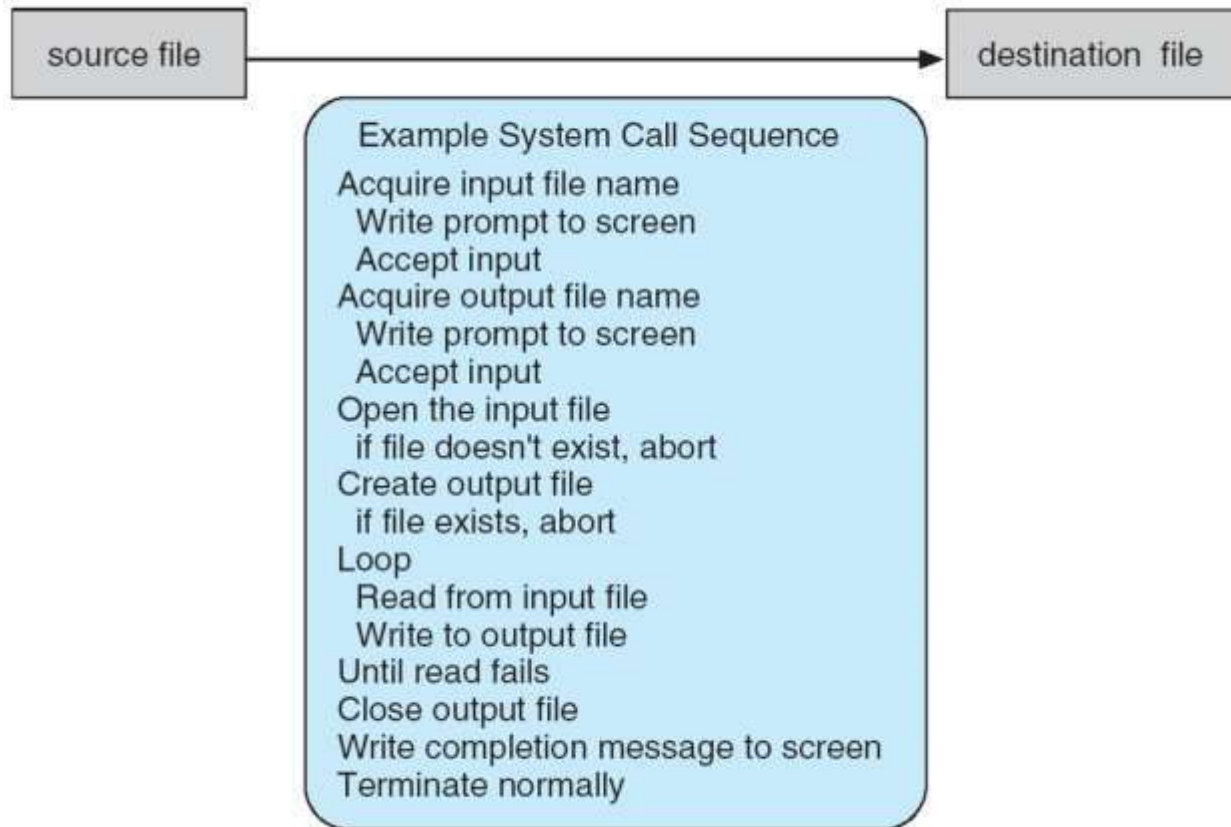
(Note that the system-call names used throughout this text are generic)





Example of System Calls

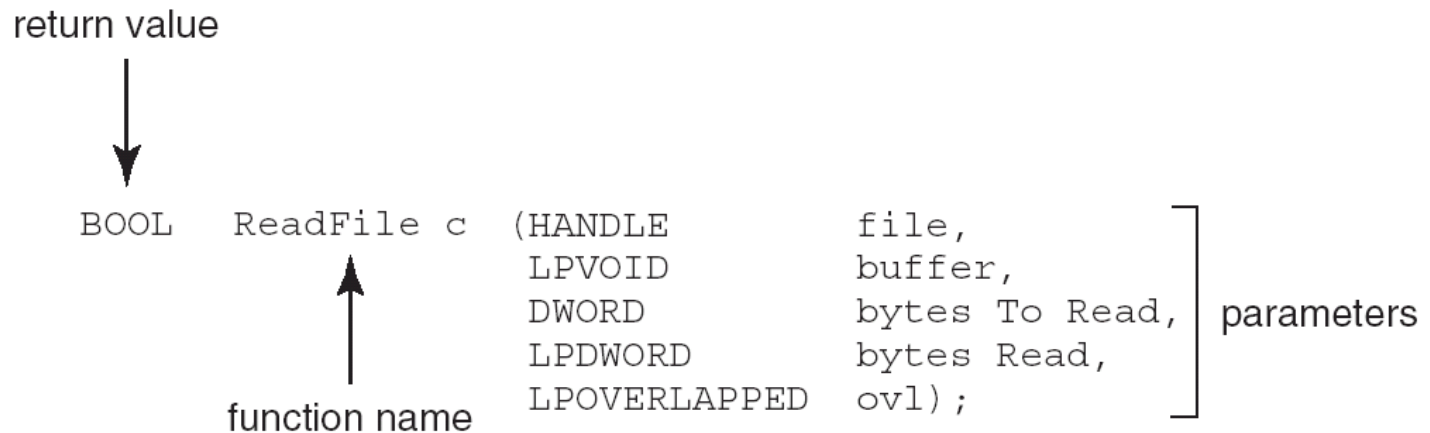
- System call sequence to copy the contents of one file to another file





Example of Standard API

- Consider the ReadFile() function in the
- Win32 API—a function for reading from a file



- A description of the parameters passed to ReadFile()
 - HANDLE file—the file to be read
 - LPVOID buffer—a buffer where the data will be read into and written from
 - DWORD bytesToRead—the number of bytes to be read into the buffer
 - LPDWORD bytesRead—the number of bytes read during the last read
 - LPOVERLAPPED ovl—indicates if overlapped I/O is being used





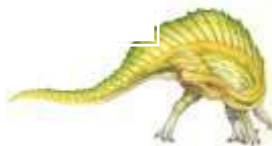
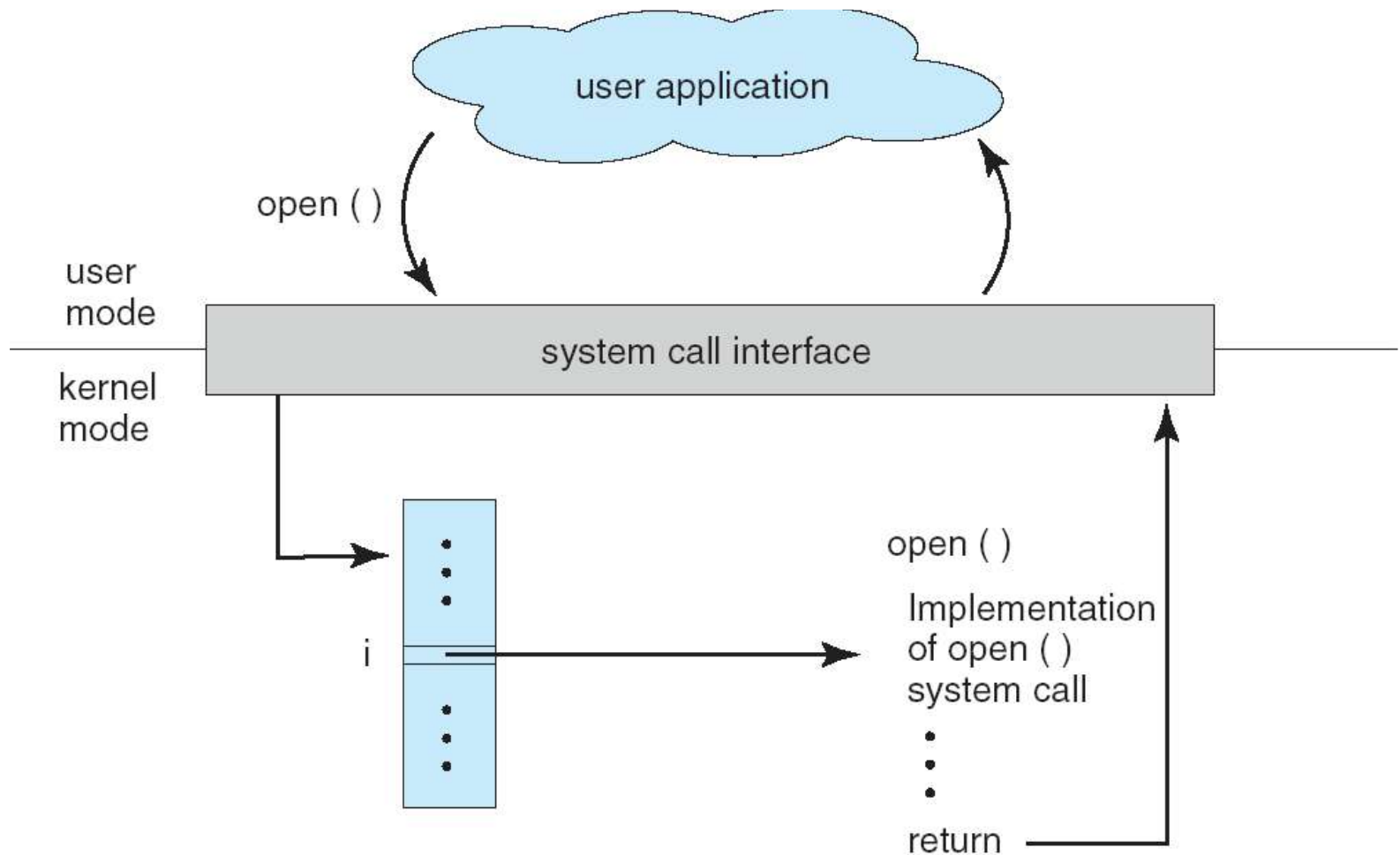
System Call Implementation

- Typically, a number associated with each system call
 - System-call interface maintains a table indexed according to these numbers
- The system call interface invokes intended system call in OS kernel and returns status of the system call and any return values
- The caller need know nothing about how the system call is implemented
 - Just needs to obey API and understand what OS will do as a result call
 - Most details of OS interface hidden from programmer by API
 - ▶ Managed by run-time support library (set of functions built into libraries included with compiler)





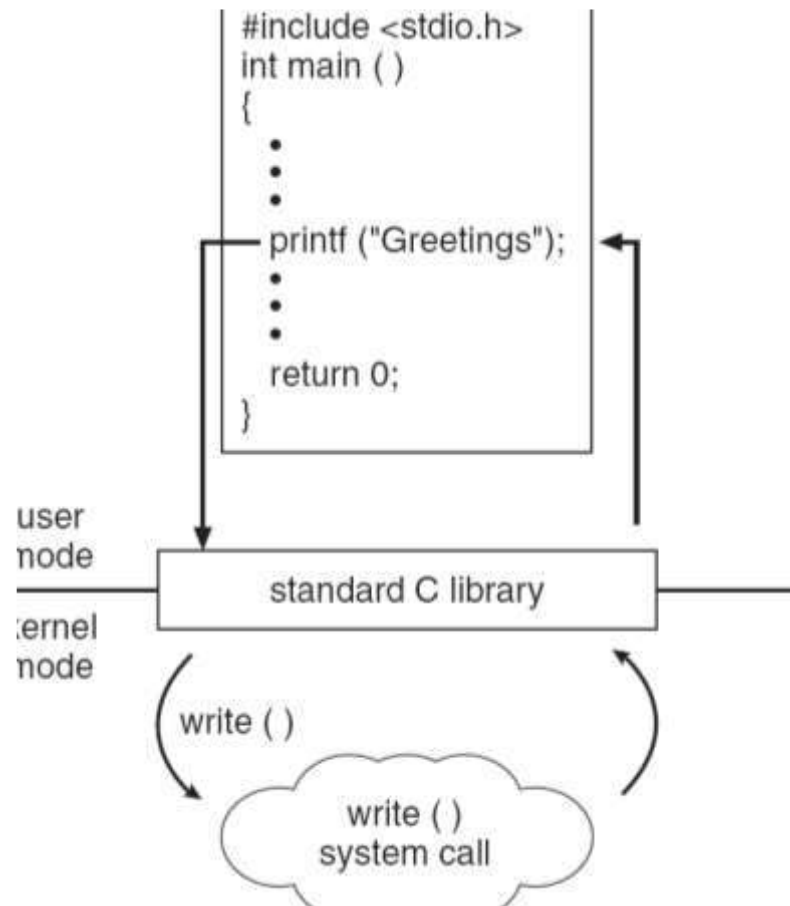
API – System Call – OS Relationship





Standard C Library Example

- C program invoking printf() library call, which calls write() system call





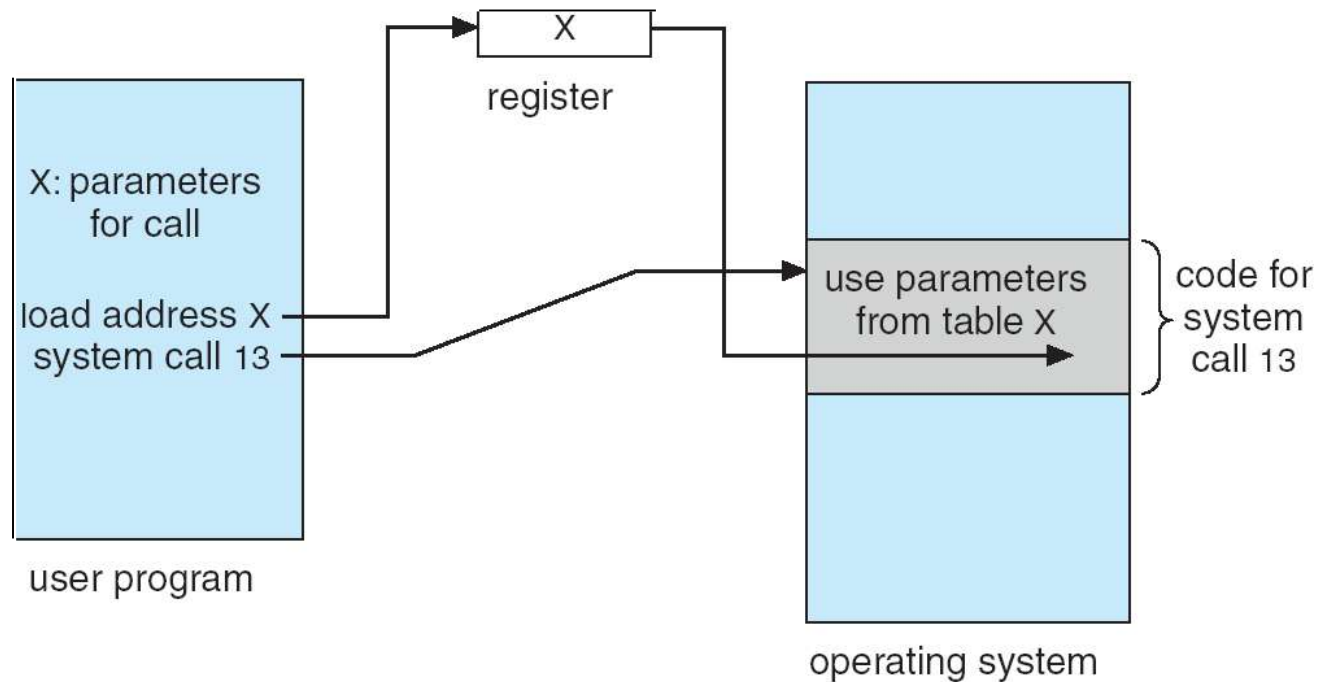
System Call Parameter Passing

- Often, more information is required than simply identity of desired system call
 - Exact type and amount of information vary according to OS and call
- Three general methods used to pass parameters to the OS
 - Simplest: pass the parameters in *registers*
 - ▶ In some cases, may be more parameters than registers
 - Parameters stored in a *block*, or table, in memory, and address of block passed as a parameter in a register
 - ▶ This approach taken by Linux and Solaris
 - Parameters placed, or *pushed*, onto the *stack* by the program and *popped* off the stack by the operating system
 - Block and stack methods do not limit the number or length of parameters being passed





Parameter Passing via Table





Types of System Calls

- Process control
- File management
- Device management
- Information maintenance
- Communications
- Protection





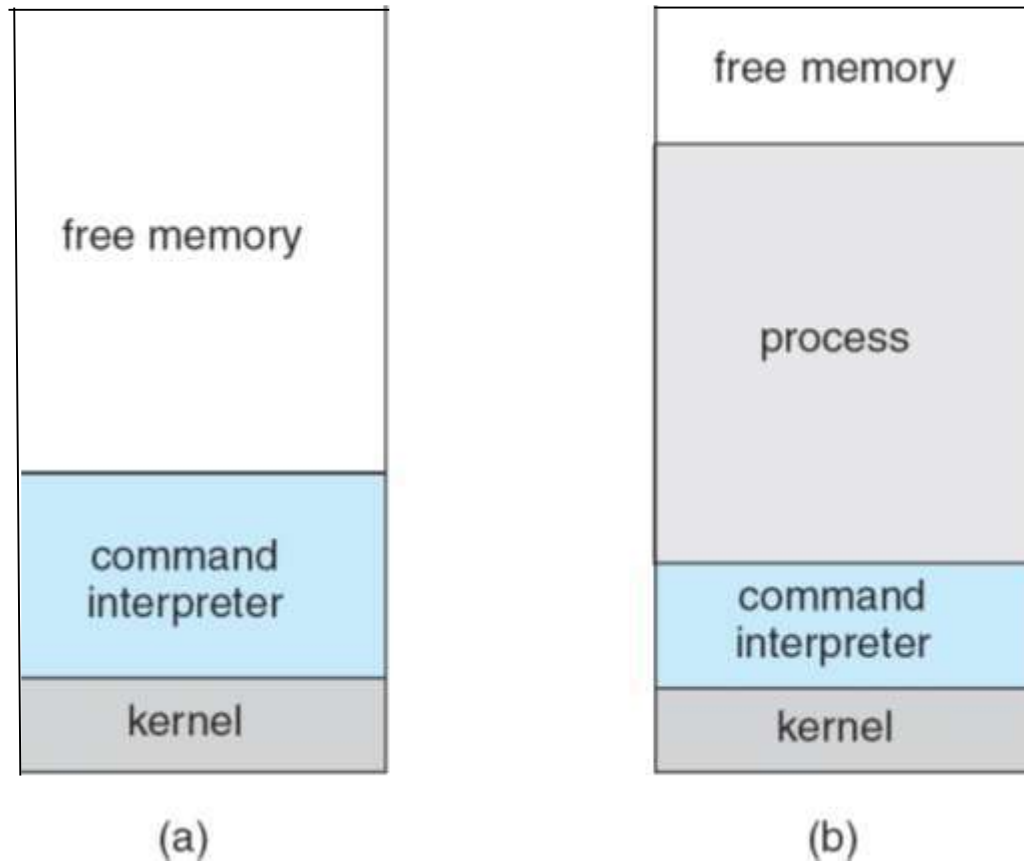
Examples of Windows and Unix System Calls

	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shmget() mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()





MS-DOS execution

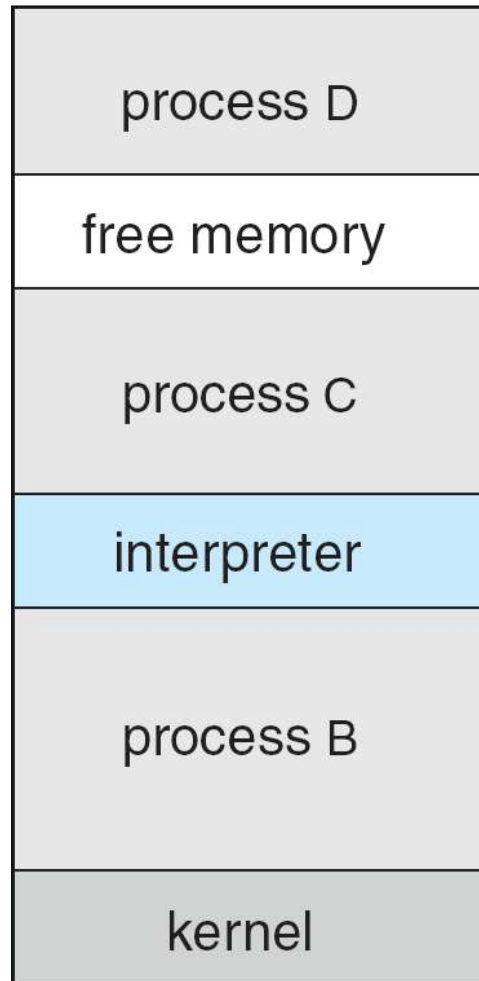


(a) At system startup (b) running a program





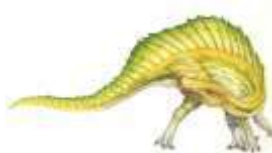
FreeBSD Running Multiple Programs





System Programs

- System programs provide a convenient environment for program development and execution. They can be divided into:
 - File manipulation
 - Status information
 - File modification
 - Programming language support
 - Program loading and execution
 - Communications
 - Application programs
- Most users' view of the operation system is defined by system programs, not the actual system calls





System Programs

- Provide a convenient environment for program development and execution
 - Some of them are simply user interfaces to system calls; others are considerably more complex
- File management - Create, delete, copy, rename, print, dump, list, and generally manipulate files and directories
- Status information
 - Some ask the system for info - date, time, amount of available memory, disk space, number of users
 - Others provide detailed performance, logging, and debugging information
 - Typically, these programs format and print the output to the terminal or other output devices
 - Some systems implement a registry - used to store and retrieve configuration information





System Programs (cont'd)

- File modification
 - Text editors to create and modify files
 - Special commands to search contents of files or perform transformations of the text
- Programming-language support - Compilers, assemblers, debuggers and interpreters sometimes provided
- Program loading and execution- Absolute loaders, relocatable loaders, linkage editors, and overlay-loaders, debugging systems for higher-level and machine language
- Communications - Provide the mechanism for creating virtual connections among processes, users, and computer systems
 - Allow users to send messages to one another's screens, browse web pages, send electronic-mail messages, log in remotely, transfer files from one machine to another





Operating System Design and Implementation

- Design and Implementation of OS not “solvable”, but some approaches have proven successful
- Internal structure of different Operating Systems can vary widely
- Start by defining goals and specifications
- Affected by choice of hardware, type of system
- *User goals and System goals*
 - User goals – operating system should be convenient to use, easy to learn, reliable, safe, and fast
 - System goals – operating system should be easy to design, implement, and maintain, as well as flexible, reliable, error-free, and efficient





Operating System Design and Implementation (Cont)

- Important principle to separate

Policy: What will be done?

Mechanism: How to do it?

- Mechanisms determine how to do something, policies decide what will be done
 - The separation of policy from mechanism is a very important principle, it allows maximum flexibility if policy decisions are to be changed later





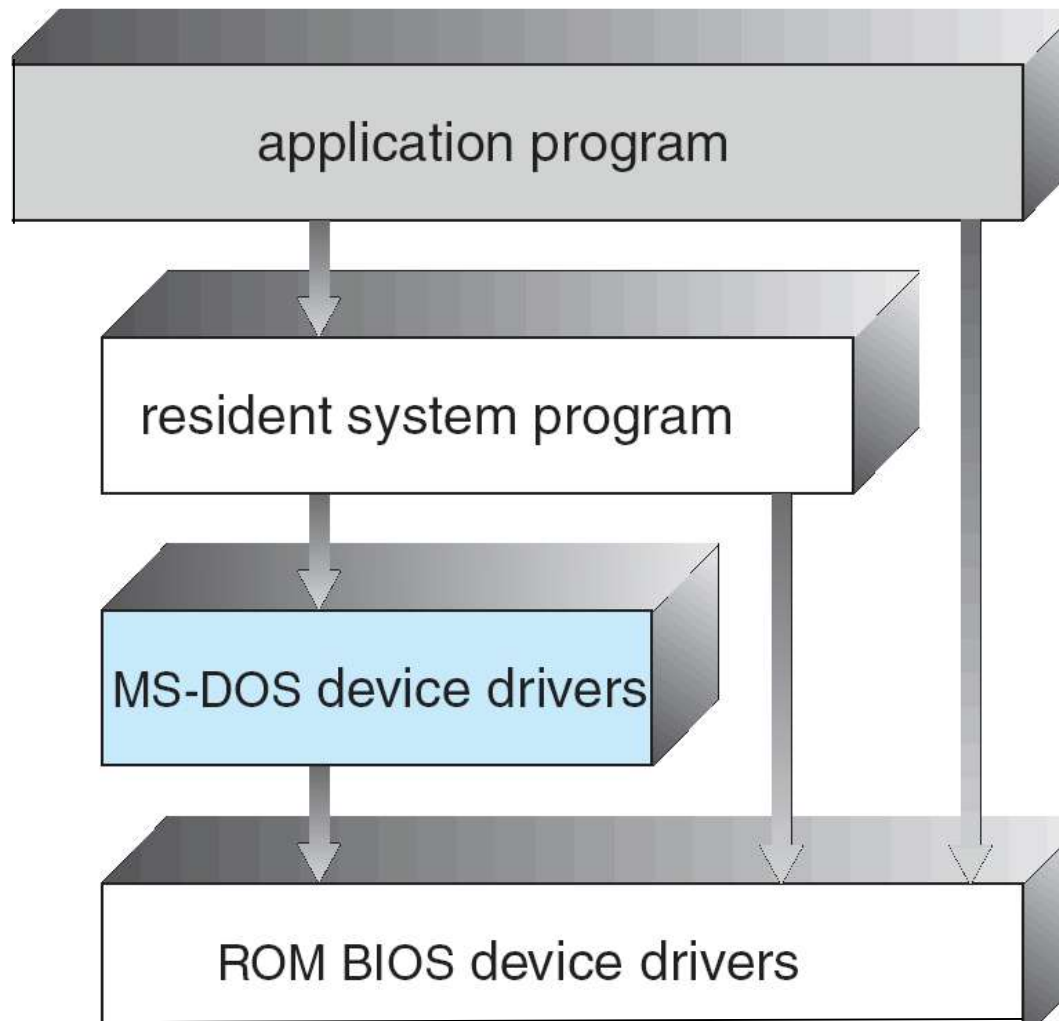
Simple Structure

- MS-DOS – written to provide the most functionality in the least space
 - Not divided into modules
 - Although MS-DOS has some structure, its interfaces and levels of functionality are not well separated





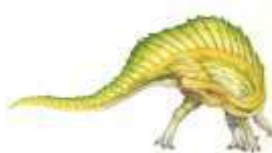
MS-DOS Layer Structure





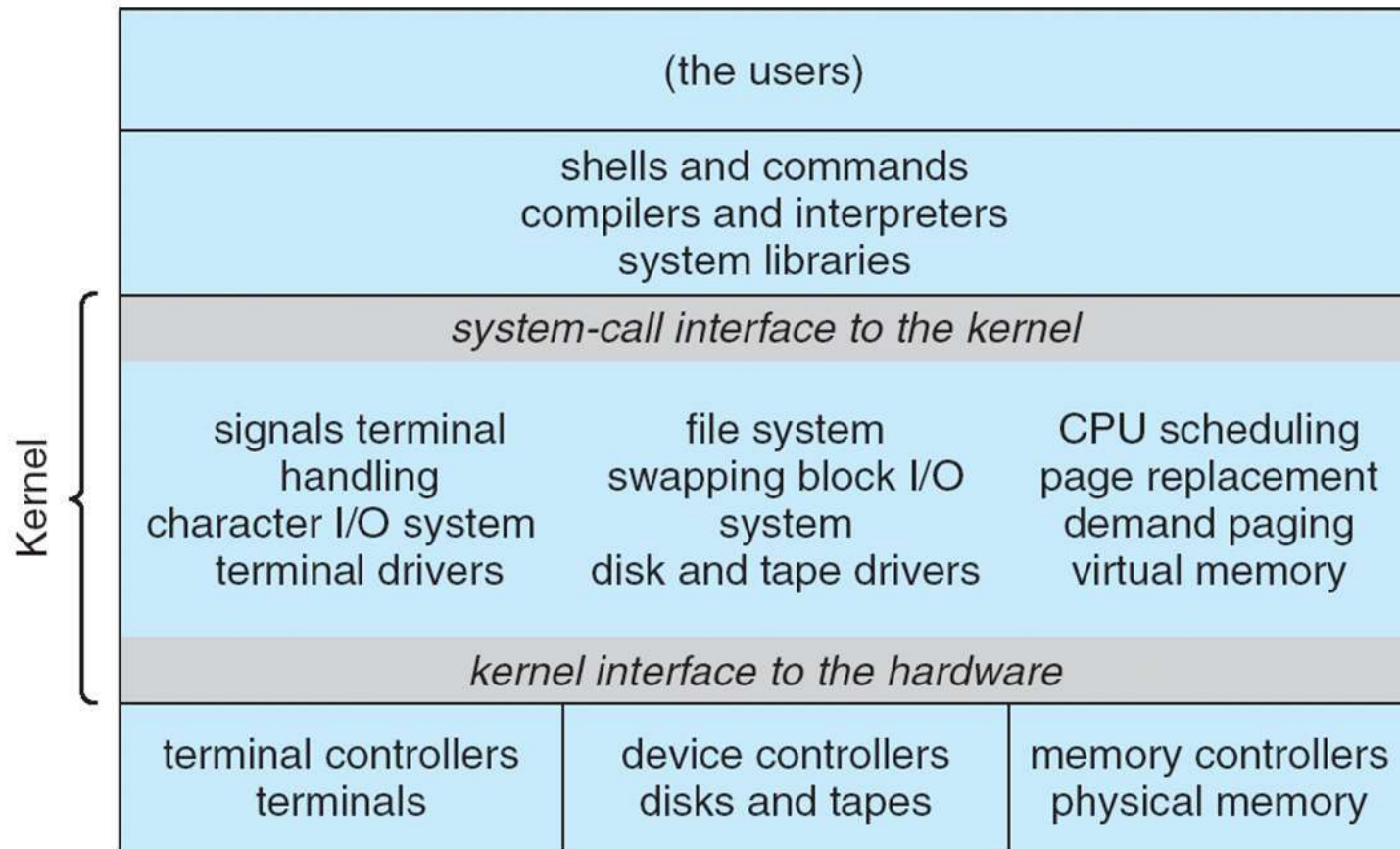
Layered Approach

- The operating system is divided into a number of layers (levels), each built on top of lower layers. The bottom layer (layer 0), is the hardware; the highest (layer N) is the user interface.
- With modularity, layers are selected such that each uses functions (operations) and services of only lower-level layers





Traditional UNIX System Structure





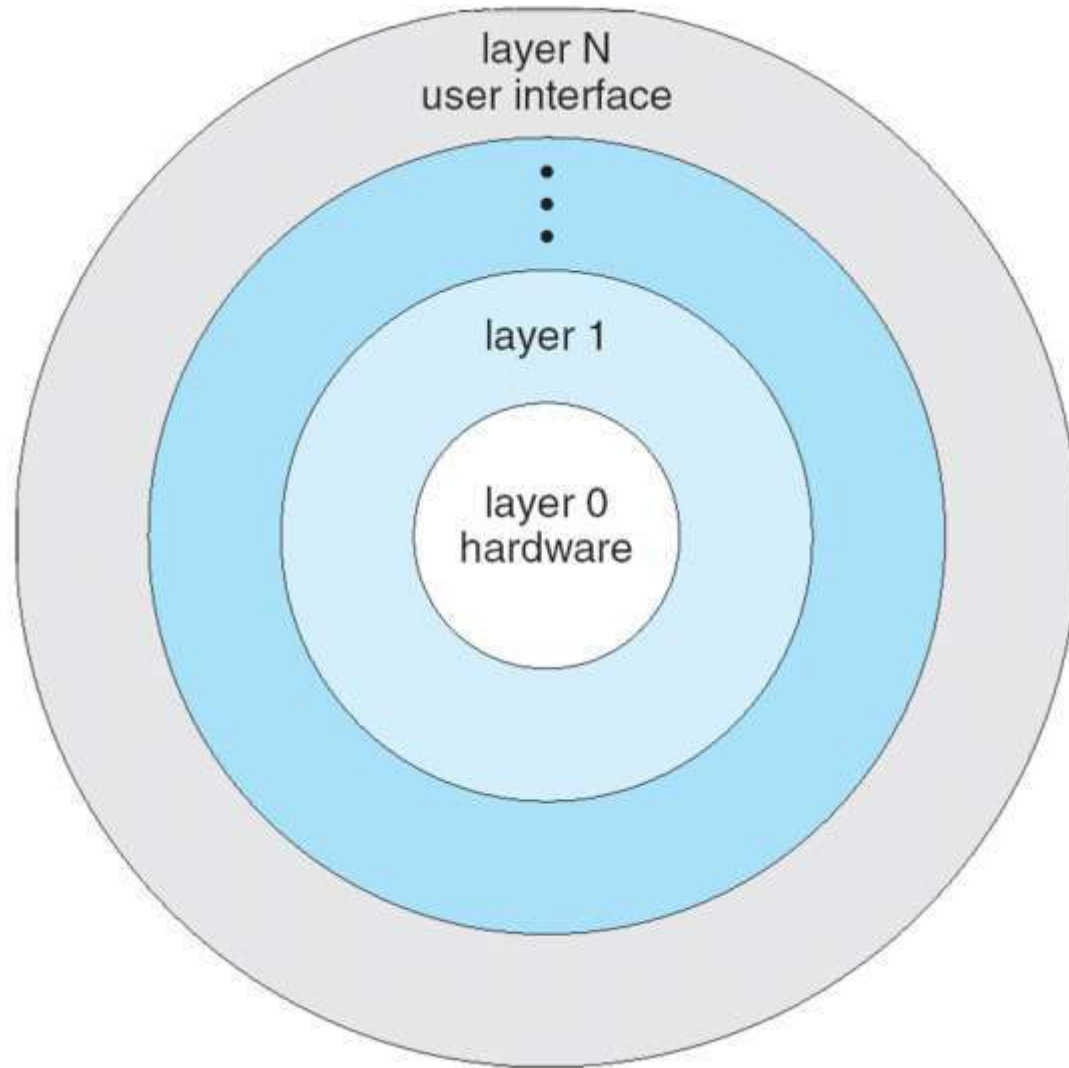
UNIX

- UNIX – limited by hardware functionality, the original UNIX operating system had limited structuring. The UNIX OS consists of two separable parts
 - Systems programs
 - The kernel
 - ▶ Consists of everything below the system-call interface and above the physical hardware
 - ▶ Provides the file system, CPU scheduling, memory management, and other operating-system functions; a large number of functions for one level





Layered Operating System





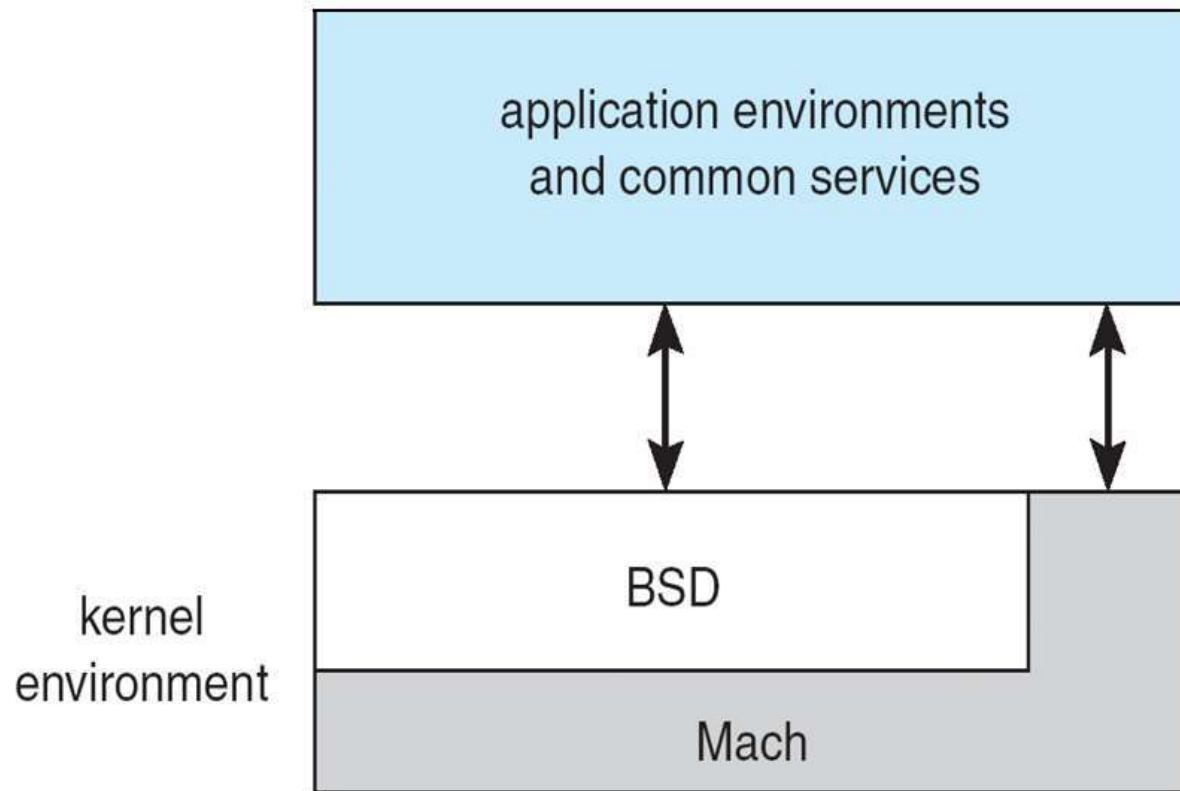
Microkernel System Structure

- Moves as much from the kernel into “*user*” space
- Communication takes place between user modules using message passing
- Benefits:
 - Easier to extend a microkernel
 - Easier to port the operating system to new architectures
 - More reliable (less code is running in kernel mode)
 - More secure
- Detriments:
 - Performance overhead of user space to kernel space communication





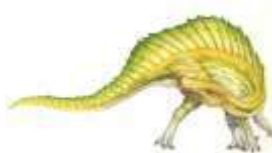
Mac OS X Structure





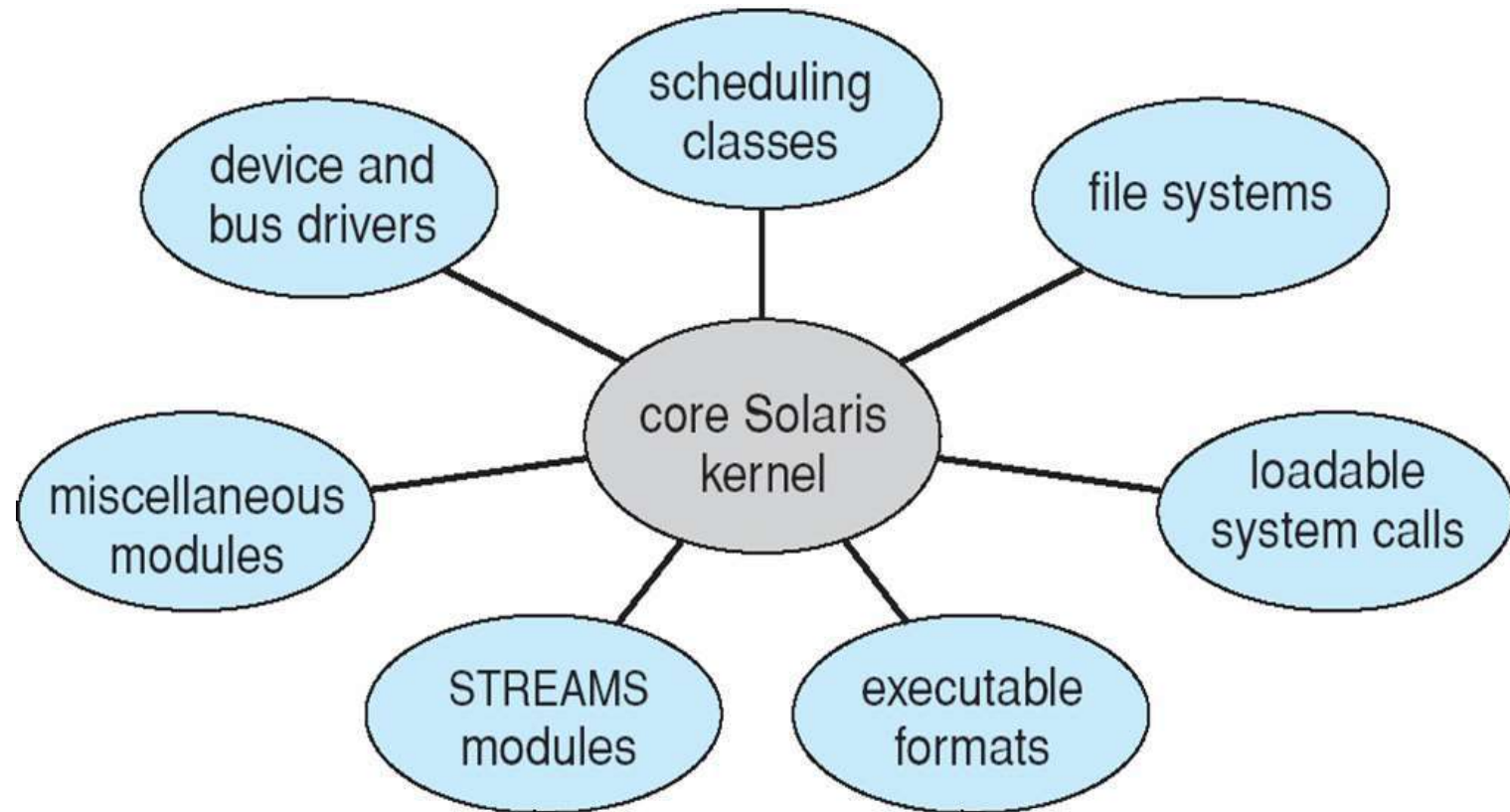
Modules

- Most modern operating systems implement kernel modules
 - Uses object-oriented approach
 - Each core component is separate
 - Each talks to the others over known interfaces
 - Each is loadable as needed within the kernel
- Overall, similar to layers but with more flexible





Solaris Modular Approach





Virtual Machines

- A **virtual machine** takes the layered approach to its logical conclusion. It treats hardware and the operating system kernel as though they were all hardware
- A virtual machine provides an interface *identical* to the underlying bare hardware
- The operating system **host** creates the illusion that a process has its own processor and (virtual memory)
- Each **guest** provided with a (virtual) copy of underlying computer





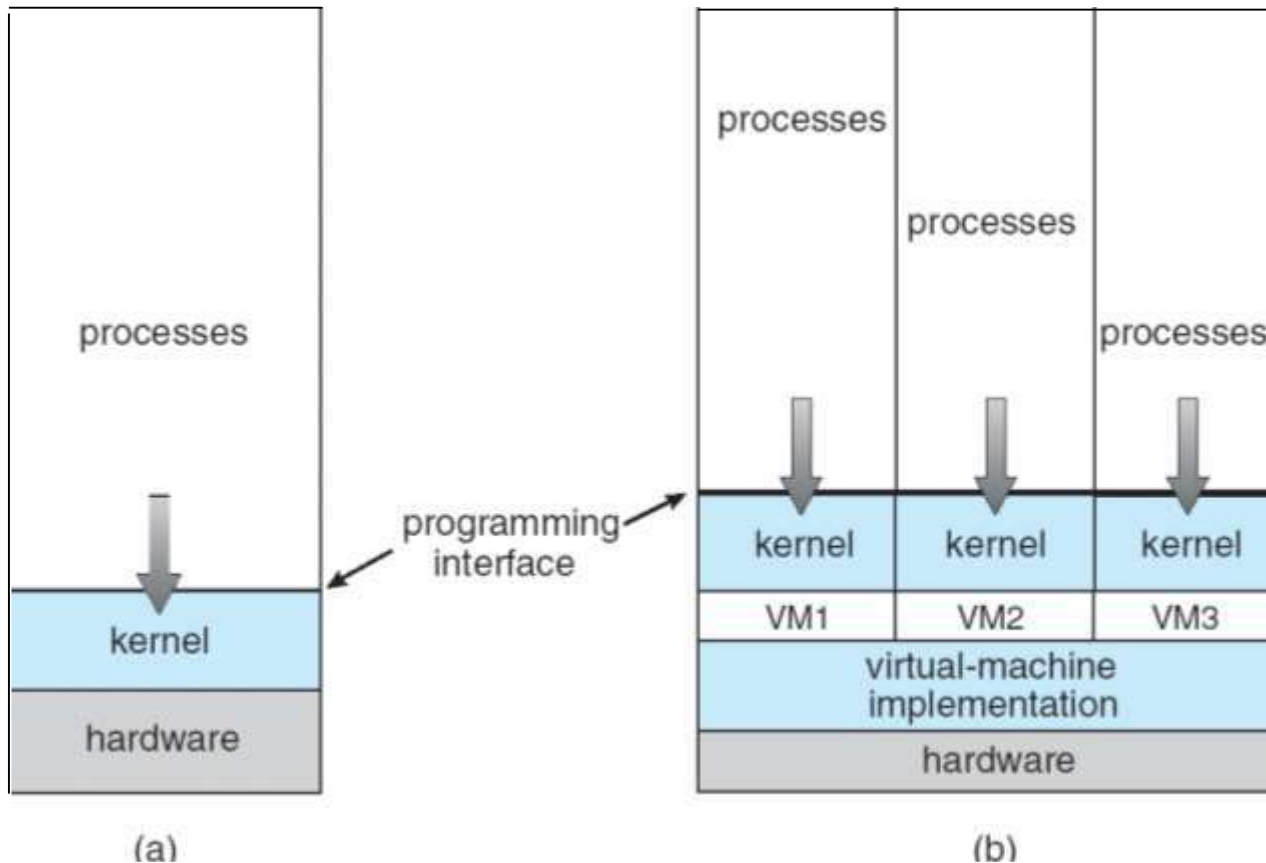
Virtual Machines History and Benefits

- First appeared commercially in IBM mainframes in 1972
- Fundamentally, multiple execution environments (different operating systems) can share the same hardware
- Protect from each other
- Some sharing of file can be permitted, controlled
- Commutate with each other, other physical systems via networking
- Useful for development, testing
- **Consolidation** of many low-resource use systems onto fewer busier systems
- “Open Virtual Machine Format”, standard format of virtual machines, allows a VM to run within many different virtual machine (host) platforms





Virtual Machines (Cont)



(a) Nonvirtual machine (b) virtual machine





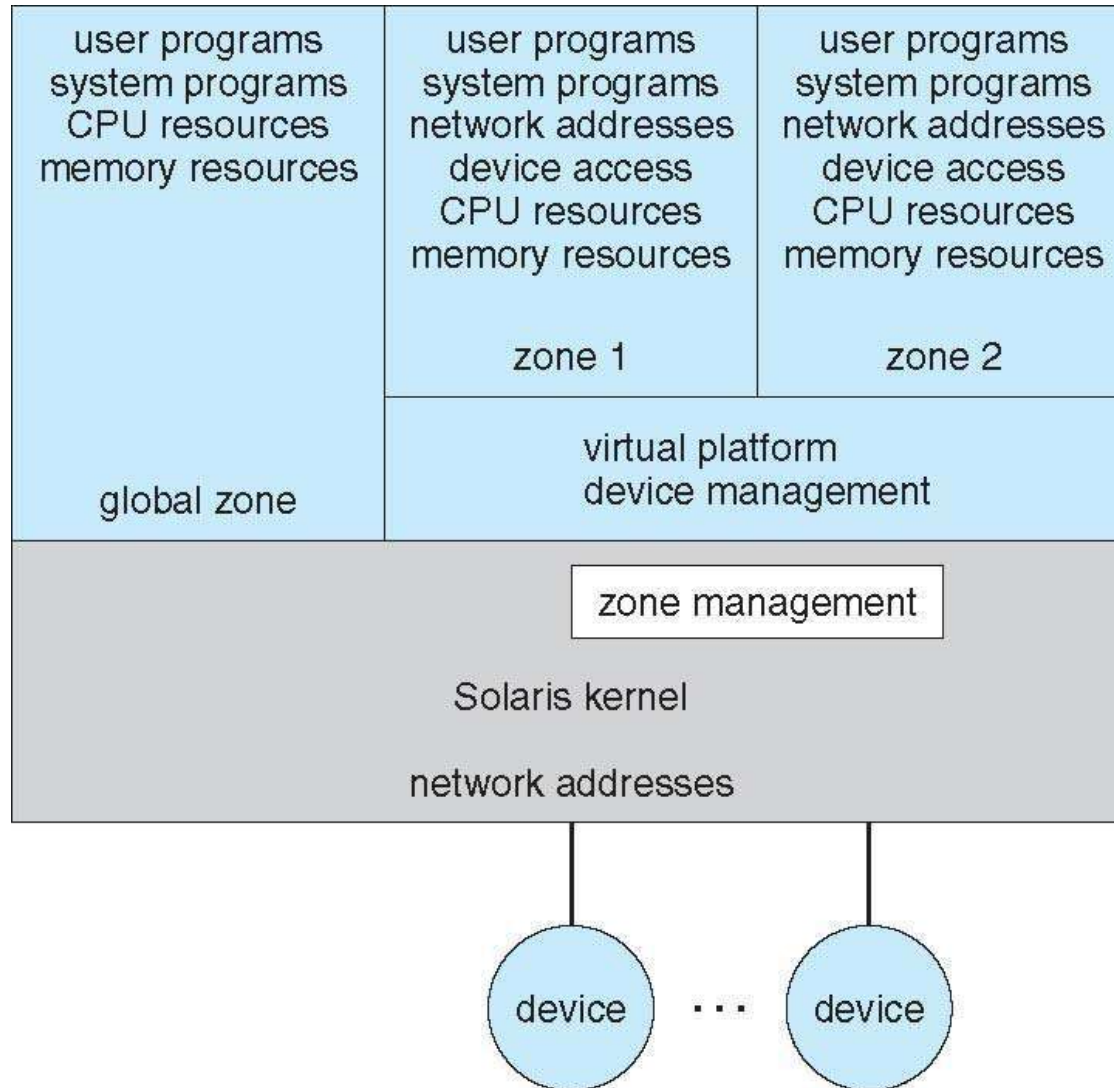
Para-virtualization

- Presents guest with system similar but not identical to hardware
- Guest must be modified to run on paravirtualized hardware
- Guest can be an OS, or in the case of Solaris 10 applications running in containers



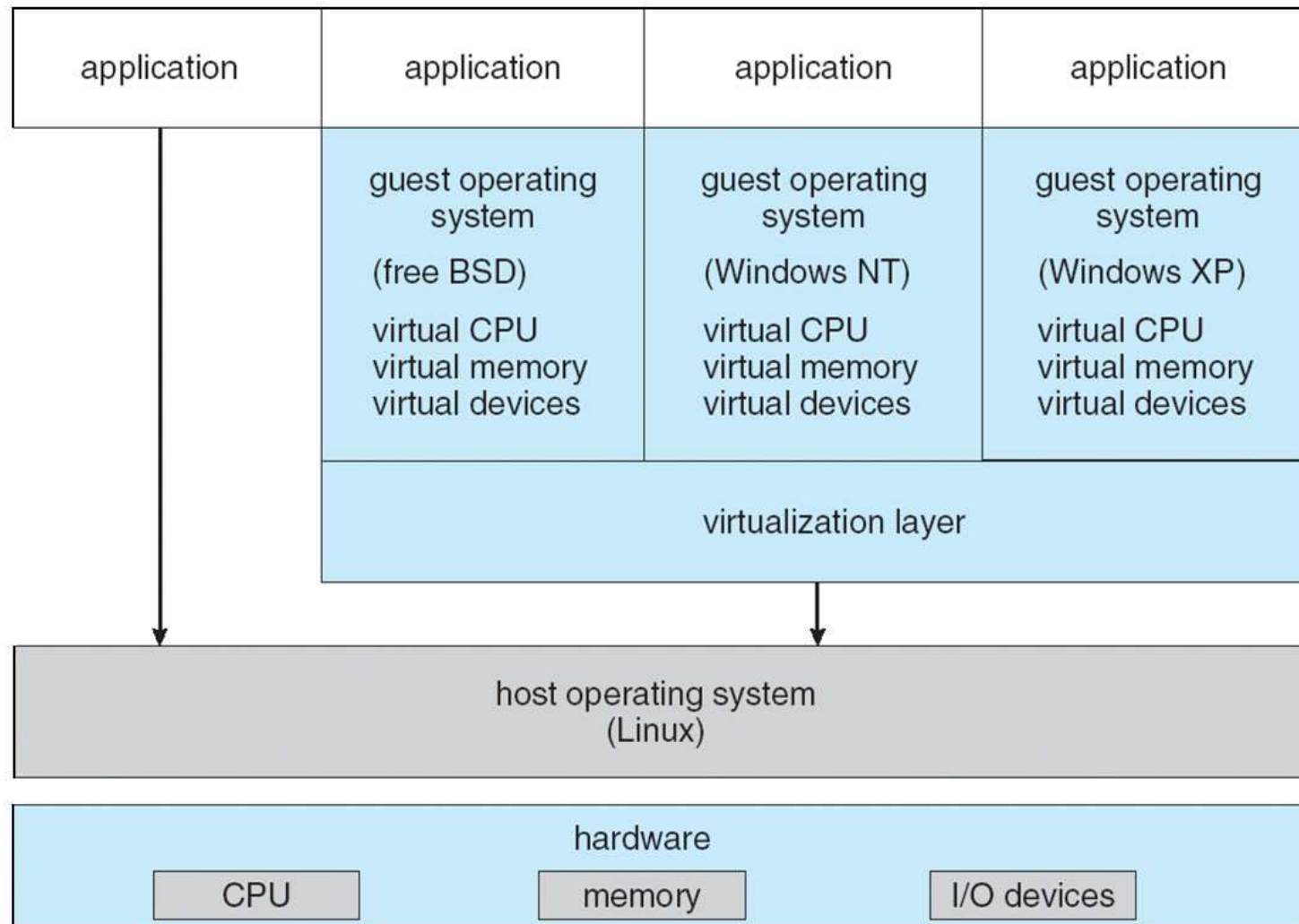


Solaris 10 with Two Containers



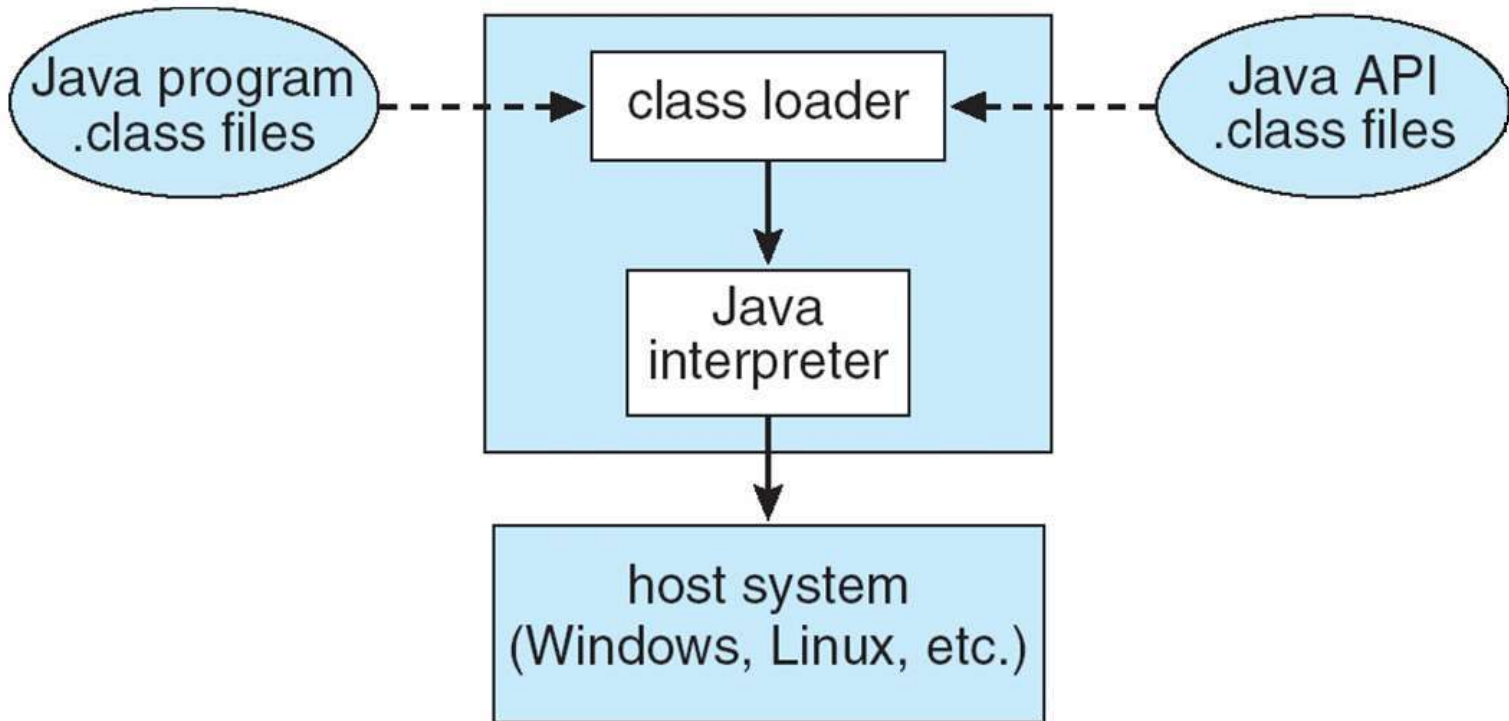


VMware Architecture





The Java Virtual Machine





Operating-System Debugging

- **Debugging** is finding and fixing errors, or **bugs**
- OSes generate **log files** containing error information
- Failure of an application can generate **core dump** file capturing memory of the process
- Operating system failure can generate **crash dump** file containing kernel memory
- Beyond crashes, performance tuning can optimize system performance
- Kernighan's Law: "Debugging is twice as hard as writing the code in the first place. Therefore, if you write the code as cleverly as possible, you are, by definition, not smart enough to debug it."
- DTrace tool in Solaris, FreeBSD, Mac OS X allows live instrumentation on production systems
 - **Probes** fire when code is executed, capturing state data and sending it to consumers of those probes





Solaris 10 dtrace Following System Call

```
# ./all.d 'pgrep xclock' XEventsQueued
dtrace: script './all.d' matched 52377 probes
CPU FUNCTION
0 -> XEventsQueued U
0 -> _XEventsQueued U
0 -> _X11TransBytesReadable U
0 <- _X11TransBytesReadable U
0 -> _X11TransSocketBytesReadable U
0 <- _X11TransSocketBytesreadable U
0 -> ioctl U
0 -> ioctl K
0 -> getf K
0 -> set_active_fd K
0 <- set_active_fd K
0 <- getf K
0 -> get_udatamodel K
0 <- get_udatamodel K
...
0 -> releasef K
0 -> clear_active_fd K
0 <- clear_active_fd K
0 -> cv_broadcast K
0 <- cv_broadcast K
0 <- releasef K
0 <- ioctl K
0 <- ioctl U
0 <- _XEventsQueued U
0 <- XEventsQueued U
```





Operating System Generation

- Operating systems are designed to run on any of a class of machines; the system must be configured for each specific computer site
- SYSGEN program obtains information concerning the specific configuration of the hardware system
- *Booting* – starting a computer by loading the kernel
- *Bootstrap program* – code stored in ROM that is able to locate the kernel, load it into memory, and start its execution



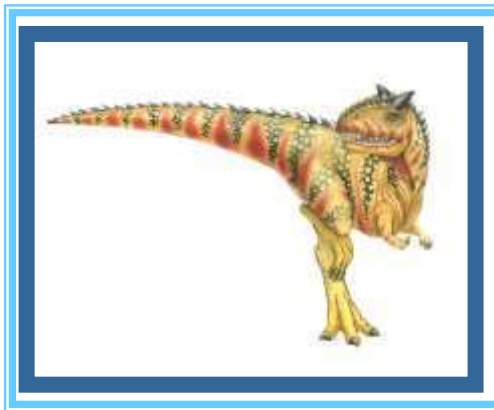


System Boot

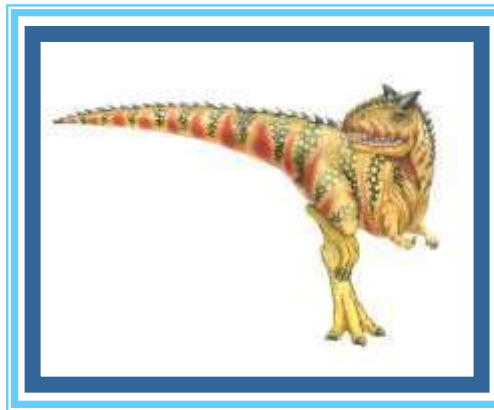
- Operating system must be made available to hardware so hardware can start it
 - Small piece of code – **bootstrap loader**, locates the kernel, loads it into memory, and starts it
 - Sometimes two-step process where **boot block** at fixed location loads bootstrap loader
 - When power initialized on system, execution starts at a fixed memory location
 - ▶ Firmware used to hold initial boot code



End of Chapter 2



Week 2: OS CT-353





- **[Bit]**

- Definition: Smallest unit of storage
- Values: 0 or 1

- **[Collections of Bits]**

- Used to represent:
- Numbers, letters, images, movies,
- sounds, documents, programs

- **[Byte]**

- Consists of: 8 bits
- Role: Smallest convenient storage unit
- Reason: Computers move bytes, not individual bits

- **[Word]**

Definition: Native data unit of a computer architecture

Size: One or more bytes

Example: 64-bit system → 64-bit (8-byte) word

- **[Computer Operations]**

Computers execute many operations

Operations occur in native word size

Not typically byte-by-byte





- Computer storage and most computer processing are measured in **bytes** and groups of bytes.
- **Binary-based storage units (used by computers):**
 - **Kilobyte (KB)** = 1,024 bytes
 - **Megabyte (MB)** = 1,024² bytes
 - **Gigabyte (GB)** = 1,024³ bytes
 - **Terabyte (TB)** = 1,024⁴ bytes
 - **Petabyte (PB)** = 1,024⁵ bytes
- These values are based on powers of **2**, because computers operate using binary (bits of 0s and 1s).
- **Manufacturer rounding (marketing usage):**
 - Manufacturers often simplify these numbers:
 - **1 MB ≈ 1,000,000 bytes**
 - **1 GB ≈ 1,000,000,000 bytes**





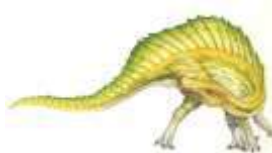
- This makes storage sizes easier to understand, but it can make actual capacity appear smaller when viewed by the operating system.
- **Important exception — Networking:**
- **Network speeds are measured in bits, not bytes**
 - Example: Mbps (megabits per second)
- This is because networks transmit data **one bit at a time**, not one byte at a time.

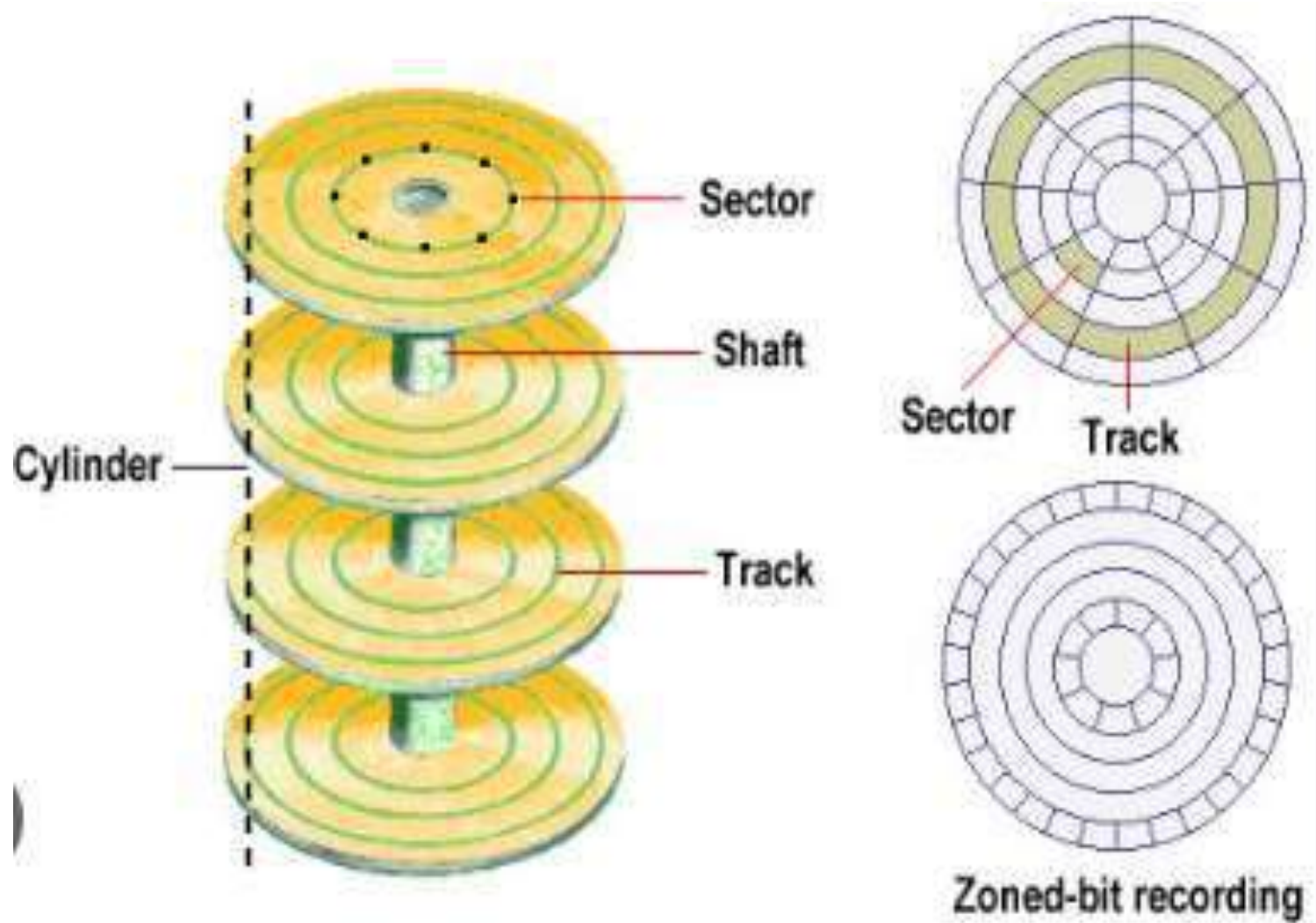




Storage Structure

- Main memory – only large storage media that the CPU can access directly
 - **Random access**
 - Typically **volatile**
- Secondary storage – extension of main memory that provides large **nonvolatile** storage capacity
- Hard disks – rigid metal or glass platters covered with magnetic recording material
 - Disk surface is logically divided into **tracks**, which are subdivided into **sectors**
 - The **disk controller** determines the logical interaction between the device and the computer
- **Solid-state disks** – faster than hard disks, nonvolatile
 - Various technologies
 - Becoming more popular

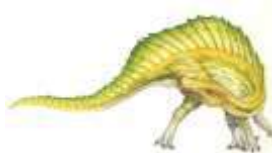






Storage Hierarchy

- Storage systems organized in hierarchy
 - Speed
 - Cost
 - Volatility
- **Caching** – copying information into faster storage system; main memory can be viewed as a cache for secondary storage





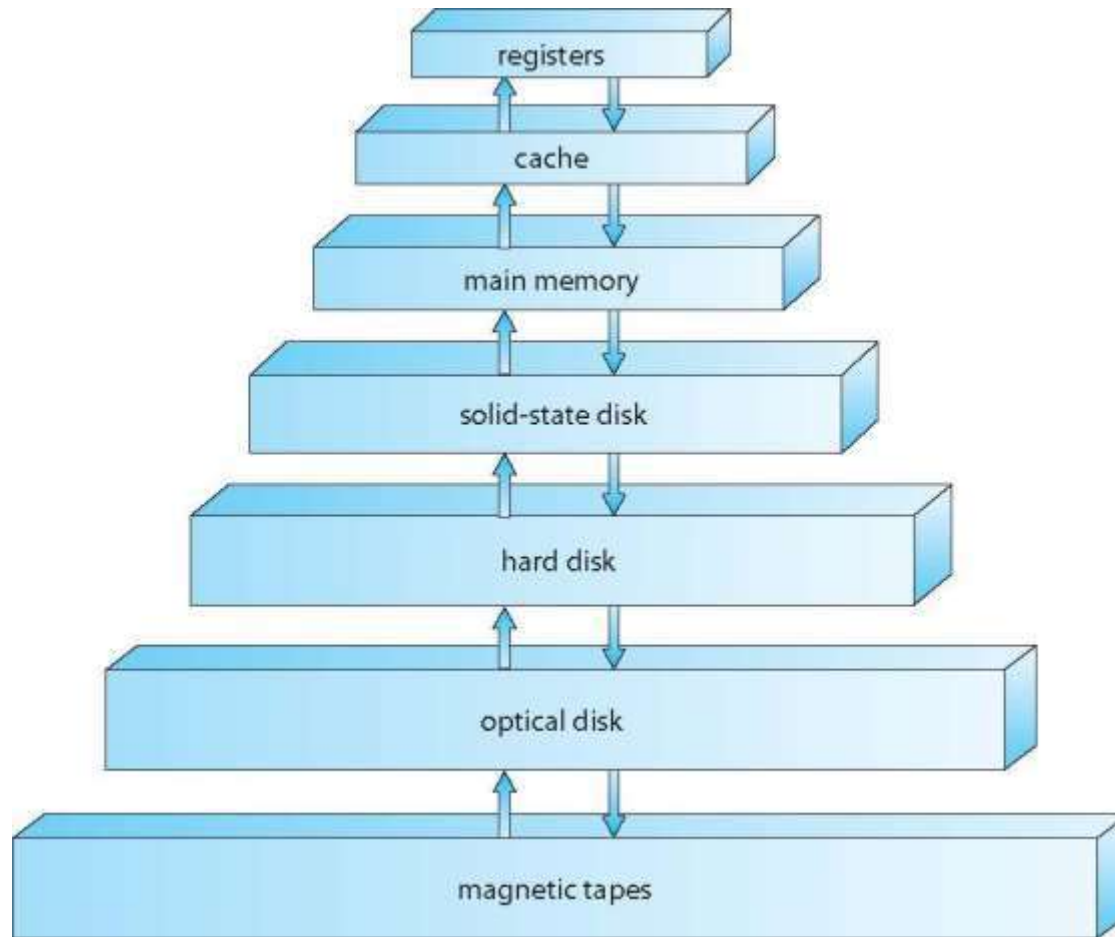
Device Drivers and I/O Management

- A **device driver** is a specialized piece of software written for **each device controller** (such as a disk controller, keyboard controller, or network card). Its job is to **manage input and output (I/O) operations** for that specific hardware device.
- Instead of the operating system kernel needing to understand the unique details of every hardware device, the **device driver acts as an intermediary** between the **hardware controller** and the **kernel**.
- **Key Roles of a Device Driver:**
 - Translates **generic OS requests** (e.g., “read data,” “write data”) into **device-specific commands**
 - Handles **low-level hardware details** such as registers, interrupts, and data transfer
 - Manages communication during I/O operations
- **Uniform Interface:**
 - Device drivers provide a **standard, uniform interface** to the kernel.
This means:
 - The kernel interacts with **all devices in the same way**
 - Hardware differences are hidden from the kernel
 - New devices can be added without changing kernel code—only the driver is needed





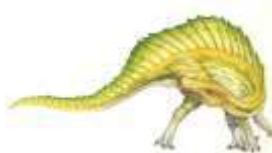
Storage-Device Hierarchy





Caching

- Important principle, performed at many levels in a computer (in hardware, operating system, software)
- Information in use copied from **slower to faster** storage temporarily
- Faster storage (cache) checked first to determine if information is there
 - If it is, information **used directly** from the cache (fast)
 - If not, data copied to cache and used there
- Cache smaller than storage being cached
 - Cache management important design problem
 - Cache size and replacement policy





Caching

A cache is a **small, fast memory** that stores copies of data from a larger, slower storage system (such as main memory or disk).

- **1. Cache is Smaller Than the Storage Being Cached**
- A cache cannot hold all the data from the main storage
- It only keeps the most frequently or recently used data
- This is done to improve performance by reducing access time
- **Example:**
- CPU cache (very small, very fast) stores copies of data from RAM (much larger, slower).
- **2. Cache Management Is an Important Design Problem because the cache is limited in size:**
- The system must decide what data to keep
- It must decide when to remove old data
- Poor cache management can reduce performance or waste cache space





Efficient cache management is critical for:

- Faster program execution
- Better use of hardware resources
- Reduced latency
- **3. Cache Size and Replacement Policy**
- Two key design factors determine cache effectiveness:
- Cache Size
- Larger cache → higher chance needed data is present (cache hit)
- Smaller cache → faster access but more cache misses
- Designers must balance speed, cost, and space
- Replacement Policy
- Determines which data to remove when the cache is full





- Common policies include:
 - LRU (Least Recently Used) — removes data not used recently
 - FIFO (First In, First Out) — removes oldest data
 - MRU
 - Optimal
 - Random — removes a random item
 - In nutshell:
 - A cache improves performance, but because it is “small”, careful “management, sizing, and replacement strategies” are essential to make it effective.





What is Page Replacement?

When a page fault occurs and all page frames are full, the operating system must decide **which page to remove** from memory.

- That decision is made using a page replacement algorithm.

1. FIFO (First In, First Out)

- Removes the **oldest page** in memory
- Pages are replaced in the order they entered memory
- Simple to implement, but **not always efficient**
- “The page that came first leaves first.”
- **Pros / Cons**
- Easy to understand
- May remove frequently used pages
- Suffers from ****Belady’s Anomaly**** (more frames can cause more faults)





2. LRU (Least Recently Used)

- Removes the page that **has not been used for the longest time**
- Based on **recent past behavior**
- “If it wasn’t used recently, it probably won’t be used soon.”
- **Pros / Cons**
- Good performance
- More complex to implement (needs tracking)

3. MRU (Most Recently Used)

- Removes the page that **was used most recently**
- Assumes recently used pages are **less likely** to be used again soon
- “The most recently used page leaves.”
- When It’s Useful
- In **sequential access patterns**
- Less common than LRU





- 4. Optimal Page Replacement
- Removes the page that **will not be used for the longest time in the future**
- Requires knowing the **future reference string**
- “Replace the page whose next use is farthest away.”
- **Pros / Cons**
- Produces **minimum page faults**
- Not possible in real systems (used for comparison only)





Questions for Practice

Question

Reference String:

7 0 1 2 0 3 0 4

Number of Frames: 3

Perform dry run using:

- **FIFO**
- **LRU**
- **Optimal**

Question (Compare Algorithms)

Reference String:

1 3 0 3 5 6 3

Frames: 3

Find:

- Page faults for FIFO
- Page faults for LRU
- Page faults for MRU

Question (Belady's Anomaly)

Reference String:

1 2 3 4 1 2 5 1 2 3 4 5

Apply FIFO with:

3 frames

4 frames

Compare page faults





Questions for practice

- Question

Reference String:

2 3 2 1 5 2 4 5 3 2 5 2

Frames: 4

Perform dry run using:

- LRU
- Optimal
- Which gives fewer faults?

- Question

1. Why does FIFO suffer from Belady's anomaly?
2. Why is Optimal not used in real systems?
3. When might MRU perform better than LRU?

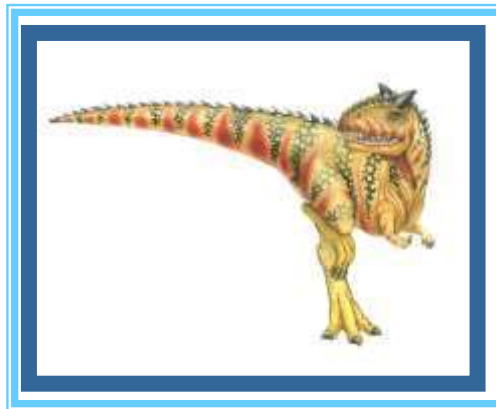




Thank
You



Chapter 2: Operating-System Structures





Contents

- Operating System Services
- User Operating System Interface
- System Calls
- Types of System Calls
- System Programs
- Operating System Design and Implementation
- Operating System Structure
- Virtual Machines
- Operating System Debugging
- Operating System Generation
- System Boot





Operating System Services (User)

- One set of operating-system services provides functions that are **helpful to the user:**
- **User interface** - Almost all operating systems have a user interface (UI)
 - Varies between **Command-Line (CLI)**, **Graphics User Interface (GUI)**, **Batch**
- **Program execution**
 - Loading programs into memory
 - Allocating necessary resources (CPU, memory, I/O)
 - Managing execution lifecycle (start, run, pause, terminate)
 - Handling abnormal termination with error reporting
- **I/O operations** - A running program may require I/O, which may involve a file or an I/O device
- **File-system manipulation**
 - Creating, deleting, reading, writing files
 - Directory management (creation, deletion, navigation)
 - File search and list file Information
 - Permission control and access management





Operating System Services (Cont)

- **Communications** – Processes may exchange information, on the same computer or between computers over a network
 - **Inter-process Communication (IPC):** Communication between processes on the same system
 - **Shared memory:** Processes share and access a common memory area. Like Two processes read/write the same data, **example: Audio player reads it and plays sound**
 - **Message passing:** Processes communicate by sending and receiving messages, **example: A browser process sends a request to a print service.**
 - **Network Communication:** Communication between processes on different systems
 - **Sockets:** Used to send data over a network, **example: A web browser talking to a web server**
 - **Protocol stack:** Handles protocol stack management (TCP/IP), **example: Ensures data arrives correctly over the internet.**





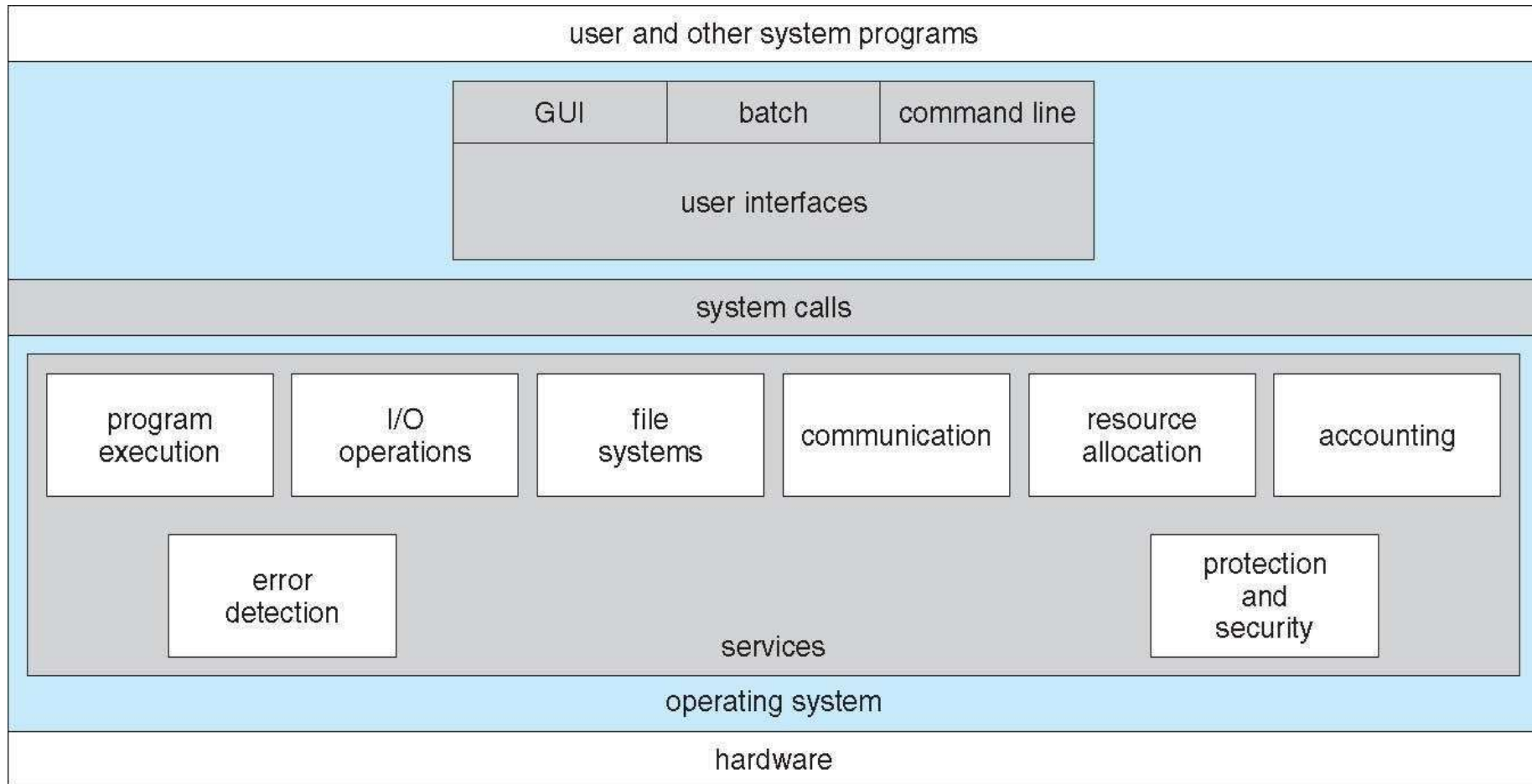
Operating System Services (Cont)

- **Error detection** – OS needs to be constantly aware of possible errors
 - May occur in the CPU and memory hardware, in I/O devices, in user program
 - For each type of error, OS should take the appropriate action to ensure correct and consistent computing
 - Debugging facilities can greatly enhance the user's and programmer's abilities to efficiently use the system





A View of OS Services





OS-System Services

- Another set of OS functions exists for ensuring the efficient operation of the **system itself via resource sharing**
 - **Resource allocation:** When multiple users or jobs run at the same time, system resources must be shared among them.
 - **Types of resources:** Some resources (CPU, memory, storage) need special allocation methods, while others (like I/O devices) use general request and release mechanisms.
 - **Accounting** - To keep track of which users use how much and what kinds of computer resources
 - **Protection and security** :
 - 4 **Protection** ensures controlled access to system resources
 - 4 **Security** prevents unauthorized access through authentication and safeguards external devices.





User Operating System Interface

CLI: Command Line Interface (CLI) or **command interpreter** allows direct command entry

- ❖ Lets users type commands directly.
- ❖ May have different versions called shells.
- ❖ Reads user commands and executes them.
- ❖ Commands can be built-in or external programs.

GUI: Graphical User Interface

- ❖ Easy-to-use, desktop-style interface.
- ❖ Uses mouse, keyboard, and screen.
- ❖ Icons represent files, programs, and actions.
- ❖ Mouse actions open files, show options, or run programs.
- ❖ **Most systems support both CLI and GUI.**
- ❖ **Examples:**
 - ❖ Windows: Mainly GUI with a command-line shell.
 - ❖ macOS: Aqua GUI with a UNIX-based system and shells.





```
[root@localhost ~]# cd /var
[root@localhost var]# ls -la
total 72
drwxr-xr-x. 18 root root 4096 Jul 30 22:43 .
drwxr-xr-x. 23 root root 4096 Sep 14 20:42 ..
drwxr-xr-x.  2 root root 4096 May 14 00:15 account
drwxr-xr-x. 11 root root 4096 Jul 31 22:26 cache
drwxr-xr-x.  3 root root 4096 May 18 16:03 db
drwxr-xr-x.  3 root root 4096 May 18 16:03 empty
drwxr-xr-x.  2 root root 4096 May 18 16:03 games
drwxrwx--T.  2 root gdm 4096 Jun  2 18:39 gnome
drwxr-xr-x. 38 root root 4096 May 18 16:03 log
drwxr-xr-x.  2 root root 4096 May 18 16:03 local
drwxr-xr-x.  2 root root 4096 May 18 16:03 private
drwxr-xr-x.  2 root root 4096 Jul  1 22:11 repository
lrwxrwxrwx.  1 root root    6 May 14 00:12 run -> /run
drwxr-xr-x. 14 root root 4096 May 18 16:03 spool
drwxrwxrwt.  4 root root 4096 Sep 12 23:50 tmp
drwxr-xr-x.  2 root root 4096 May 18 16:03 yp

[root@localhost var]# yum search wiki
plugins: langpacks, presto, refresh-packagekit, re
primary_db
73% [=====
```

Start

VS





Bourne Shell Command Interpreter

```
Terminal
File Edit View Terminal Tabs Help
fd0      0.0    0.0    0.0    0.0  0.0  0.0    0.0  0  0
sd0      0.0    0.2    0.0    0.2  0.0  0.0    0.4  0  0
sd1      0.0    0.0    0.0    0.0  0.0  0.0    0.0  0  0
          extended device statistics
device   r/s    w/s    kr/s    kw/s wait actv  svc_t  %w  %b
fd0      0.0    0.0    0.0    0.0  0.0  0.0    0.0  0  0
sd0      0.6    0.0   38.4    0.0  0.0  0.0    8.2  0  0
sd1      0.0    0.0    0.0    0.0  0.0  0.0    0.0  0  0
(root@pbg-nv64-vn)-(11/pts)-(00:53 15-Jun-2007)-(global)
- (/var/tmp/system-contents/scripts)# swap -sh
total: 1.1G allocated + 190M reserved = 1.3G used, 1.6G available
(root@pbg-nv64-vn)-(12/pts)-(00:53 15-Jun-2007)-(global)
- (/var/tmp/system-contents/scripts)# uptime
12:53am up 9 min(s), 3 users, load average: 33.29, 67.68, 36.81
(root@pbg-nv64-vn)-(13/pts)-(00:53 15-Jun-2007)-(global)
- (/var/tmp/system-contents/scripts)# w
4:07pm up 17 day(s), 15:24, 3 users, load average: 0.09, 0.11, 8.66
User      tty          login@ idle   JCPU   PCPU   what
root      console      15Jun07 18days 1      /usr/bin/ssh-agent -- /usr/bi
n/d
root      pts/3        15Jun07 18      4 w
root      pts/4        15Jun07 18days w
(root@pbg-nv64-vn)-(14/pts)-(16:07 02-Jul-2007)-(global)
- (/var/tmp/system-contents/scripts)#
```





The Mac OS X GUI





System Calls

- **System Call:**

A low-level request made by a program to the OS kernel to perform a task (e.g., read a file, create a process).

- Usually written in high-level languages like C or C++.

OS-specific, complex, and rarely used directly by programs.

Programs normally use APIs instead of calling system calls directly as they are easier and safer than direct system calls.

- **API (Application Programming Interface):**

A high-level interface used by programmers.

Provides easy-to-use functions that **wrap system calls**, hide OS details, and improve portability.

- **Common APIs:**

- **Win32 API** – Windows
- **POSIX API** – UNIX, Linux, macOS
- **Java API** – Java Virtual Machine (JVM)





System Calls

Low-level request means:

A request that **communicates directly** with the operating system kernel, very close to the hardware.

So,

- You must specify **exact details** (memory addresses, registers, parameters).
- It is **harder to write and understand**.
- It is **OS- and hardware-specific**.

Example:

- **Low-level:** Making a system call to read bytes directly from a file.
- **High-level:** Calling an API function like **ReadFile()** or **PrintFile()** that hides those details.

In short:

Low-level = **closer to hardware, more control, more complexity.**





System Calls

Key Idea

- API = what programmers use
- System call = what the kernel uses

Example

`ReadFile() Or PrintFile()` (API) → system call → OS kernel → result returned

Direct System Calls

- Programs **can** make direct system calls, but it's **not recommended**.
- Reasons:** low-level, OS-specific, error-prone, hard to maintain.
- Used mainly in **system programming, OS development, or research**.

So,

APIs are safer, easier, and preferred; system calls are the direct but complex way to access OS services.





Example of Standard API

ReadFile() in the Win32 API reads data from a file.

Parameters:

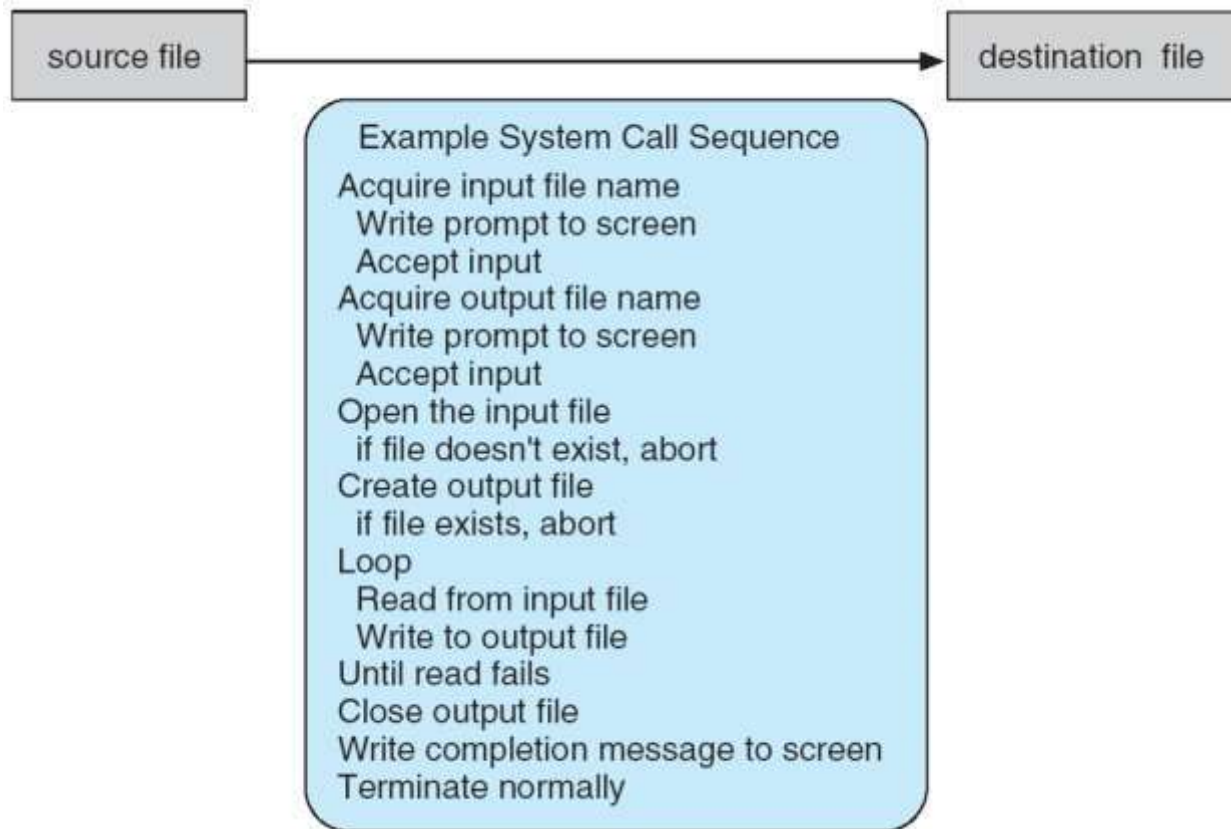
- **file** – the file to read from
 - **buffer** – where the data is stored
 - **bytesToRead** – how much data to read
 - **bytesRead** – how much data was actually read
 - **ovl** – indicates overlapped (asynchronous) I/O
- (Simply, ovl tells the system if the program should wait for I/O to finish or do it in the background while continuing.)





Example of System Calls

- System call sequence to copy the contents of one file to another file





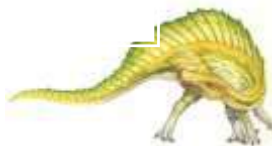
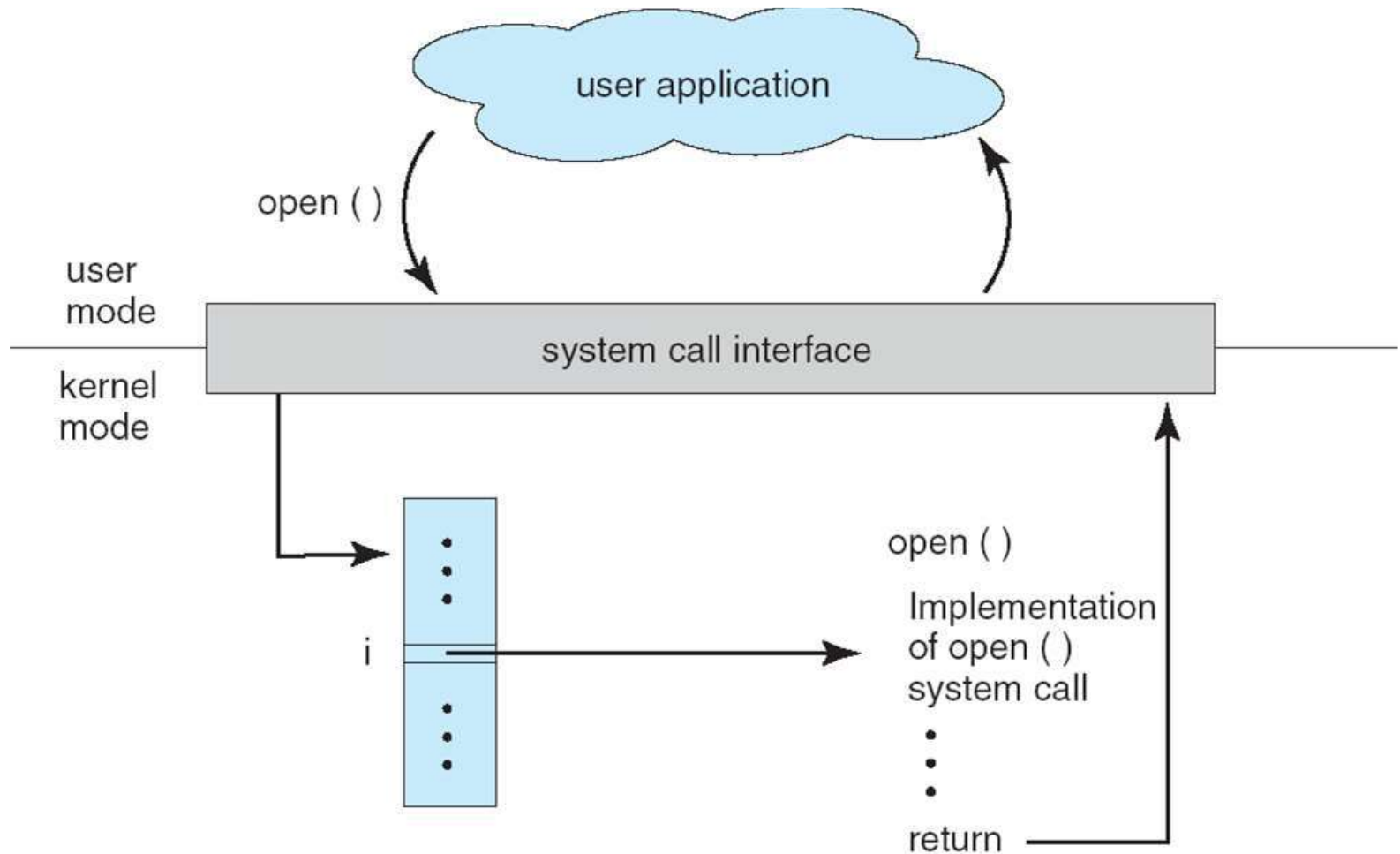
System Call Implementation

- 1. Unique Number:** Each **system call** has a unique number used internally by the OS.
- 2. System Call Table:** The **system call interface** (part of the OS) uses this number to find in table and run the right system call.
- 3. Calling the System Call:**
 - When a program needs a system call, it uses an **API function** (like `ReadFile()` or `open()`) instead of calling the system call directly. The API acts as a user-friendly wrapper that handles the system call behind the scenes, hiding low-level details.
 - The program only needs to know the **API rules** -what functions to call and what to expect back.
 - The **runtime support library** (part of the compiler's tools) provides these APIs.
- 4. OS Executes the Request:** The OS performs the requested task (like reading a file) **inside the kernel**.
- 5. Return to Program:** After finishing, the **OS sends back the result or status to the program**.





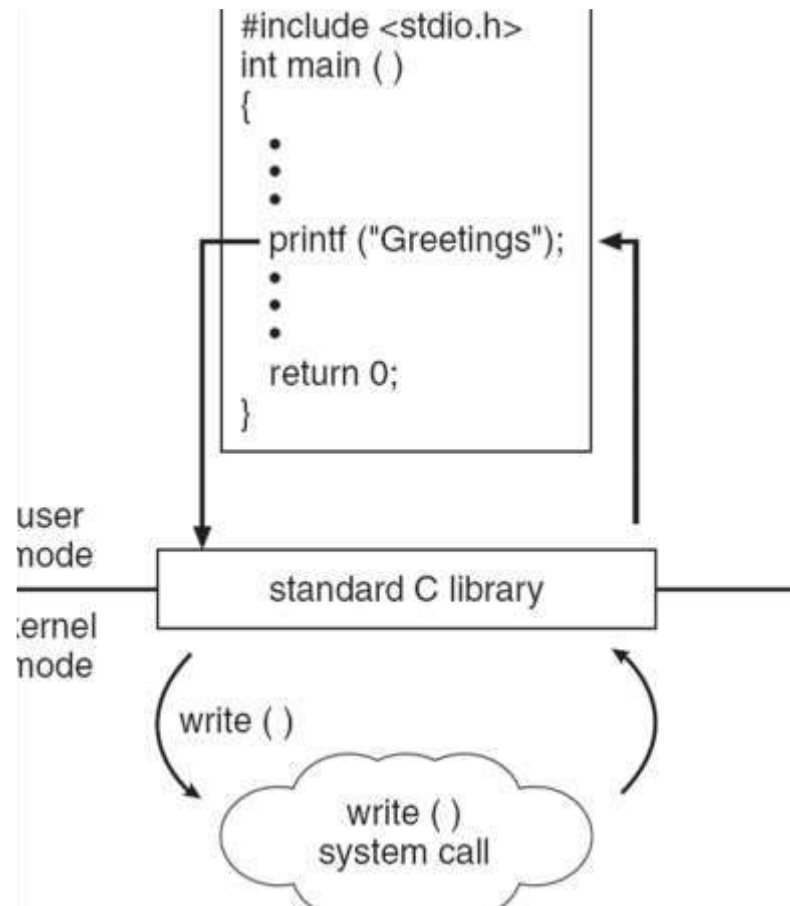
API – System Call – OS Relationship





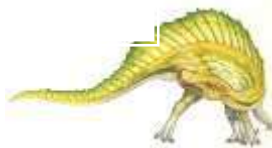
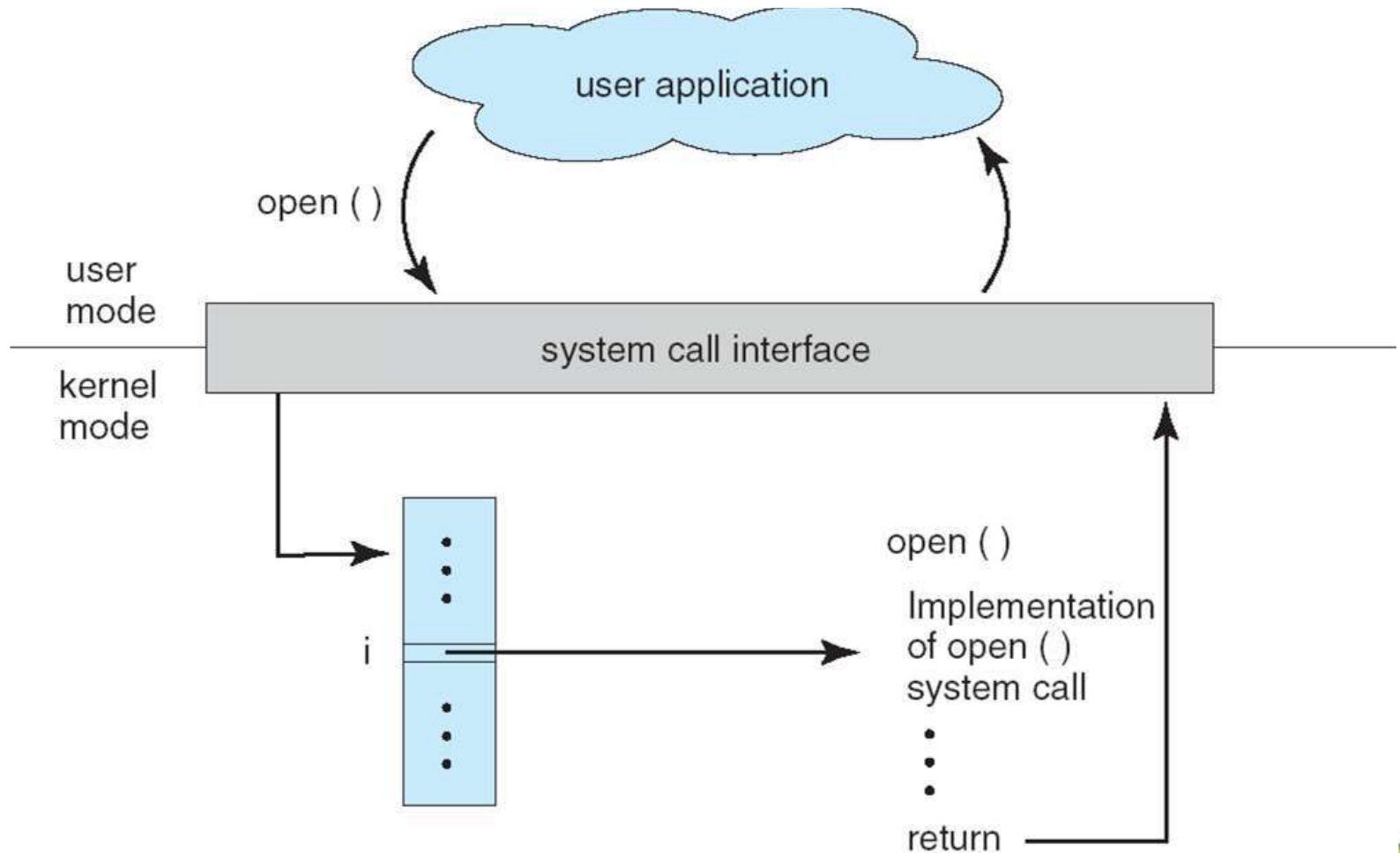
Standard C Library Example

- C program invoking printf() library call, which calls write() system call





API – System Call – OS Relationship





System Call Parameter Passing

- When a program asks the OS to do something (like **open a file**), it must give extra information:
 - file name
 - access mode (read/write)
 - size, location, etc.
- This extra information is called **parameters**.
- System call parameter passing involves transferring data from a user-level process to the operating system kernel to request services.
- **Ways to pass parameters to the OS:**
 1. Registers
 2. Memory block (table)
 3. Stack





Passing parameters using Registers

- The program puts parameters directly into CPU registers.
- The OS reads them from those registers.
- **Example:**
 - Register 1 → system call number
 - Register 2 → file name
 - Register 3 → access mode
- **Pros:** Fast and simple
- **Cons:** **very few registers**, so you can pass only a small number of parameters





Passing parameters using a Memory Block (Table)

- The program stores all parameters together in memory (a block/table).
- The program puts only the address of that block into a register.
- The OS goes to that memory address and reads all parameters.

Example:

- Register contains: address of this memory block
- Memory block contains:
 - o file name
 - o access mode
 - o file size

So,

- Can pass many parameters
- No limit on size
- Used by Linux and Solaris





Passing parameters using the Stack

- The program pushes parameters onto the stack.
- The OS pops parameters off the stack in order.
- **Example:**
 1. Program pushes file name
 2. Pushes access mode
 3. Pushes file size
 4. Makes system call
 5. OS pops parameters

SO,

- Can pass many parameters
- No size limit
- Slightly slower than registers





Types of System Calls

- Process control
- File management
- Device management
- Information maintenance
- Communications
- Protection





Examples of Windows and Unix System Calls

	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shmget() mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()





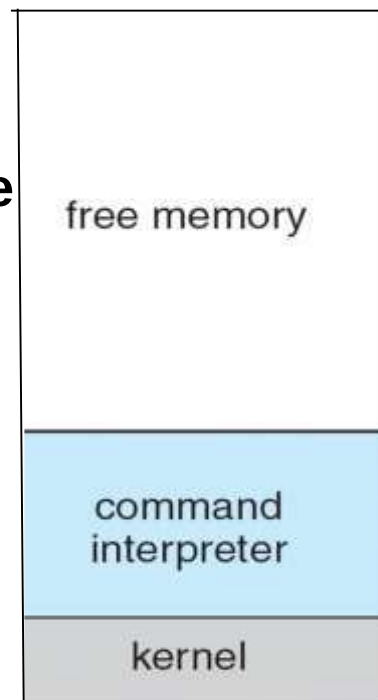
MS-DOS execution

MS-DOS

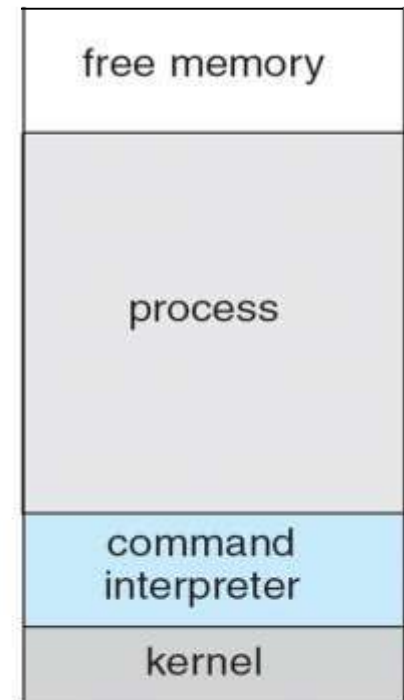
(Microsoft Disk Operating System)

- A **simple operating system**
- Runs **one program at a time** (no multitasking)
- Uses a **command-line interface**
- Little memory management
- Used mainly in **older PCs** (1980s-1990s)

**Best described as:
A single-tasking OS**



(a)



(b)

(a) At system startup (b) running a program

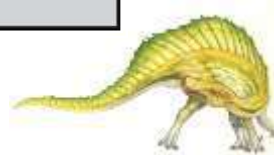
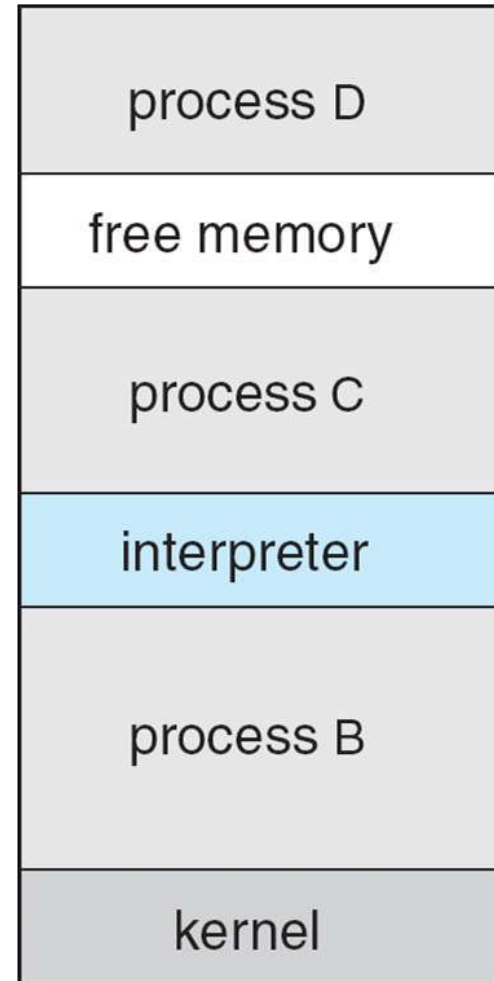


FreeBSD Running Multiple Programs

FreeBSD

- A **modern, UNIX-like** operating system
- Can run **many programs at the same time**
- Strong **memory and process management**
- Supports **multiuser and multitasking**
- Commonly used for **servers and networking**

Best described as: A multi-tasking OS





System Programs

- System programs = what users see and use in an OS.
- They provide the **user interface and tools** that people see as the operating system.
- users interact with **system programs**, not directly with **system calls**.
- **Categories of system programs:**
 - File manipulation
 - Status information
 - File modification
 - Programming language support
 - Program loading and execution
 - Communications
 - Application programs





Functions of System Programs

- **File Management**
 - Create, delete, copy, rename, print, and list files and directories
- **Status Information**
 - Provide system information such as date, time, memory usage, disk space, and number of users
 - May also provide performance, logging, and debugging information
- **File Modification**
 - Text editors
 - Tools to search and modify file contents
- **Programming Language Support**
 - Compilers, assemblers, interpreters, and debuggers
- **Program Loading and Execution**
 - Loaders, linkers, and execution support for machine-level and high-level programs
- **Communications**
 - Enable interaction between users and systems
 - Support messaging, email, web browsing, remote login, and file transfer





Operating System Design and Implementation

- There is **no single perfect way** to design an OS, but some approaches work well.
- The **internal structure** of operating systems can be very different.
- OS design starts by defining **goals and specifications**.
- Design depends on:
 - **Hardware**
 - **Type of system** (desktop, server, mobile, etc.)
- **Goals of an Operating System**
- **User Goals**
 - Easy to use and learn
 - Reliable and safe
 - Fast and efficient
- **System Goals**
 - Easy to design, implement, and maintain
 - Flexible and reliable
 - Error-free and efficient





Operating System Design and Implementation (Cont)

- Important Design Principle
- Policy vs Mechanism
 - **Policy:** *What* should be done
 - **Mechanism:** *How* it is done
- Separating policy from mechanism:
 - Makes the OS **more flexible**
 - Allows policies to be **changed easily** without redesigning mechanisms.
Simply separate **what to do** from **how to do** it for better flexibility





Operating System Structure

When we say **OS structure**, we are answering:

How is the operating system internally built and organized?

Not what the OS *does*, but **how its code is arranged** and **how components interact**.





Simple Structure (MS-DOS)

Concept

Early operating systems were built when:

- Hardware was weak
- Memory was extremely limited
- Single user, single task

How MS-DOS was organized

- No clear division between:
 - File system
 - Memory management
 - Device drivers
- Applications could access **hardware directly**
- No protection between components





MS-DOS Layer Structure

Even though it looks layered,
it's **not truly layered**.

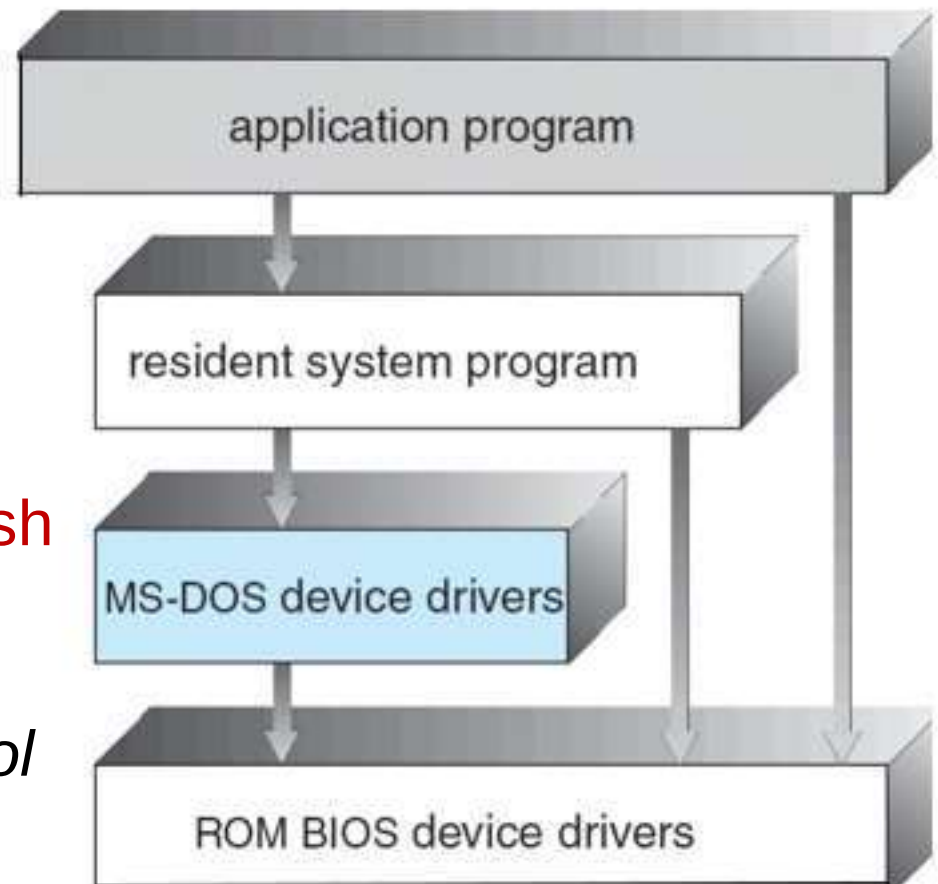
As the figure means:

- Programs → BIOS → Hardware
- But programs can bypass layers

Why this structure failed

- A buggy program could crash the whole system
- No security

So, OS needs *internal control and separation*





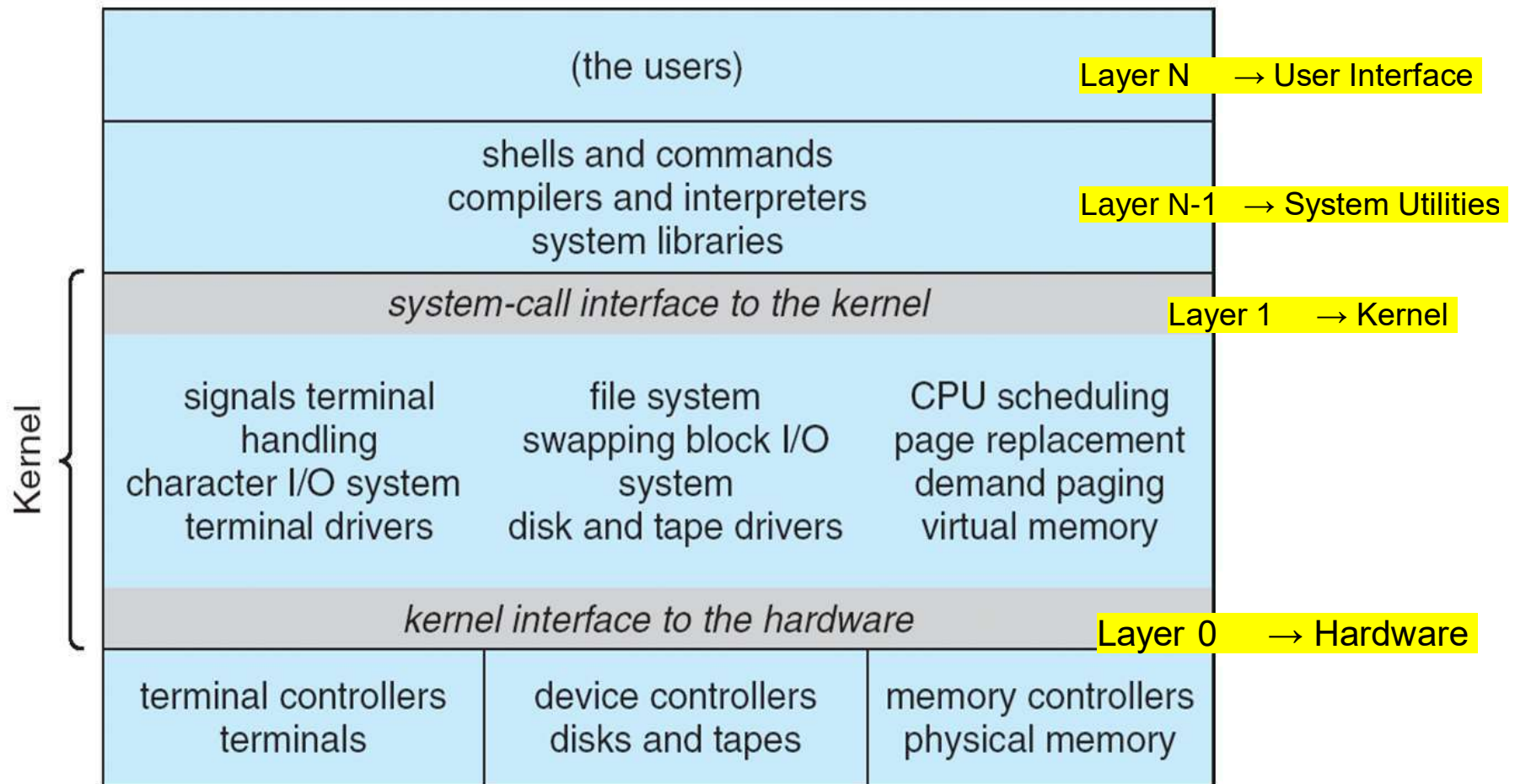
Layered Approach

- The operating system is divided into a number of layers (levels), each built on top of lower layers.
- The bottom layer (layer 0), is the hardware; the highest (layer N) is the user interface.
- With modularity, layers are selected such that each uses functions (operations) and services of only lower-level layers
- **Core idea**
 - Divide OS into layers, each with a specific role.
- **Each layer:**
 - Uses services of only the layer below
 - Provides services to only the layer above





Traditional UNIX System Structure



- No shortcuts
- No layer skipping
- Clear responsibility





Traditional UNIX Structure

- UNIX was developed when:
 - Hardware was still limited & Performance was critical
 - So UNIX used **partial structuring**.
- **UNIX divided OS into two parts:**
 - **1. System Programs**
 - Shell, Editors, Compilers, Commands (ls, cat, clear, echo)
 - **2. Kernel**
 - Process scheduling
 - Memory management
 - File system
 - Device drivers
 - System calls





Layered Operating System

User → System Call Interface → Kernel → Hardware

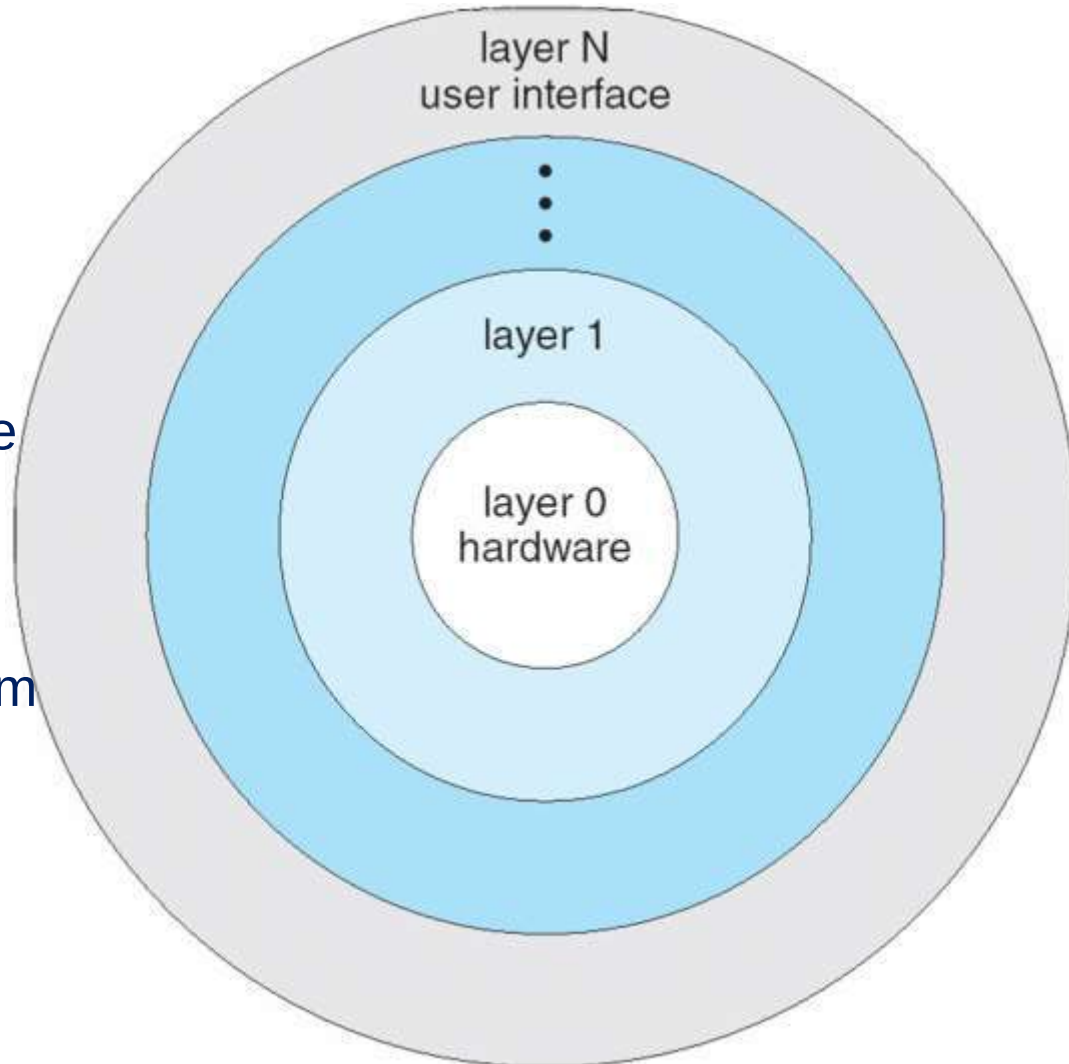
What this means:

- All important work happens inside **one large kernel**
- Kernel is **monolithic**

Problem with UNIX kernel

- Too many responsibilities in one place
- Large kernel = harder to maintain
- One bug can crash entire system

So, next step:
Shrink the kernel.





Microkernel System Structure

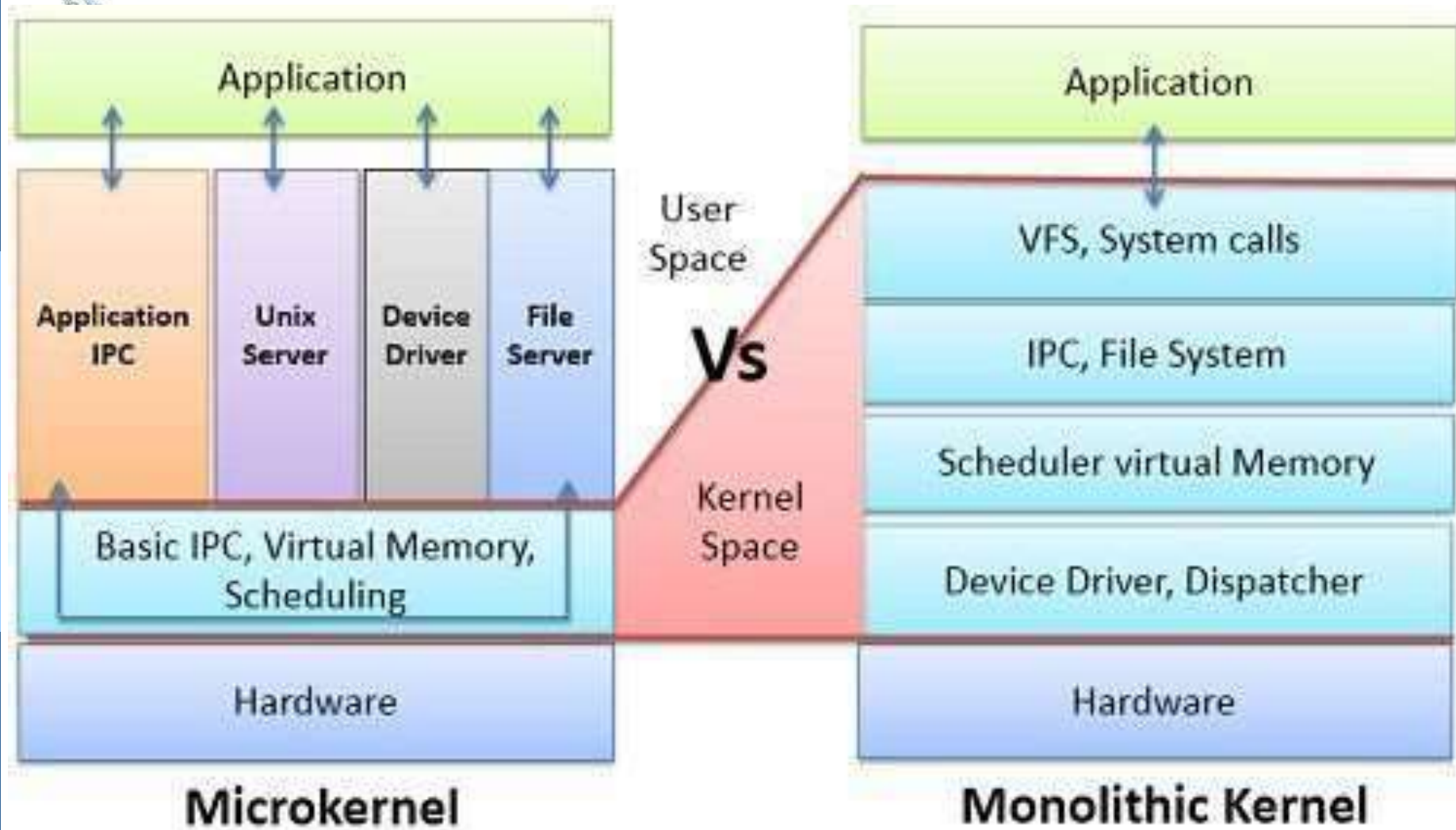
Keep kernel as small as possible.

- **What stays inside the microkernel?**
 - Only **critical** functions:
 - CPU scheduling
 - Memory management
 - Inter-process communication (IPC)
- **What moves outside?**
 - Runs in **user space**:
 - File system
 - Device drivers
 - Network services

Microkernel System Structure

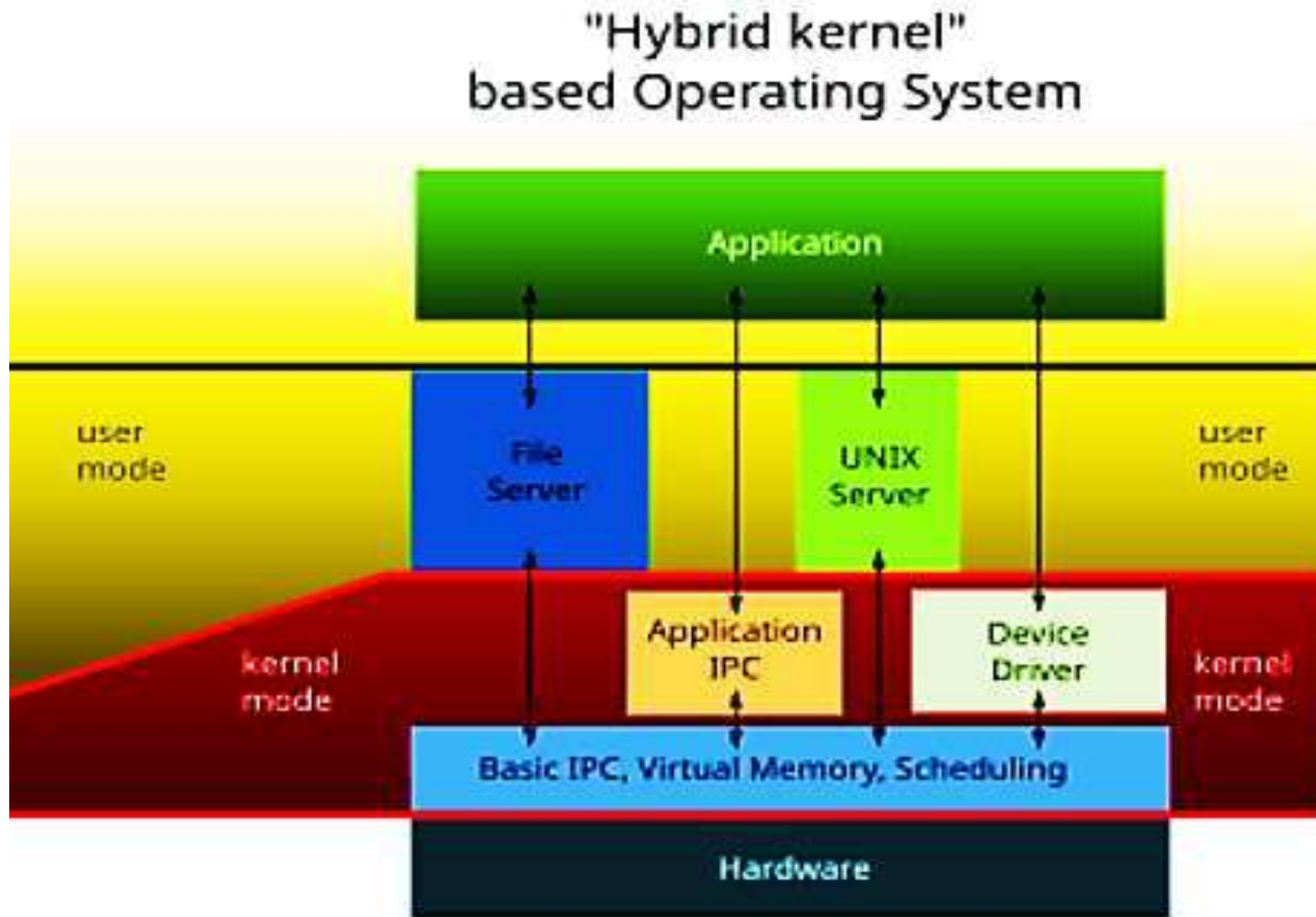
- Small kernel at the center
- User services around it
- Communication via **message passing**







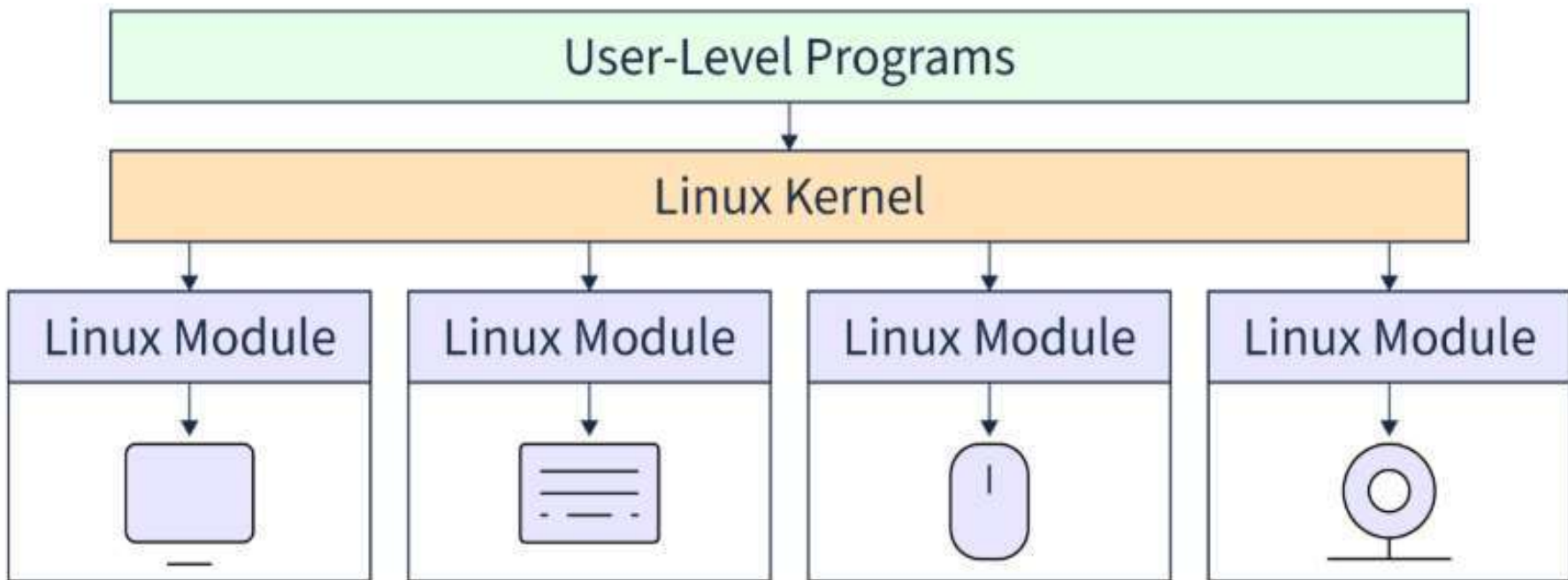
Mac OS X Structure (Hybrid Structure)





Modules- *Flexible Kernel*

- A **kernel module** is a piece of operating system code that can be **loaded into or removed from the kernel while the system is running** (adding features **only when needed.**)
- **kernel modules?** Independent components, Loaded/unloaded at runtime, Communicate via defined interfaces
- **Example:** USB driver loads only when device is plugged in
- Network module loads when network is used





Virtual Machines

A software version of a physical computer.

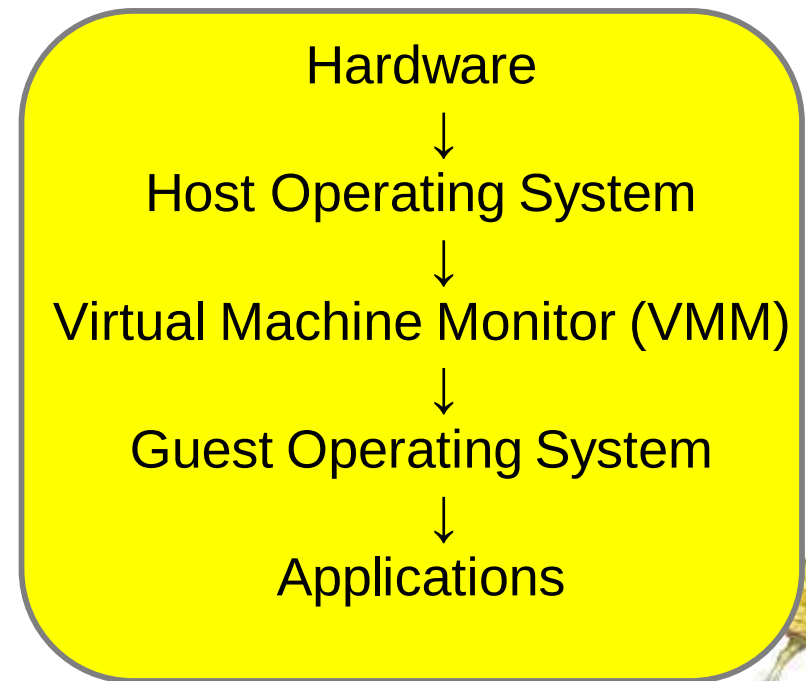
- A **Virtual Machine** is a **software-based computer** that behaves like a **real physical machine**.
- It provides:
 - Virtual CPU
 - Virtual memory
 - Virtual storage
 - Virtual I/O devices
- To the **guest OS**, it looks like real hardware.





Virtual Machines

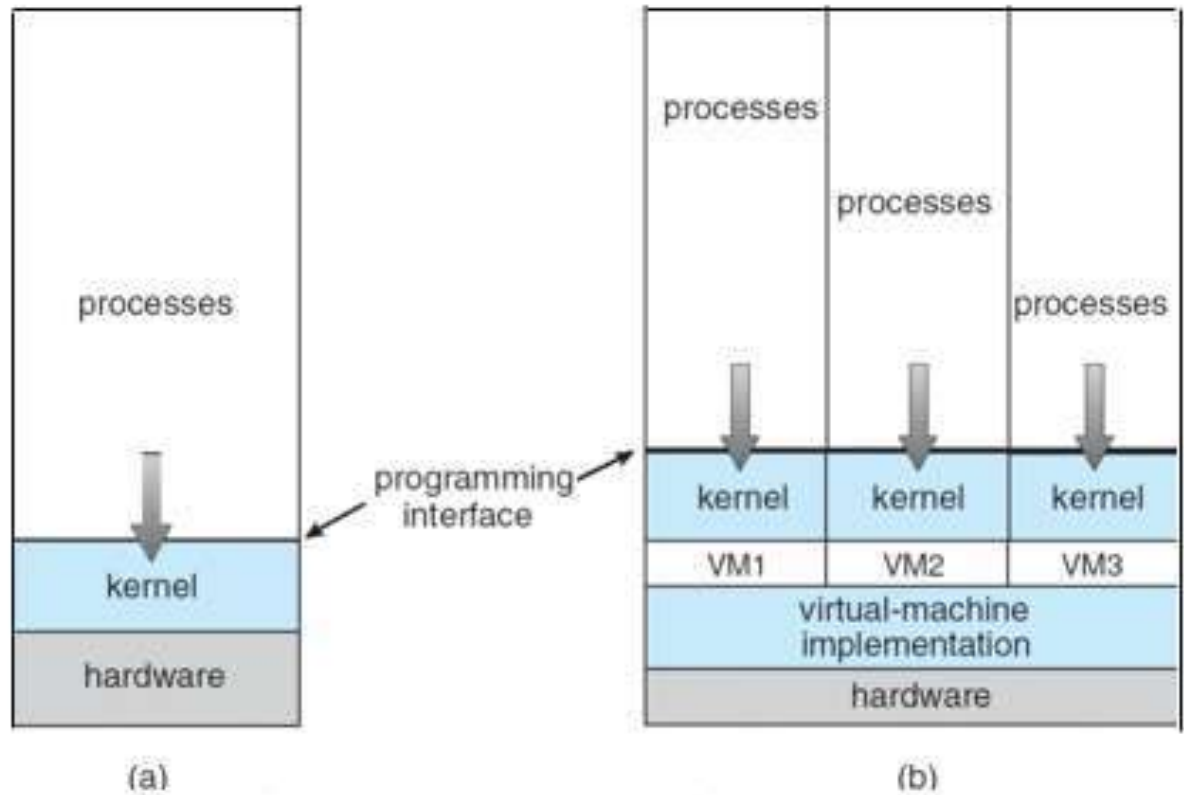
- **Why Virtual Machines are Important**
- Run **multiple OS** on same hardware
- Strong **isolation** (one VM crash doesn't affect others)
- Ideal for **testing & development**
- Backbone of **cloud computing**
- Better hardware utilization (consolidation)





Virtual Machines (Cont)

- **Non-VM:** One OS → One machine
- **VM:** Many OS → Same machine



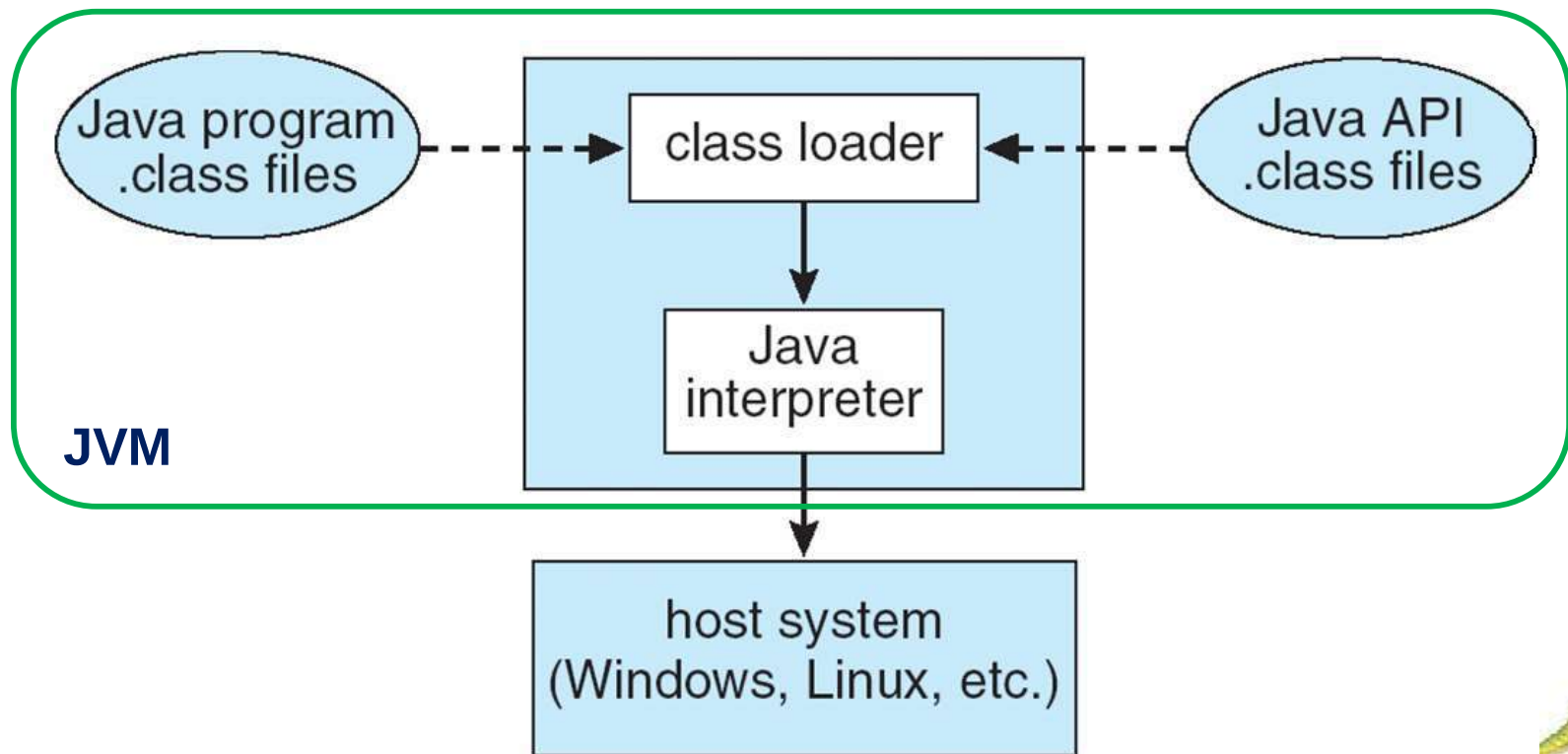


The Java Virtual Machine

JVM is a **runtime environment** that allows Java programs to run on any operating system.

Why Java is platform independent?

Because Java programs run on the JVM, not directly on hardware.





Operating-System Debugging

OS debugging is the process of:

Detecting, analyzing, and fixing errors in the operating system or applications.

Types of Debug Information

1. Log Files

- Text files
- Store error messages and events
- First thing administrators check

2. Core Dump (Application crash)

- Happens when an **application** crashes
- Saves the memory of that program
- Used to analyze *why* it crashed

3. Crash Dump (OS crash)

- Happens when **OS itself** crashes
- Saves kernel memory
- Used by OS developers

- **DTrace tool** in Solaris, FreeBSD, Mac OS X allows
- Live monitoring
- Observes OS behavior without stopping system





Operating System Generation (SYSGEN)

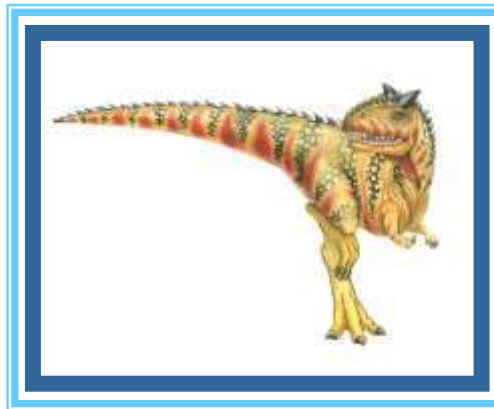
- Customizing the OS

- An OS is written to support **many machines**, but:
 - Every machine has different CPU
 - Different memory
 - Different devices
- So,

OS generation adapts a general OS to a specific hardware configuration.
- **What SYSGEN does:** **SYSGEN customizes OS to hardware**
 - Selects CPU type
 - Configures memory size
 - Enables required device drivers
 - Removes unnecessary components



Chapter 3: Processes





OS-Process

- A **process** is a program **while it is running** (active).
 - When you open Chrome → it becomes a **process**
- A **program** is just a file on disk (inactive).
 - Example: chrome.exe stored on your computer
- **So, A process is a program in execution, together with its current activity and allocated resources.**





Process in Memory

When a program runs, its memory is divided into **different sections**, and each section has a **specific job**. This helps the computer manage data efficiently.

max

Memory sections:

1. Text section

- This holds the **actual program code** (the instructions).
- The code does not change while the program is running.

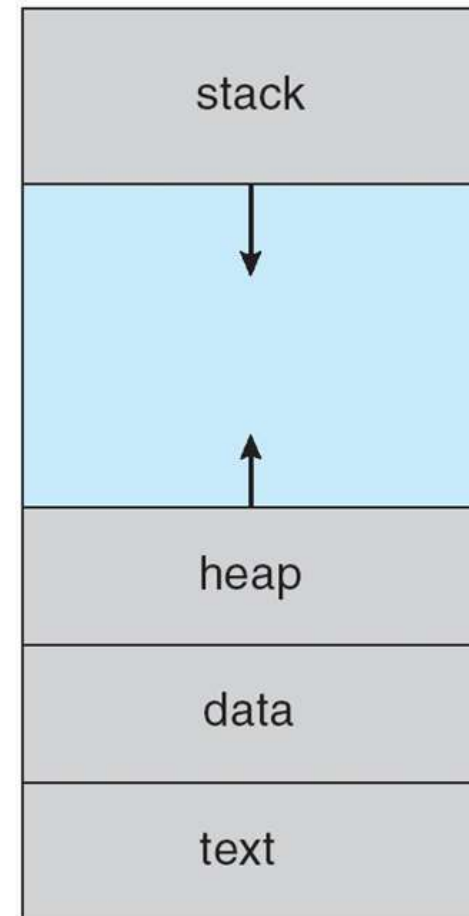
2. Data section

- This stores **global and static variables**.
- These variables exist for the **entire lifetime** of the program.

3. Heap

- This is used for **memory that is allocated during runtime**.
- Used when the program asks for memory dynamically (for example, creating objects or arrays while running).
- Memory here stays allocated until the program explicitly frees it.
- The heap **grows upward** as more memory is requested.

0





Process in Memory

4. Stack

- This stores **temporary data** needed while functions run.
- Includes:
 - Function parameters
 - Local variables
 - Return addresses (where to go back after a function finishes)
- Each function call gets its own stack space.
- The stack **grows downward**.
- When a function finishes, its stack memory is automatically removed.

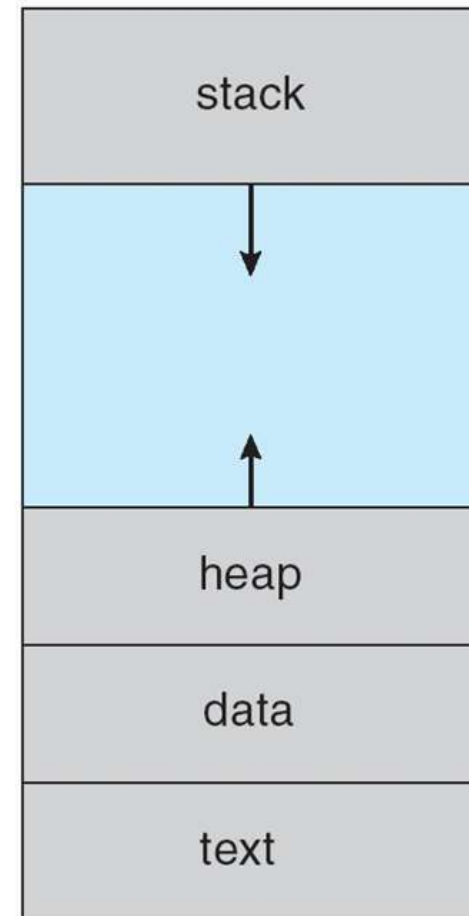
max

Why stack and heap move toward each other

- Stack grows downward
- Heap grows upward
- If they ever meet → **out of memory**

- So,
- Text → program instructions
- Data → global variables
- Heap → dynamic memory during runtime
- Stack → temporary function data (function parameters, return addresses, local variables).

0





Process State

- A process does **not run continuously**.
- **Why?**
 - CPU is shared
 - I/O devices are slow
- So OS tracks process **states**.
- **The 5 fundamental states**
 - **new**: The process is being created
 - **running**: Instructions are being executed
 - **waiting**: The process is waiting for **some event to occur (e.g I/O)**
 - **ready**: The process is waiting to be assigned to a processor (**waiting for CPU**)
 - **terminated**: The process has finished execution





Diagram of Process State

1. new → ready

•**Label:** admitted

- The OS **admits** the **new process into memory** and places it in the **ready queue**.
- Example:** You double-click an app → OS loads it into RAM.

2. ready → running

•**Label:** scheduler dispatch

- The **CPU scheduler** selects this process from the ready queue and gives it the CPU.
- Example:** Your browser gets CPU time after waiting.

3. running → ready

•**Label:** interrupt

- The process is **interrupted** (e.g., time slice expires, higher-priority process arrives).
- Example:** Your music player is paused so a system update can run.

4. running → waiting

•**Label:** I/O or event wait

- The process requests an **I/O operation or waits for an event** (e.g., file read, user input).
- Example:** Word processor waits for you to type something.





Diagram of Process State

5. waiting → ready

- **Label:** I/O or event completion
- The I/O operation finishes or the awaited event occurs.
- **Example:** File finishes loading → process is ready to run again.

6. running → terminated

- **Label:** exit
- Process finishes execution or is killed.
- **Example:** You close a program or it crashes.

Why This Matters?

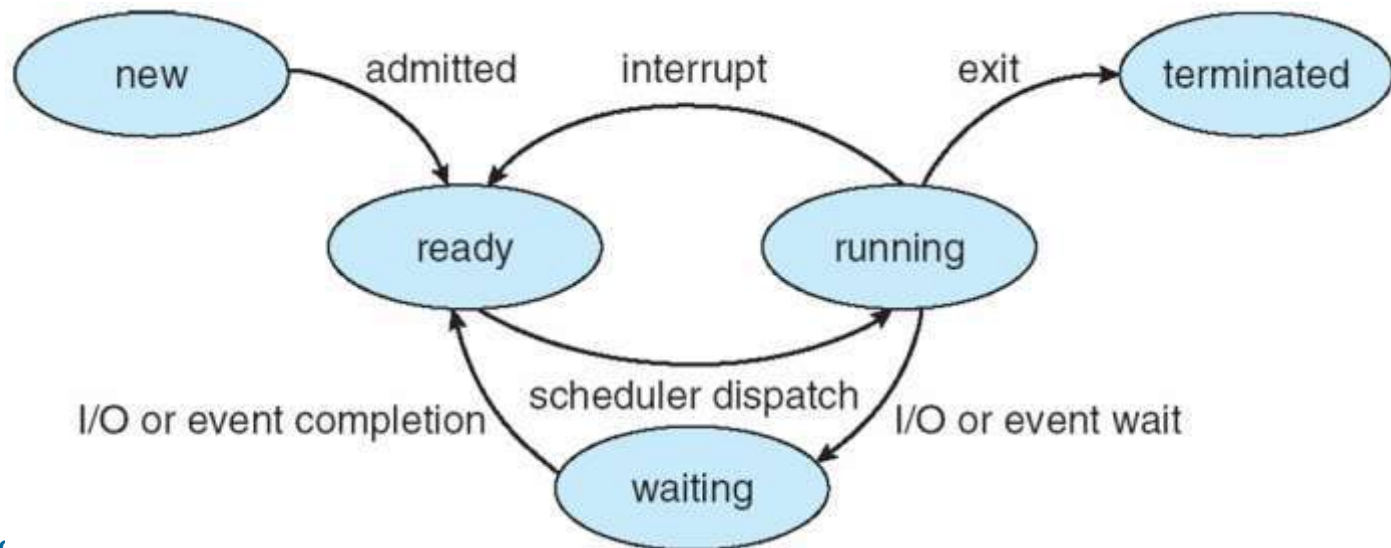
- **CPU Utilization:** Keeps CPU busy by switching between processes.
- **Responsiveness:** Lets you interact with programs while they do I/O.
- **Fairness:** No single process hogs the CPU.





Process State-Example

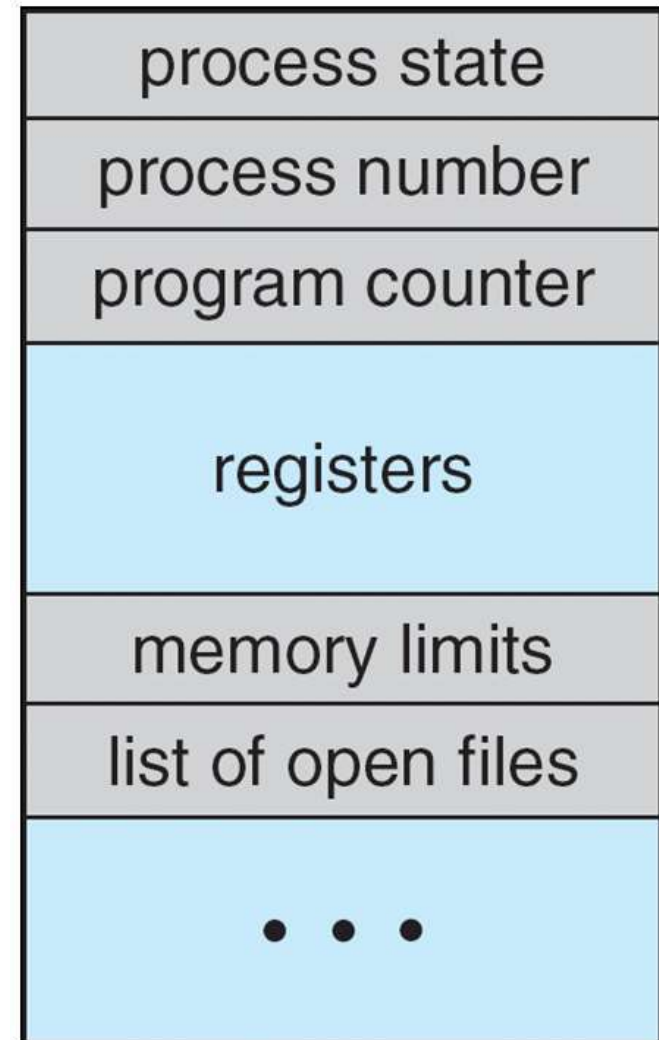
- **New** → You double-click a text editor.
- **Admitted / Ready** → OS loads it into memory and puts it in the ready queue.
- **Running** → CPU starts executing the text editor.
- **Waiting** → Editor waits for keyboard input (I/O).
- **Ready** → You type; input is received.
- **Running** → CPU continues executing the editor.
- **Ready (Interrupt)** → Time slice expires; CPU is given to another process.
- **Running** → Editor gets CPU again.
- **Terminated** → You close the editor.





Process Control Block (PCB)

- **Problem:**
- CPU switches between processes constantly.
How does OS remember where each process stopped?
- **Solution:**
- **PCB**
- PCB is a **data structure** that stores **complete information** about a process.
- **Without PCB, multitasking is impossible**





Process Control Block (PCB)

Each process has a PCB, which stores all information the OS needs to manage it. (also called **task control block**)

■ PCB contains:

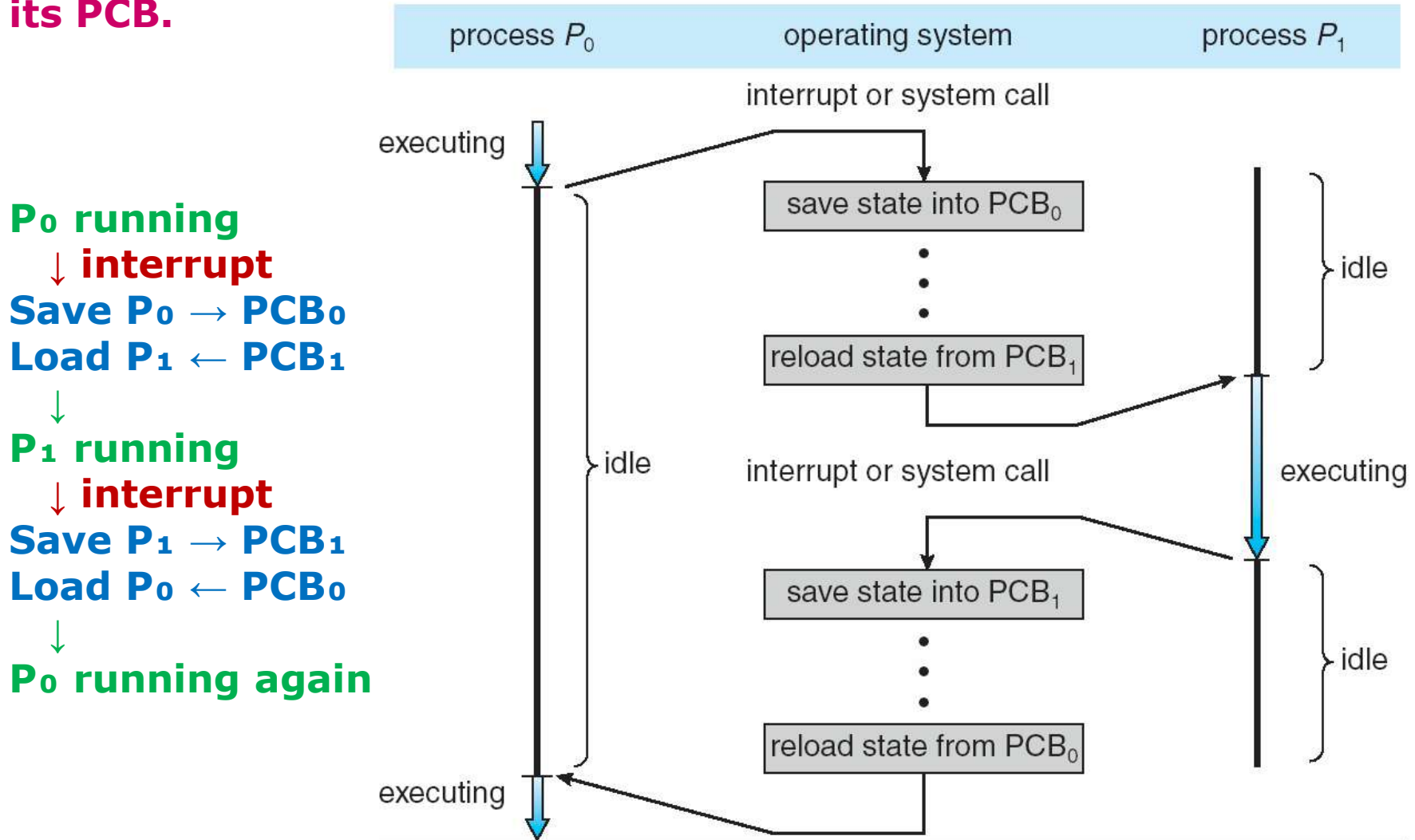
- **Process state** → whether the process is running, ready, or waiting
- **Program counter** → address of the next instruction to execute
- **CPU registers** → current values needed to resume execution
- **Scheduling info** → priority and position in the ready queue
- **Memory info** → memory assigned to the process
- **Accounting info** → how much CPU time the process has used
- **I/O info** → devices in use and files opened by the process





CPU Switch From Process to Process

CPU switch occurs when the OS saves the state of the running process in its PCB and loads the state of another process from its PCB.



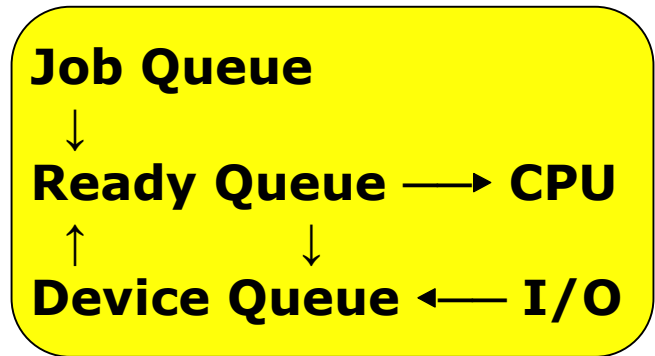


Process Scheduling

The goal of process scheduling is to keep the CPU busy by quickly switching between processes. **The process scheduler** decides which process runs next on the CPU.

Why scheduling is needed

- CPU is shared
- Many processes want CPU
- OS must decide **who runs next**



Scheduling Queues

- **Job queue** → all processes in the system
 - **Ready queue** → processes in memory waiting for CPU
 - **Device queues** → processes waiting for I/O devices
- Processes **move between these queues** as they execute, wait for I/O, or finish.





Types of Schedulers

1. Short-Term Scheduler (CPU Scheduler)

- Chooses **which process runs next on the CPU**
- Works **very frequently** (every few milliseconds)
- In some systems, this is the **only scheduler**

So,

Decides *who gets the CPU right now*

2. Long-Term Scheduler (Job Scheduler)

- Decides **which processes are allowed into memory**
- Runs **rarely** (seconds or minutes)
- Controls the **number of processes in memory**
(degree of multiprogramming)

So,

Decides *how many processes enter the system*





Types of Schedulers

Types of Processes

•I/O-bound process (Mostly waiting, little thinking)

- Mostly waits for I/O
- Short CPU usage
- Example:
Text editor (waiting for typing)
Browser (waiting for network)

•CPU-bound process (Mostly thinking, little waiting)

- Mostly does computation
- Long CPU usage
- Example:
Image processing
Video rendering

Why OS even cares about this difference?

Because **good performance needs balance.**

- Too many CPU-bound → I/O devices idle
- Too many I/O-bound → CPU idle

So, **Long-term scheduler keeps a good mix of both types.**





Types of Schedulers

3. Medium-Term Scheduler (Optional)

- Used when system is overloaded
- **Temporarily removes processes from memory**
- Stores them on disk (**swapping**)
- Brings them back later to continue execution

So,

Pauses some processes to reduce memory load





Multitasking in Mobile Systems

- Early mobile systems (e.g early iOS) allowed only **one active process**; others were **suspended**.
- Limited screen space and UI constraints affect multitasking design.

■ iOS

- Supports **one foreground process** (user-controlled).
- Allows **multiple background processes** with restrictions.
- Background limits include:
 - Short tasks
 - Event notifications
 - Specific long-running tasks (e.g., audio playback)

■ Android

- Supports both **foreground and background processes** with fewer limits.
- Background work is done using **services**:
 - Can continue running even if the process is suspended
 - No user interface
 - Low memory usage





Context Switch

- A **context switch** occurs when the CPU switches from one process to another.
- The system:
 - Saves the current process state
 - Loads the next process state
- Process state is stored in the **Process Control Block (PCB)**.
- Context switching is **overhead** (no useful work is done during the switch).
- More complex OS/PCB → **longer context switch time**.
- Switch time depends on **hardware support**.
- Some CPUs have **multiple register sets**, allowing multiple contexts to be loaded simultaneously.





Operations on Processes

- System must provide mechanisms for:
 - process creation,
 - process termination,





Process Creation

- **Parent** process create **children** processes, which, in turn create other processes, forming a **tree** of processes
- When a process creates another process:
 - the original one is the **parent**
 - the new one is the **child**
- This keeps happening → you get a **process tree**
- Generally, process identified and managed via a **process identifier (pid)**





Process Creation

- **Why does the OS want this parent-child relationship at all?**
- **A process tree is needed so the OS can** manage resources, control execution, enforce security, support synchronization, and cleanly terminate related processes through parent-child relationships.





Process Creation

1. Resource management (biggest reason)

The OS needs to know **who owns what & how resources are shared.**

If a child process:

- opens files
- allocates memory
- uses CPU time

The OS can say:

“These resources belong to this process, which belongs to that parent.”

Example

If a parent process exits:

- the OS can **clean up all children**
- no orphan processes eating memory/CPU forever

This is why: Cascading termination exists





Process Creation

■ Resource Sharing

■ Three possibilities:

1. Parent and child share all resources

- Same files
- Same memory
- Very rare (dangerous)

2. Child shares a subset of parent's resources (most common)

- Child inherits open files
- Memory is copied (or copy-on-write)
- Safer and flexible

3. Parent and child share no resources

- Totally isolated
- Used in secure systems





Process Creation

2. Controlled process creation

Processes **should not appear randomly.**

Instead:

- Only existing processes can create new ones
- This creates accountability

Example:

- Shell (bash) creates your program
- Your program creates helper processes

The OS can trace:
Who launched this?
Who is responsible for it?





Process Creation

3. Execution (who runs when?)

also called Parent–child synchronization

Two choices:

1. Parent and child execute concurrently

- Both run at the same time
- OS scheduler switches between them

2. Parent waits for child to finish

- Parent pauses
- Child runs
- Parent resumes after child exits

This is controlled by system calls like wait().





Process Creation

4. Job control (especially in UNIX)

- The shell is the **parent**
- Background jobs are **children**

The process tree lets the shell:

- pause jobs
- resume jobs
- kill jobs

No tree → no job control.





Process Creation

5. Security & permissions

- The OS can enforce rules like:
 - A process can only control **its own children**
 - Random processes cannot kill each other

This prevents:

- accidental system damage

So the tree is a **trust boundary**.





Process Creation

■ Why Process Tree is Needed?

- Manages and tracks resource ownership.
- Enables parent–child synchronization.
- Supports controlled process creation.
- Ensures security and accountability.





UNIX Process Creation

- Address Space (memory stuff)
- Address space = process memory

Two possibilities:

1. Child is a duplicate of parent

- Same code
- Same variables
- Same memory layout (initially)

2. Child loads a new program

- Old memory is wiped
- New program is loaded

- This leads us to UNIX...

UNIX uses **two system calls together**:

- ◆ fork system call creates new process
- ◆ exec system call used after a fork to replace the process' memory space with a new program

fork()

- Creates a **new process**
- Child is a **copy** of parent

exec()

- Replaces the child's memory with a **new program**
- Used after fork()





Process Termination

Normal termination

- Process finishes last statement
- Calls `exit()`
- OS cleans everything up

Process' resources are deallocated by operating system

Parent collecting child's result

- Parent calls `wait()`
- Child sends **exit status** to parent

Output data from child to parent (via `wait`)





Process Termination

Parent killing child (abort)

A parent can terminate a child if:

- Child used too many resources

- Child's task is no longer needed

- Parent itself is exiting

Important rule:

OS does not allow child to continue if its parent terminates

called Cascading termination

So if a parent dies → all its children die too.





Cooperating vs Independent Processes

Independent Process

- Shares **NO data**
- Cannot affect others
- Example:
 - Calculator app
 - Notepad app

Cooperating Process

- Shares **data or resources**
- Can affect or be affected by others
- Example:
 - Browser + download manager
 - Editor + compiler

So:

If processes share data → they are cooperating.





Cooperating Processes-Advantages

- **Information Sharing**

Multiple processes can access shared data or files.

- **Computation Speedup**

Tasks are divided into subtasks that execute in parallel on multiple processors.

- **Modularity**

System is divided into smaller, manageable processes or threads.

- **Convenience**

Users can perform multiple tasks simultaneously (editing, compiling, printing).





Cooperating vs Independent Processes

Independent Process

- Shares **NO data**
- Cannot affect others
- Example:
 - Calculator app
 - Notepad app

Cooperating Process

- Shares **data or resources**
- Can affect or be affected by others
- Example:
 - Browser + download manager
 - Editor + compiler

So:

If processes share data → they are cooperating.





Interprocess Communication

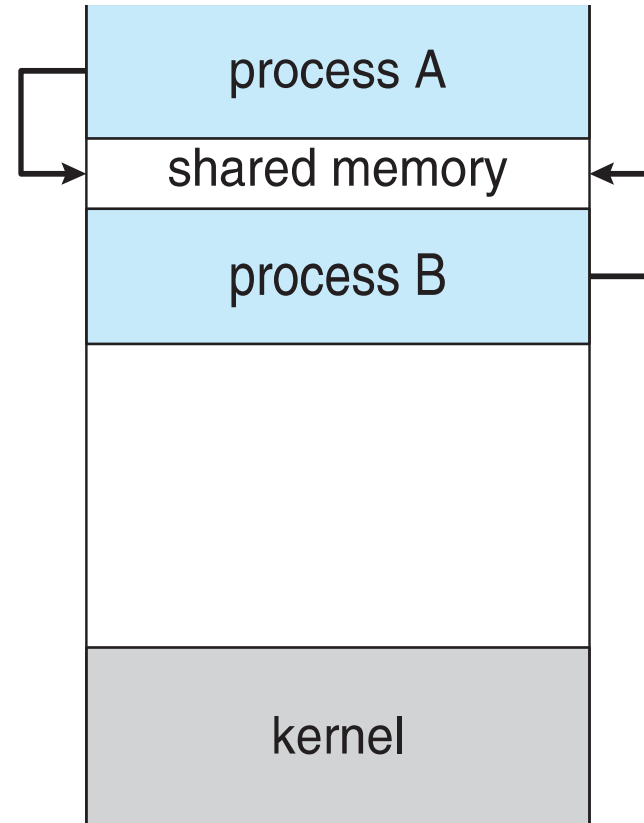
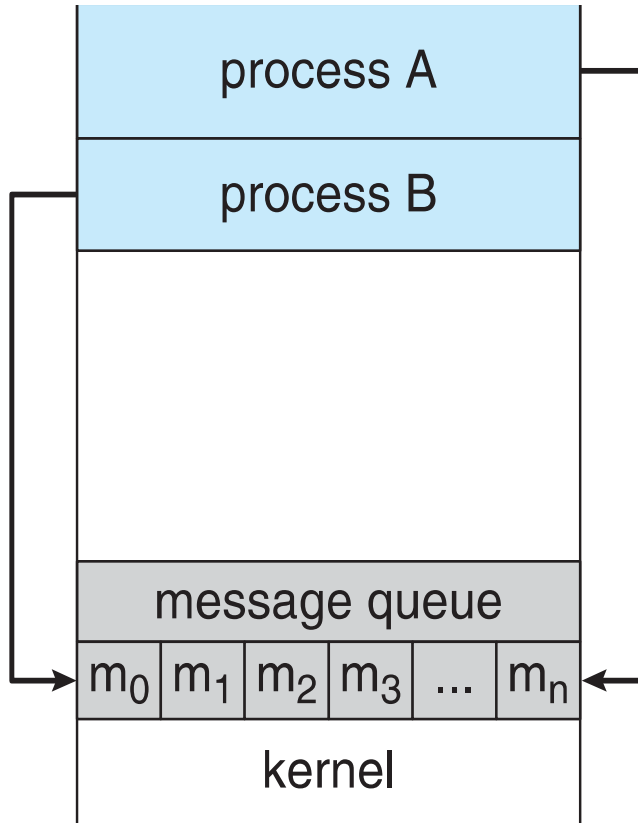
- **Interprocess Communication (IPC) is just how different running programs (processes) talk to each other.**
- Cooperating processes need **interprocess communication (IPC)**
- Two models of IPC
 - **Message Passing:**
 - ▶ Processes send messages to each other through the operating system (kernel).
 - ▶ They do NOT share memory.
 - **Shared Memory**
 - ▶ Processes share the same memory space.
 - ▶ They read and write directly to it.





Interprocess Communication

(a) Message passing. (b) shared memory.



- Process A → sends a message
 - Kernel → stores it in a **message queue**
 - Process B → receives the message
- The kernel controls everything.**

- OS creates a **shared memory region**
- Process A and B both attach to it
- Data is **written/read directly**
- **Kernel sets it up once, then steps back.**





Core problem in shared memory

Multiple processes access the same data at the same time, and that can mess things up because multiple processes may access and modify shared data simultaneously, leading to **data inconsistency or data corruption**.

- That's why Shared memory requires **synchronization***

***processes synchronization is coordination mechanism in OS manages concurrent process execution to ensure data consistency and prevent conflict**





Producer-Consumer Problem

The Producer-Consumer problem exists to safely exchange data between two independent activities using a temporary storage area called a buffer.

Key Roles

Producer

- Creates or generates data
- Example: keyboard

Consumer

- Uses or processes data
- Example: text editor reading keys, printer, display

Buffer

- Temporary **waiting area** between producer and consumer
- Stores data until it is used
 - Unbounded Buffer**
 - Bounded Buffer**





•**Unbounded Buffer:** has no size limit, Producer can keep producing forever, Consumer just consumes whenever it wants, **No risk of buffer getting full**

Bounded Buffer

- Buffer has **fixed size, requires synchronization:**
- If buffer is full → producer must wait
- If buffer is empty → consumer must wait

If there isn't any synchronization, then?

Situation	What Goes Wrong
Produce when buffer is full	Old data overwritten
Consume when buffer is empty	Invalid or garbage data





Interprocess Communication – Message Passing

- In **Message Passing**, processes:
 - DO NOT share memory directly
 - Communicate by sending messages to each other
- IPC Message Passing uses only **two operations**:
- **send(message)**: A process sends a message.
- **receive(message)**: A process receives a message.

- **Messages can be**:
 - **Fixed size** (example: always 256 bytes)
 - **Variable size** (can change depending on data)
 - **Message 1**: "Hi" → 2 characters
 - **Message 2**: "Hello, how are you?" → 18 characters
 - **So, Each message has a different size.**





Message Passing (Cont.)

- If processes P and Q wish to communicate, they need to:
 - Establish a **communication link** between them
 - Exchange messages via send/receive
- **Implementation issues:**
 - **1. How are links established?**
 - How do processes connect?
 - Example: Automatically by OS Or manually created by programmer
 - **2. Can a link connect more than 2 processes?**
 - There are two possibilities:
 - **One-to-one:** Only P and Q communicate.
 - **Many-to-many:** Multiple processes share one link.
 - ▶ Example: Group chat vs private chat.





Message Passing (Cont.)

■ 3. How many links can exist between two processes?

- Just one?
- Or multiple communication channels?
- **Example:** One link for text or Another for video

■ 4. What is the capacity of a link?

- Capacity = How many messages can wait in the queue?
- **Example:** Capacity = 10 messages so
 - ▶ If full → sender must wait?

■ 5. Is message size fixed or variable?

- Fixed → easier to manage
- Variable → more flexible

■ 6. Is communication one-way or two-way?

- **Unidirectional:** One direction only. $P \rightarrow Q$
- **Bidirectional:** Both can send and receive. $P \leftrightarrow Q$





Message Passing (Cont.)

- **Communication links can be implemented in two ways:**
- **(Implementation of communication link)**
 - **Physical:** This means communication depends on **physical system structure**.
 - Shared memory
 - Hardware bus
 - Network
 - **Logical:** This is **about how communication behaves**, not hardware.
 - ❖ **Direct:** Process sends message directly to another process.
 - ❖ **Indirect:** Messages go to a mailbox.
 - ❖ **Synchronous (Blocking):** Sender waits until receiver gets message. **Ex:** You call someone and wait until they answer.
 - ❖ **Asynchronous (Non-blocking):** Sender sends and continues working. **Ex:** You send a text and continue doing your work.
 - ❖ **Buffer = Temporary storage for messages**
 - Automatic buffering:** OS manages message storage.
 - Explicit buffering:** Programmer must manage buffer.





Direct Communication

Processes must **know each other's name**

They send messages directly to a specific process

■ How It Works

■ If Process A wants to send a message to Process B:

▶ `send(B, message)`

■ If Process B wants to receive from Process A:

▶ `receive(A, message)`

■ So both processes must explicitly mention each other.





Direct Communication

Properties of communication link

■ Links are established automatically

- ▶ When A sends to B, the **OS automatically creates the communication link.**
- ▶ Programmer doesn't need to manually create it.

■ One link = One pair of processes

- ▶ Each communication link **connects only two processes.**

■ Exactly one link between two processes

- ▶ If A and B communicate:
- ▶ There is **only one communication path** between them.
- ▶ Not multiple.

■ Link may be unidirectional or bidirectional

- ▶ Unidirectional $\rightarrow A \rightarrow B$ only
- ▶ Bidirectional $\rightarrow A \leftrightarrow B$ (both can send)





Indirect Communication

■ Messages are directed and received from mailboxes (also referred to as ports)

- Each mailbox has a unique id
- Processes can communicate only if they share a mailbox

■ Properties of communication link

- **Link exists only if they share a mailbox**
 - ▶ Two processes can communicate only if they **both use the same mailbox.**
 - ▶ **No shared mailbox = No communication.**
- **A link may be associated with many processes**
 - ▶ One mailbox can be used by: P1, P2, P3, P4
- **Each pair of processes may share several communication links**
 - ▶ P1 and P2 share: Mailbox A & Mailbox B
 - ▶ So they have **multiple communication links.**
- **Link may be unidirectional or bi-directional**





Indirect Communication

■ Operations

- create a new mailbox (port)
- send and receive messages through mailbox
- destroy a mailbox

■ Primitives are defined as:

send(*A, message*) – send a message to mailbox A

receive(*A, message*) – receive a message from mailbox A





Indirect Communication

■ Mailbox sharing

- P_1 , P_2 , and P_3 share mailbox A
- P_1 sends; P_2 and P_3 receive
- Who gets the message?

■ Solutions

- **Only two processes per mailbox:** Restrict mailbox to only 2 processes, Then no confusion.
- **Only one process can receive at a time:** System ensures only one receiver is active.
- **OS randomly selects receiver:** System decides who gets the message, Sender is informed who received it.





Indirect Communication (working)

■ When:

- **P1 sends a message → It is placed inside mailbox A.**
- **P2 reads the message → The message is removed.**

■ Now it is gone. So,

- **P3 cannot read it anymore, it will be assigned with some other task**

In standard IPC mailbox:

- Message is stored in buffer
- One process receives it
- Message is removed
- Next message is delivered to next process

It behaves like: FIFO Queue

Imagine:

P1 → sends tasks

P2, P3, P4 → worker processes

Mailbox contains tasks: Task1, Task2, Task3

P2 gets Task1

P3 gets Task2

P4 gets Task3

Each task is processed once.



Indirect Communication

■ Then Why Allow Multiple Processes to Share Mailbox?

- Multiple producers can send messages
- Multiple consumers can compete to receive messages
- OS decides who gets which message

■ Why Not Allow All Processes to Read the Same Message?

● 1: Duplicate Work

Multiple processes may process same data.

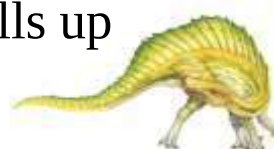
Example: Bank transaction processed 3 times

● 2: Synchronization Issues

All processes accessing same message at same time may cause:
Race conditions, Data corruption, System instability

● 3: Memory Control

If message is not removed: Mailbox keeps growing and Memory fills up





Synchronous vs Asynchronous

- **Message passing may be either blocking or non-blocking**
- **Blocking (Synchronous)**
 - **Blocking Send** → Sender waits until message is received.
 - **Blocking Receive** → Receiver waits until a message arrives.
 - If *both* send and receive are blocking → **Rendezvous** (they meet at the same time).
- **Non-blocking (Asynchronous)**
 - **Non-blocking Send** → Sender sends and continues immediately.
 - **Non-blocking Receive** → Receiver:
 - Gets a message if available
 - Gets **null** if no message
- **Different combinations are possible (e.g., blocking send + non-blocking receive).**





Buffering (Message Storage)

■ Who manages the buffer?

- **Automatic buffering** → OS manages it.
- **Explicit buffering** → Programmer manages it.

■ Buffer Capacity Types

● Zero Capacity

- No message storage
- Sender must wait for receiver → Rendezvous

● Bounded Capacity (size = n)

- Limited number of messages
- Sender waits if buffer is full

● Unbounded Capacity

- Unlimited storage
- Sender never waits





Client-Server Systems

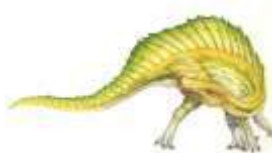
- In a **client–server system**:
 - **Client** → Requests something (like your browser)
 - **Server** → Provides the service (like Google's server)
- Example: When you open YouTube:
 - Your **browser (client)** sends a request
 - YouTube's **server** responds with the video
- **How do they talk to each other?**
- They use different communication methods.
 - Sockets
 - Remote Procedure Calls
 - Pipes





Sockets

- A **socket** is an **endpoint (a communication point)** that allows two computers (or two programs) to talk to each other over a network.
- Think of it like:
 - A **phone call**
 - Each person has a **phone number**
 - When two numbers connect → communication happens
- In networking:
 - The **IP address** is like the house address
 - The **Port number** is like the apartment number
 - The **Socket** = IP address + Port number





Sockets

An **IP address** identifies a device on a network.

- Example: 161.25.19.8 (Tells: Which computer (host) we are talking to.)

A **port number** identifies a specific program/service running on that computer.

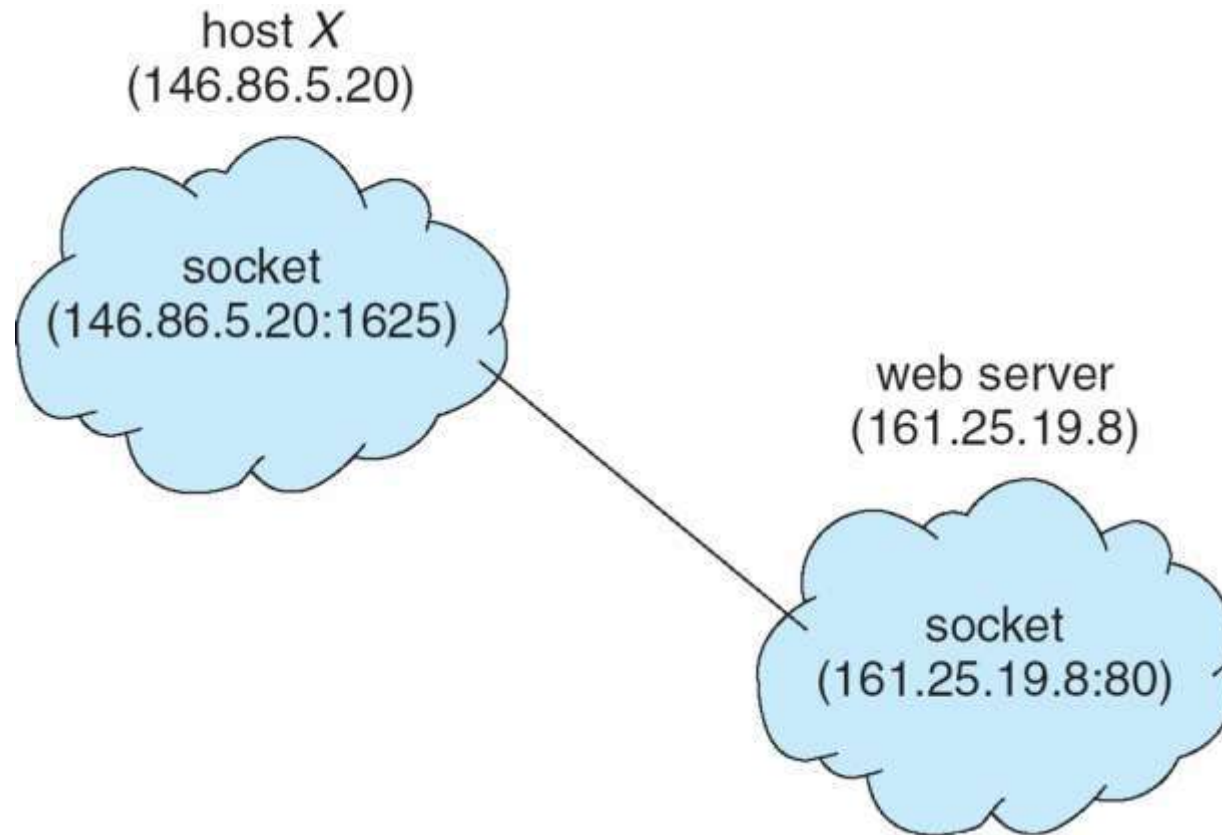
- Because one computer runs many services:
 - Web server
 - Email server
 - Database
- All share the same IP address.
- So we use **port numbers** to tell the system: “Which service should receive this message?”





Socket Communication

Socket = IP address + Port





Sockets in Java

- In Java, sockets are represented by the **Socket** class for clients and **ServerSocket** for servers.

Three types of sockets

1. Connection-oriented (TCP)

- TCP = Transmission Control Protocol.
- Like a **phone call**:
 - **you call, wait for the other side to pick up, talk, then hang up.**
- Guarantees that:
 - Data **arrives safely.**
 - Data **arrives in the right order.**
- Java uses **Socket** (client) and **ServerSocket** (server) for TCP.

Example analogy:

Sending a letter via **registered mail**. You know it will get there, and the order of letters is maintained.





Sockets in Java

2. Connectionless (UDP)

- UDP = **User Datagram Protocol**.
- Like **sending a postcard**:
 - You just drop it in the mailbox.
 - No guarantee it will arrive.
 - No guarantee of order.
- Java uses DatagramSocket for UDP.
- Faster than TCP but less reliable.

Example analogy:

Quickly shouting a message across a playground. Someone may hear it, or it may get lost in the noise.

3. MulticastSocket class— data can be sent to multiple recipients

- Sends data to multiple recipients at once (group communication).like conference call or TV broadcast:
- You speak once, Many people listen at the same time.
- **Useful for streaming, online games, or chat groups.**





Remote Procedure Calls

Normally, a program calls a function like this:

```
result = add(5, 3) # local function
```

- Everything happens **inside your computer**.
- You get the result immediately.

But what if the function is on **another computer** somewhere in your network?

- You can't just call it directly, your CPU can't reach over the network and run someone else's code.
- Without RPC, you'd have to manually:
 1. Open a network connection
 2. Send the function name and parameters
 3. Wait for a response
 4. Parse the response

This is error-prone, and complex.





Remote Procedure Calls

What RPC Does?

RPC makes calling a function on another computer look like calling a local function.

```
result = add_remote(5, 3)  
# looks like local, actually runs on a server
```

- Your program doesn't worry about networking or data formatting.
- RPC handles all the **communication behind the scenes**.





How RPC Works (Step by Step)

Step 1: Client Calls a Remote Function

- Your program calls `add_remote(5, 3)`.
- This is handled by a **client stub**, a piece of code that acts as a **local placeholder** for the remote function.

Step 2: Client Stub Packages the Call (Marshalling)

- Converts the function name (`add`) and parameters (`5, 3`) into a **message** that can travel over the network.

Step 3: Send the Message

- The message is sent **via TCP or UDP** to the server.

Step 4: Server Stub Unpacks the Message (Unmarshalling)

- The **server stub** receives the message.
- It extracts the function name and parameters so the server knows what to do.





How RPC Works (Step by Step)

Step 5: Server Runs the Function

- The actual function `add(5, 3)` runs **on the server**.
- Result (8) is generated.

Step 6: Server Sends Back Result

- The server stub **packages the result** and sends it back over the **network**.

Step 7: Client Receives Result

- The **client stub** receives the result, unpacks it, and your program gets: `result = 8`
- Your code sees it **as if it ran locally**.





How RPC Works (Step by Step)

Think of RPC like **ordering food from a restaurant**:

Step	RPC	Analogy
Call function	<code>add_remote(5,3)</code>	You call a restaurant to order food
Marshal	Package parameters	Write down your order
Send	Network sends message	Waiter takes order to kitchen
Server stub	Receives & unpacks	Chef reads the order
Run function	Server executes	Chef cooks the food
Return	Send result back	Waiter brings food to you
Client stub	Unpacks result	You eat the food

- From your perspective, it's just a normal function call.
- Behind the scenes, a lot of communication happens automatically.





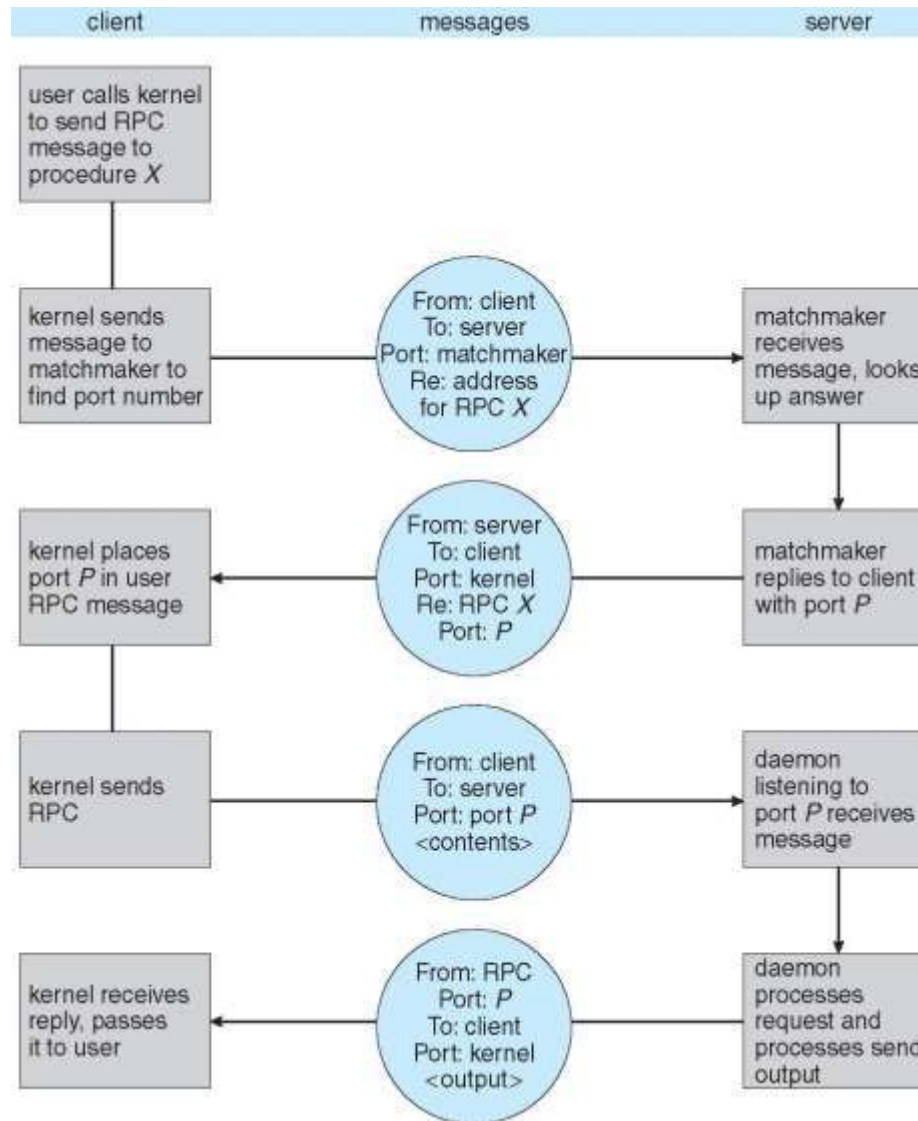
Remote Procedure Calls (Cont.)

- Data representation handled via **External Data Representation (XDL)** format to account for different architectures
 - **Big-endian** and **little-endian**
- Remote communication has more failure scenarios than local
 - Messages can be delivered ***exactly once*** rather than ***at most once***
- OS typically provides a rendezvous (or **matchmaker**) service to connect client and server





Execution of RPC





Pipes

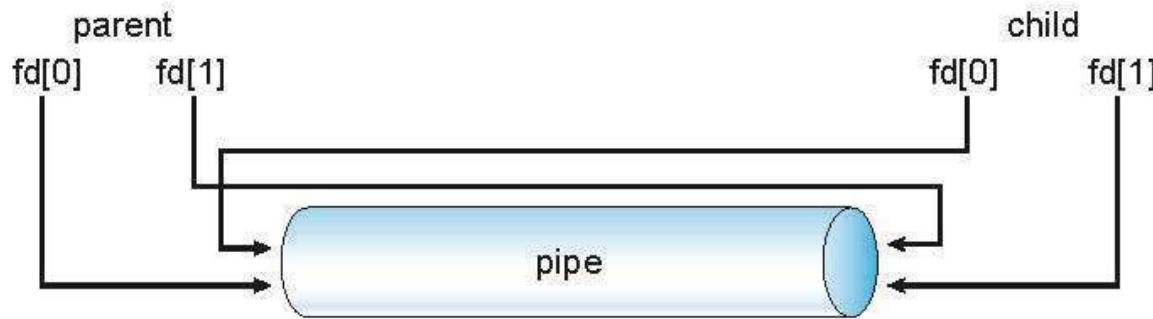
- Pipes are used whenever processes need to exchange data efficiently, **without** creating files or using network sockets, especially **on the same machine**.
- Issues:
 - Is communication unidirectional or bidirectional?
 - Must there exist a relationship (i.e., ***parent-child***) between the communicating processes?
- **Ordinary pipes** – cannot be accessed from outside the process that created it.
 - Typically, a parent process creates a pipe and uses it to communicate with a child process that it created.
- **Named pipes** – can be accessed without a parent-child relationship.





Ordinary Pipes

- Ordinary Pipes allow communication in standard producer-consumer style
- Producer writes to one end (the **write-end** of the pipe)
- Consumer reads from the other end (the **read-end** of the pipe)
- Ordinary pipes are therefore **unidirectional**
- **Require parent-child relationship between communicating processes**



- Windows calls these **anonymous pipes**
- See Unix and Windows code samples in textbook



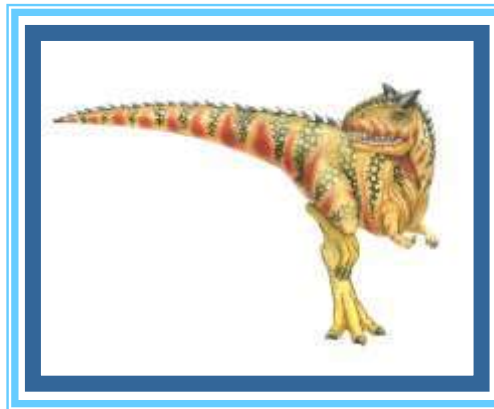


Named Pipes

- Named Pipes are more powerful than ordinary pipes
- Communication is **bidirectional**
- **No parent-child relationship** is necessary between the communicating processes
- **Multiple processes can use the same named pipe.**
- Provided on both UNIX and Windows systems
- Useful for **complex interprocess communication**, even between unrelated programs.



Chapter 4: Threads





Threads

- **What are threads?**
- A **thread** is a small unit of work inside a process.
- One process can have **multiple threads running at the same time**.

- **Why do we use threads? (Motivation)**
- Modern apps do many things at once, so they use threads.
- **Example: One app can:**
 - Update the screen
 - Download data
 - Handle network requests
- **Instead of doing these one by one, threads do them at the same time**





Benefits

- **Responsiveness** – If one task is stuck (like loading data), others still run , **Example:** App doesn't freeze while loading
- **Resource Sharing** – threads share resources of process, easier than shared memory or message passing



Why are threads easier?

- **1. Threads already share everything**
- Threads belong to the **same process**, so they automatically share:
 - Memory
 - Variables
 - Files
- No extra setup needed.





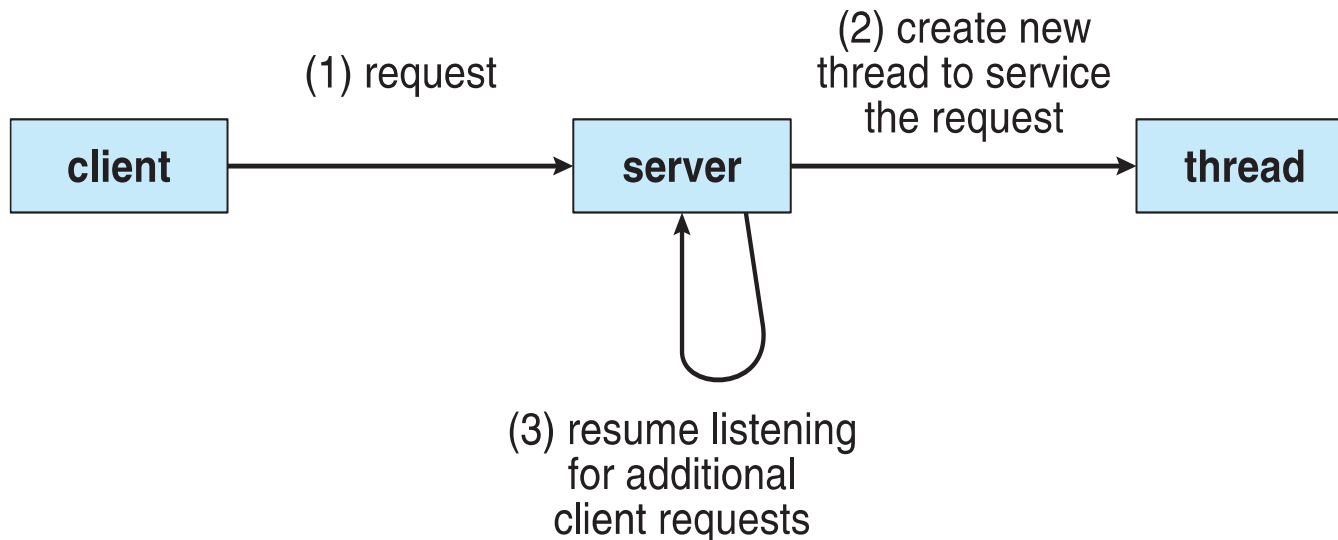
Benefits

- **2. Shared Memory (between processes) is harder**
- Different processes **do NOT** share memory by default.
- Kernel must:
 - Create shared memory space
 - Manage access (avoid conflicts)
 - Handle synchronization (locks, semaphores)
- **3. Message Passing is also complex**
- Processes communicate by **sending messages**:
- Example:
 - Process A → sends data to kernel
 - Process B → receives data from kernel
- Kernel must:
 - Set up communication channels
 - Send/receive messages
 - Handle delays, ordering, errors
- **Slower + more complicated**





Multithreaded Server Architecture



- **Client sends a request**

- A client (like a browser or app) sends a request to the server.
- Example: opening a website or calling an API.

- **Server creates a new thread**

- Instead of handling the request itself, the server **spawns a new thread** (a lightweight unit of execution).
- That thread is responsible for processing the client's request.
- **This allows the main server process to stay free.**

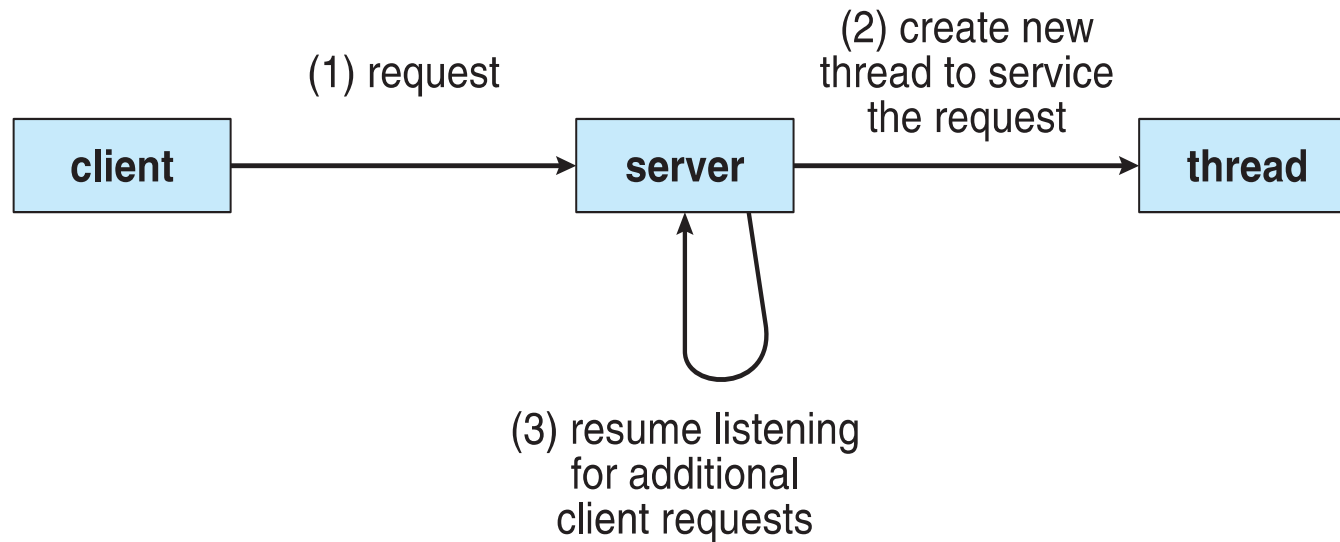
- **Server keeps listening**

- After creating the thread, the server immediately goes back to **listening for new incoming requests**.
- It doesn't wait for the current request to finish.





Multithreaded Server Architecture



Key idea:

Each request is handled **independently in its own thread**. This enables **concurrent processing** of multiple clients.

Why this is useful:

Concurrency – multiple users can be served at once

Responsiveness – new requests aren't blocked by slow ones

Better resource utilization





Multicore Programming

- Modern computers have **multiple cores (CPUs)**, so programs can run **multiple things at the same time**. This makes programs faster—but also harder to design.
- **Challenges programmers face:**
 - **Dividing work** → Break a program into smaller tasks
 - **Balancing load** → Make sure all cores do equal work
 - **Splitting data** → Share data across cores properly
 - **Data dependency** → Some tasks depend on others
 - **Testing & debugging** → Harder because many things run at once
- **Key Concepts**
- **A. Parallelism: Doing many things at the exact same time**
- Happens when you have **multiple cores**
- Each core works on a different task **simultaneously**
- **Example: Core 1** → watching a video & **Core 2** → downloading a file
➡ □ Both happen at the **same time**





Multicore Programming (Cont.)

■ Types of parallelism

■ 1. Data Parallelism

■ Same job, different pieces of data

- You **split the data**, Each core does the **same work** on its part

■ Example:

- You have 1000 numbers to add

- Core 1 → adds first 500
- Core 2 → adds next 500

- ➡□ Same task (adding), just different data parts

■ “Same work, split data”

■ Example: Searching

- Searching for a number in a big list
- Each core searches a different portion
- ➡□ Same search task, split data

2. Task Parallelism

Different jobs at the same time

- You **split the work itself**, Each core does a **different task**

Example:

- Core 1 → reads file
- Core 2 → processes data
- Core 3 → saves results

➡□ Different tasks happening **simultaneously**

“Different work, same time”

Example: Online Shopping App

Thread 1 → handles user input

Thread 2 → processes payment

Thread 3 → updates database

➡□ Different operations at the same time





Multicore Programming (Cont.)

■ Key Concepts

- B. Concurrency: Handling many things at once (but not truly at the same time)

- Tasks **take turns** quickly

- Gives the illusion everything is happening together

■ Example (single core):

- Task A runs for a bit

- Then Task B runs

- Then back to Task A

➡ □ They **share time**

Scheduler (on single core)

The **scheduler** decides which task runs next

- Switches between tasks very fast
- This is called **context switching**

Example:

- 0–1 ms → Task A
- 1–2 ms → Task B
- 2–3 ms → Task A

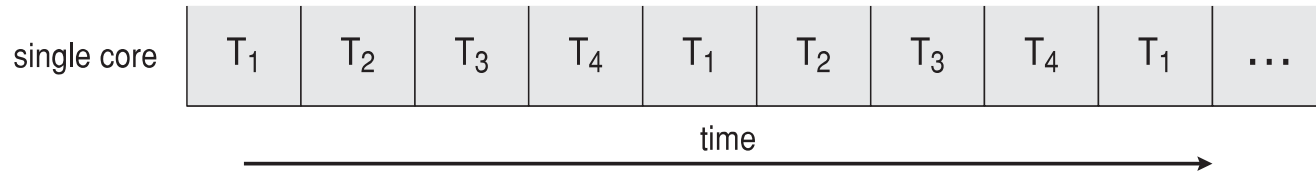
➡ □ Looks like both are running, but actually **one at a time**



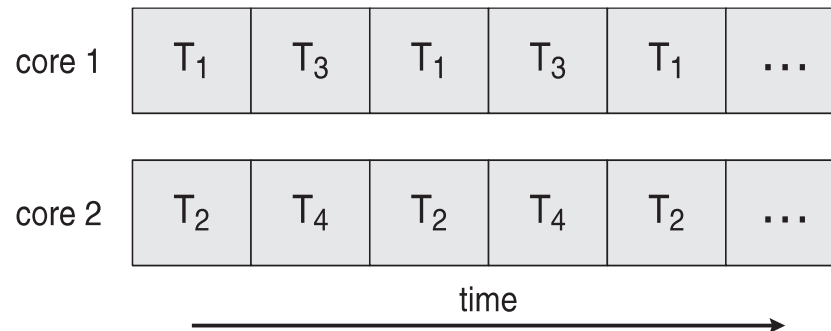


Concurrency vs. Parallelism

■ Concurrent execution on single-core system:

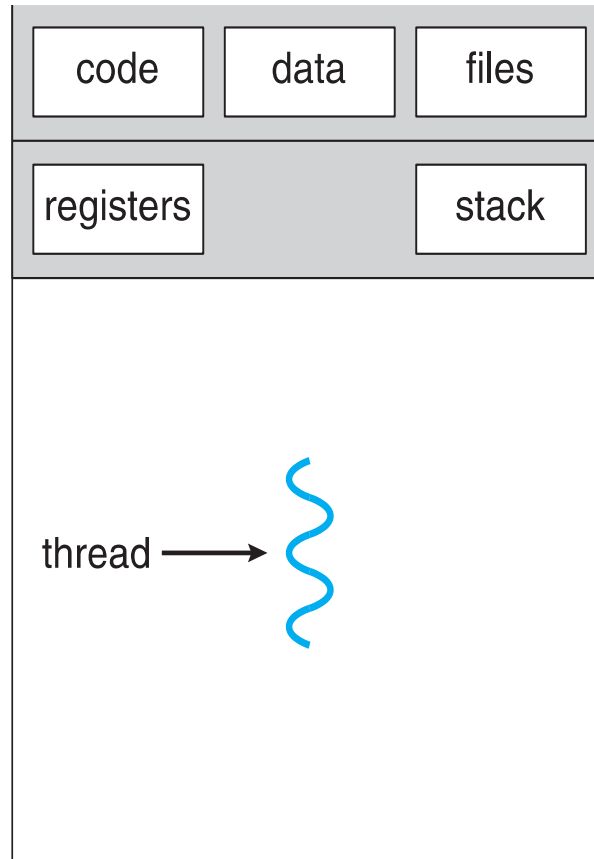


■ Parallelism on a multi-core system:

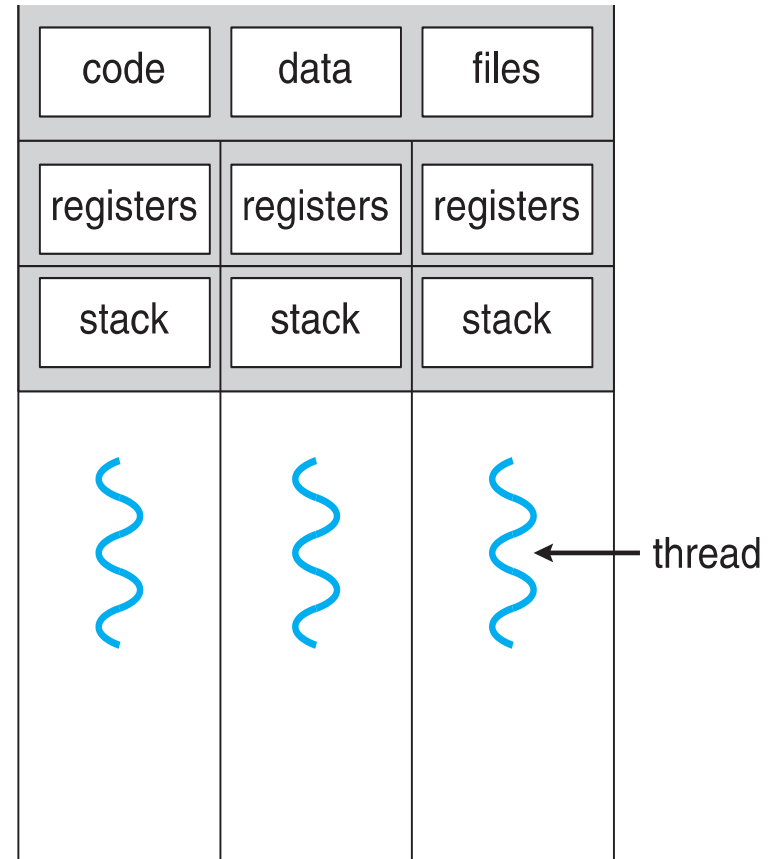




Single and Multithreaded Processes



single-threaded process



multithreaded process





Single and Multithreaded Processes

Single-threaded process: Only one task at a time

- One thread does everything
- Tasks are done **step by step (one after another)**

Example: Read file → then process → then save

If one task is slow → everything waits

Multithreaded process: Multiple tasks at the same time

Many threads inside one program

Each thread handles a **different part of the work**

Example:

- Thread 1 → read file
- Thread 2 → process data
- Thread 3 → save results

Faster and more efficient

Simply,

Single-threaded = one worker doing everything

Multithreaded = multiple workers doing different jobs together





Amdahl's Law

- “How much faster a program becomes when we add more CPU cores”

- S is serial portion
- N processing cores

$$\text{speedup} \leq \frac{1}{S + \frac{(1-S)}{N}}$$

- If a program is 75% parallel and 25% serial, using 2 cores makes it only about $1.6 \times$ faster (not $2 \times$).
- Even if you keep adding more and more cores:
Maximum speed = $1 / S$ (limited by serial part)
- So, Even a small serial portion can slow everything down (The more serial work you have, the less benefit you get from extra cores)
- What is SERIAL (cannot run in parallel)?
 - Reading input / loading data
 - Dependent operations (next step needs previous result)
- What's PARALLEL?
 - Processing large datasets
 - Image/video rendering
 - Searching multiple items

More cores help, but the serial part is the bottleneck





User Threads and Kernel Threads

There are **two ways to manage these threads**:

1. User threads

Your program itself handles the threads (not the OS)

- Managed by **user-level libraries** (not OS)
- Faster and lightweight
- Examples of libraries:
 - Windows threads
 - Java threads

Advantages:

Very **fast** & Easy to create many threads

Problem:

If one thread gets stuck- **All threads can stop**





User Threads and Kernel Threads

2. Kernel threads

The operating system (OS) manages threads

- Managed by the **operating system (kernel)**
- Slower but more powerful
- Used by most OS:
 - Windows, Linux, macOS, Solaris

Advantages:

If one thread stops, others **keep running**

Can use **multiple CPU cores** (real parallel work)

Disadvantage:

Slightly **slower** (OS involvement)





Multithreading Models

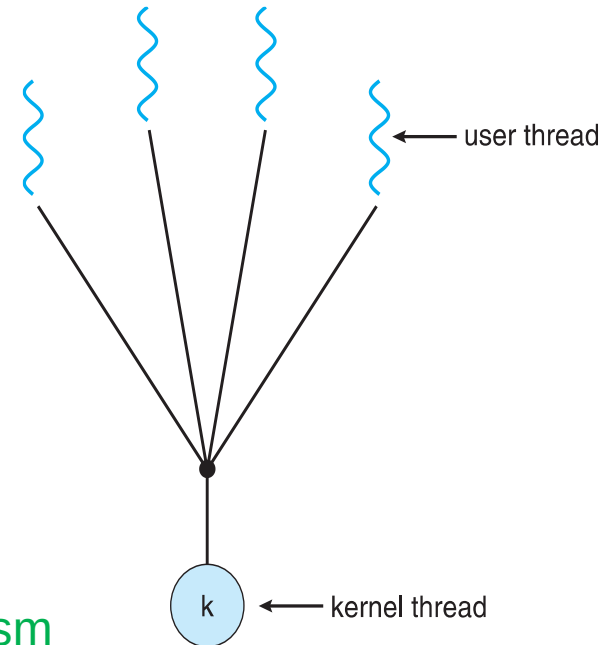
- Programs use **threads** to do multiple tasks at once.
- Who manages these threads?
 - Your program? (**user threads**)
 - The OS? (**kernel threads**)
- Multithreading Models:
“How are user threads connected to kernel threads?”
- Types:
 - Many-to-One
 - One-to-One
 - Many-to-Many





Many-to-One

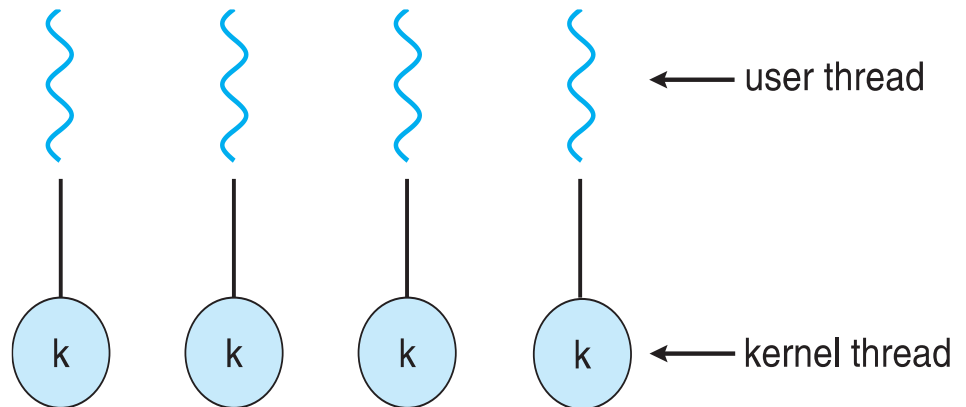
- Many user-level threads mapped to single kernel thread
 - (Many user threads → 1 kernel thread)
 - Only kernel threads use CPU cores
So only one thread runs at a time
- One thread blocking causes all to block
- Multiple threads may not run in **parallel** on muticore system because only one may be in kernel at a time
- Means, Even on a multicore system:
No parallel execution
Other threads must wait
- Idea: Lightweight and fast, but no true parallelism (one core only)
- Examples:
 - **Solaris Green Threads** (a user-level threading library)





One-to-One

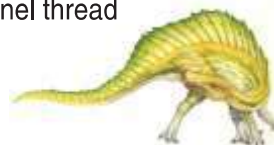
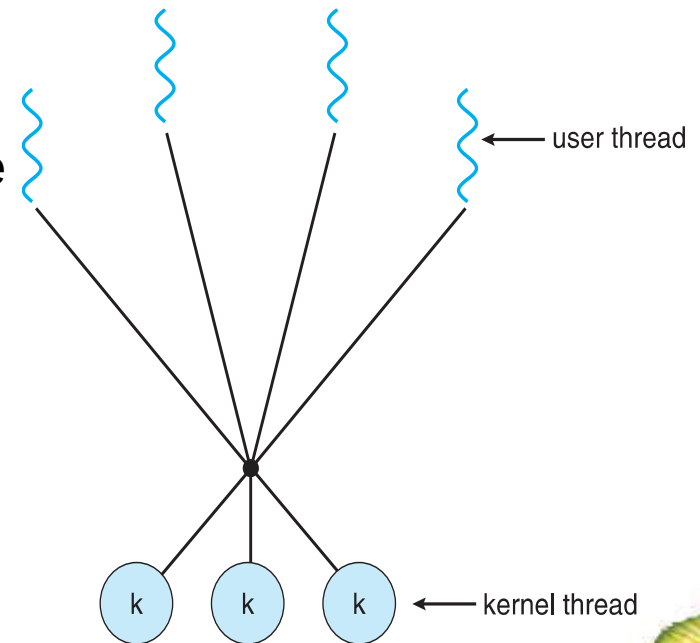
- Each user-level thread maps to kernel thread
 - ▶ (1 user thread → 1 kernel thread)
- Creating a user-level thread creates a kernel thread (**If one blocks → others still run**)
- More parallel work than Many-to-One
- Too many threads can slow system (high overhead)
- **Idea:** Fast but costly if too many threads
- Examples
 - Windows
 - Linux





Many-to-Many Model

- Allows many user level threads to be mapped to many kernel threads
 - (Many user threads → many kernel threads)
- Allows the operating system to create a sufficient number of kernel threads
- Balances performance and efficiency
- **Idea:** Flexible and efficient use of CPU
- Solaris prior to version 9
- Windows with the *ThreadFiber* package

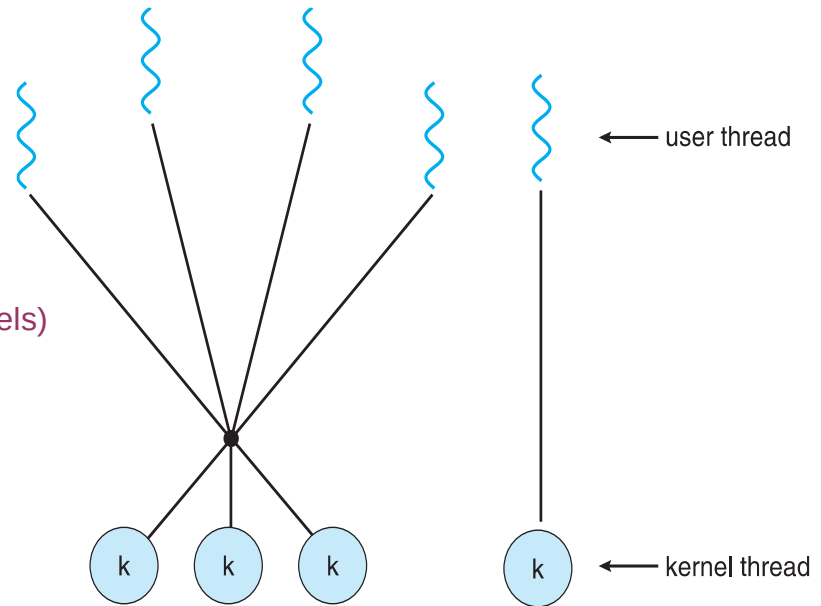




Two-level Model

- Similar to M:M, except that it allows a user thread to be **bound** to kernel thread **means You can assign a specific thread to a specific kernel thread**
- Important tasks can get dedicated CPU
- Idea: Flexible + priority for important tasks
- Examples
 - IRIX
 - HP-UX
 - Tru64 UNIX

(Unix systems that supported advanced threading models)





Multithreading Models

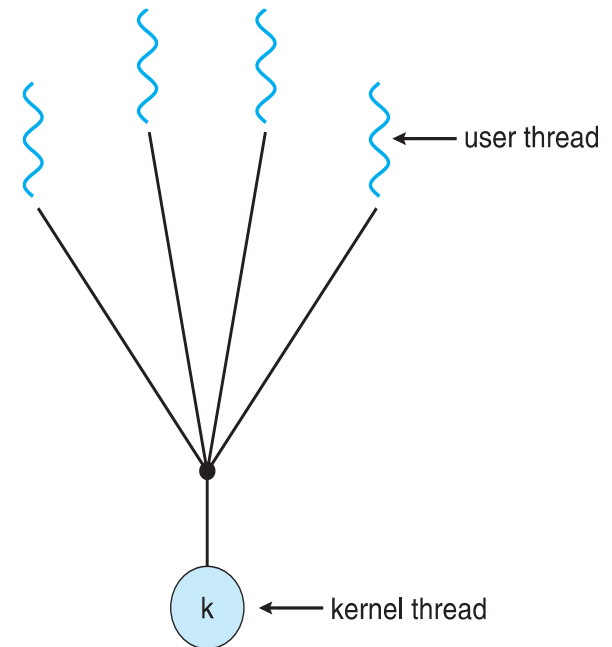
- Programs use **threads** to do multiple tasks at once.
- Who manages these threads?
 - Your program? (**user threads**)
 - The OS? (**kernel threads**)
- Multithreading Models:
“How are user threads connected to kernel threads?”
- Types:
 - Many-to-One
 - One-to-One
 - Many-to-Many
 - Two-level Model





Many-to-One

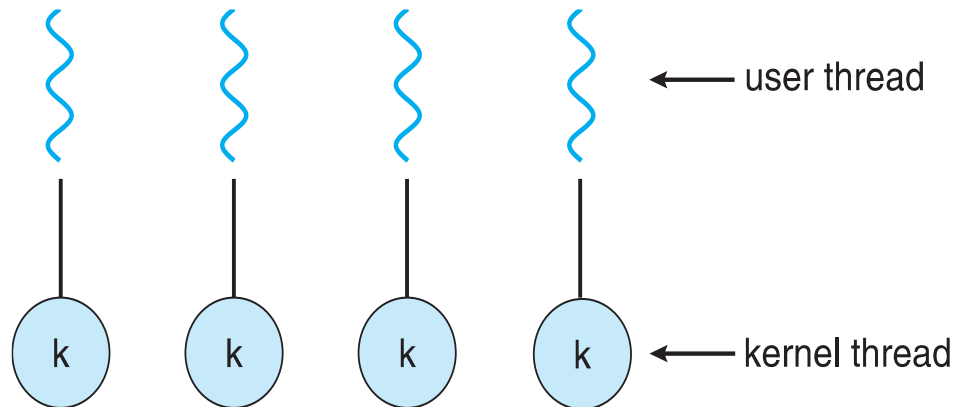
- Many user-level threads mapped to single kernel thread
 - (Many user threads → 1 kernel thread)
 - Only kernel threads use CPU cores
So only one thread runs at a time
- One thread blocking causes all to block
- Multiple threads may not run in **parallel** on muticore system because only one may be in kernel at a time
- Means, Even on a multicore system:
No parallel execution
Other threads must wait
- Idea: Lightweight and fast, but no parallelism (one core only)
- Examples:
 - **Solaris Green Threads** (a user-level threading library)





One-to-One

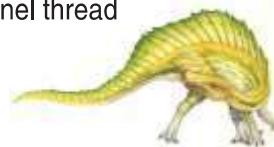
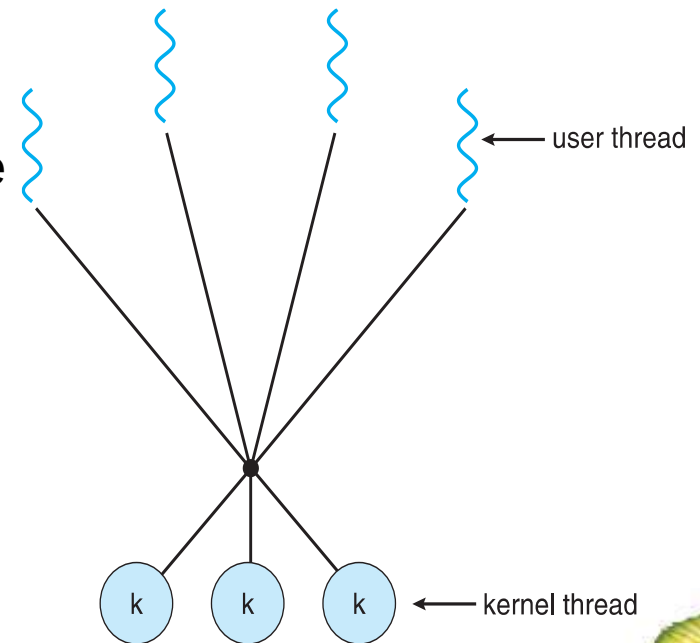
- Each user-level thread maps to kernel thread
 - ▶ (1 user thread → 1 kernel thread)
- Creating a user-level thread creates a kernel thread (**If one blocks → others still run**)
- More parallel work than Many-to-One
- Too many threads can slow system (high overhead)
- Idea: Fast but costly if too many threads
- Examples
 - Windows
 - Linux





Many-to-Many Model

- Allows many user level threads to be mapped to many kernel threads
 - (Many user threads → many kernel threads)
- Allows the operating system to create a sufficient number of kernel threads
- **Can improve performance, but can slow system (high overhead)**
- **Idea:** Flexible and efficient use of CPU
- Solaris prior to version 9
- Windows with the *ThreadFiber* package

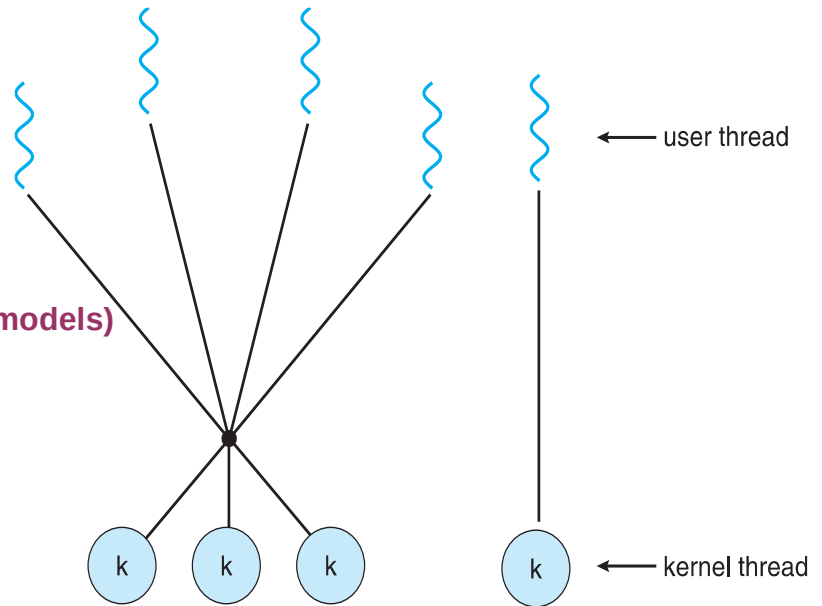




Two-level Model

- Similar to M:M, except that it allows a user thread to be **bound** to kernel thread **means You can assign a specific thread to a specific kernel thread**
- Important tasks can get dedicated CPU
- Idea: Flexible + priority for important tasks
- Examples
 - IRIX
 - HP-UX
 - Tru64 UNIX

(Unix systems that supported advanced threading models)





Thread Libraries

- Thread libraries give programmers tools (APIs) to create and manage threads.
- There are two main types:
 - **User-level threads:** Managed entirely in user space (by the application, not the OS).
 - **Kernel-level threads:** Managed and supported directly by the operating system.





Pthreads

- **Pthreads (POSIX threads)** are a standard way to create and manage threads in C program.
- **Key points:**
 - This can be **user-level or kernel-level**.
 - They follow a **POSIX (IEEE 1003.1c)** standard.
 - They are an **API (a set of rules/functions)**, as it defines **how threads should behave**, but different systems implement it differently.
 - Commonly used in **UNIX systems** like Linux, macOS, and Solaris.





Pthreads

- **How to use:**
- `#Include <pthread.h>` in your C code.
- **Key Thread Functions:**
- `pthread_create()` – Start a new thread running a function.
 - **What it does:** Creates a new thread that **runs a specific function** in parallel with your main program.
- `pthread_join()` – Wait for a thread to finish.
 - **What it does:** Stops the current thread **until another thread finishes**.
- `pthread_exit()` – End the current thread.
 - **What it does:** Ends the thread that calls it, **without stopping the whole program**.
- `pthread_detach()` – Let the system clean up a thread automatically when it finishes.
 - **What it does:** Tells the system to automatically **free resources when a thread ends**.





Java Threads

- Java threads are managed by the JVM
- Typically implemented using the **threads model provided by underlying OS**
- Two ways to create threads:
 - Extend the Thread class
 - Implement the Runnable interface

Technique	API Methods
Extend Thread	<code>start()</code> , <code>run()</code> , <code>join()</code> , <code>sleep()</code>
Implement Runnable	<code>start()</code> , <code>run()</code>





```
class MyThread extends Thread {  
    public void run() {  
        System.out.println("Thread using Thread class is running!");  
    }  
}  
  
class MyTask implements Runnable {  
    public void run() {  
        System.out.println("Thread using Runnable is running!");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        MyThread t1 = new MyThread();  
        Thread t2 = new Thread(new MyTask());  
  
        t1.start();  
        t2.start();  
  
        System.out.println("Main method finished");  
    }  
}
```

Output (order may vary because threads run in parallel):

Main method finished

Thread using Thread class is running!

Thread using Runnable is running!





Implicit Threading

- **Explicit threading** → the programmer writes code to **create, start, and manage threads** manually using `Thread` (built-in class), `Runnable`, or `pthread_create()`.
- **Implicit threading** → moves this responsibility to the **compiler or runtime**. The programmer just writes the task logic, and the system decides **how many threads to use and how to run them**.
- Good for **programs with many threads**, because manually managing lots of threads gets hard and error-prone.





Implicit Threading

Example:

Thread Pools – system reuses a set of threads for tasks.

- **What it is:** A **set of threads (workers)** that wait for tasks.
- **How it works:** You give tasks → system assigns them to **available threads automatically**.
- **Advantages:**
 - **Faster** than creating a new thread every time.
 - **Control:** can limit number of threads in the pool.
- **Example (Java):** ExecutorService (API)





Implicit Threading

Example:

OpenMP – mark loops as parallel, compiler handles threads.

- **Programmer just marks which loops or code blocks can run in parallel.**
- **Compiler automatically makes threads.**

```
#pragma omp parallel for
for(i=0; i<N; i++) {
    c[i] = a[i] + b[i];
}
```

Without OpenMP
(normal loop):
i=0 -> c[0]=6
i=1 -> c[1]=8
i=2 -> c[2]=10
i=3 -> c[3]=12

With OpenMP, the **system can do this at the same time:**

- Core 1: i=0 → c[0]=6
- Core 2: i=1 → c[1]=8
- Core 3: i=2 → c[2]=10
- Core 4: i=3 → c[3]=12

Result is the same, but loop finishes faster because multiple cores are working together.

- **#pragma omp parallel for:** “This **for loop** can run in **parallel** on multiple CPU cores.”
- **You don’t create threads manually;** OpenMP does it.
- Each loop iteration (i) is **independent** and runs as a **separate task**.
- OpenMP automatically **assigns iterations to available cores**.





Implicit Threading

Example:

Grand Central Dispatch (GCD)

- You put **tasks in a queue**, system runs them on available threads (system schedules tasks on CPU cores automatically.)
- Two types of queues:
 - **Serial** – one task at a time (FIFO).
 - **Concurrent** – multiple tasks at a time.
- System handles **thread management automatically**.





Implicit Threading

- **Explicit threading** → the programmer writes code to **create, start, and manage threads** manually using `Thread` (built-in class), `Runnable`, or `pthread_create()`.
- **Implicit threading** → moves this responsibility to the **compiler or runtime**. The programmer just writes the task logic, and the system decides **how many threads to use and how to run them**.
- Good for **programs with many threads**, because manually managing lots of threads gets hard and error-prone.





Implicit Threading

Example:

Thread Pools – system reuses a set of threads for tasks.

- **What it is:** A **set of threads (workers)** that wait for tasks.
- **How it works:** You give tasks → system assigns them to **available threads automatically**.
- **Advantages:**
 - **Faster** than creating a new thread every time.
 - **Control:** can limit number of threads in the pool.
- **Example (Java):** `ExecutorService` (API)





Implicit Threading

Example:

OpenMP – mark loops as parallel, compiler handles threads.

- **Programmer just marks which loops or code blocks can run in parallel.**
- **Compiler automatically makes threads.**

```
#pragma omp parallel for
for(i=0; i<N; i++) {
    c[i] = a[i] + b[i];
}
```

Without OpenMP
(normal loop):
i=0 -> c[0]=6
i=1 -> c[1]=8
i=2 -> c[2]=10
i=3 -> c[3]=12

With OpenMP, the **system can do this at the same time:**

- Core 1: i=0 → c[0]=6
- Core 2: i=1 → c[1]=8
- Core 3: i=2 → c[2]=10
- Core 4: i=3 → c[3]=12

Result is the same, but loop finishes faster because multiple cores are working together.

- **#pragma omp parallel for:** “This **for loop** can run in **parallel** on multiple CPU cores.”
- **You don’t create threads manually;** OpenMP does it.
- Each loop iteration (i) is **independent** and runs as a **separate task**.
- OpenMP automatically **assigns iterations to available cores**.





Implicit Threading

Example:

Grand Central Dispatch (GCD)

- You put **tasks in a queue**, system runs them on available threads (system schedules tasks on CPU cores automatically.)
- Two types of queues:
 - **Serial** – one task at a time (FIFO).
 - **Concurrent** – multiple tasks at a time.
- System handles **thread management automatically**.





Threading Issues

Multithreading improves performance, but it introduces problems because **threads share memory and run concurrently**.

Key Issues:

- Race conditions
- Deadlocks
- Synchronization complexity
- Resource sharing conflicts





Threading Issues

- **Race Conditions**

Happen when multiple threads modify shared data at the same time, causing unpredictable and inconsistent results.

Example: Two threads updating the same variable may overwrite each other's changes.

- **Deadlocks**

Occur when threads wait on each other's resources, causing all of them to get stuck indefinitely.

Example: Thread A waits for B's resource while B waits for A's.

- **Synchronization Complexity**

Coordinating threads using locks is difficult as too little causes errors, too much reduces performance.

- **Resource Sharing Conflicts**

When threads use shared resources **without control**, it can lead to data corruption, crashes, or incorrect behavior.

Example: Two threads writing to a file at the same time can mix data.





Semantics of fork() and exec()

- These system calls are important for **process creation and program execution**, especially in multithreaded systems.

fork()

- Creates a new process (child) by duplicating the parent process
- After fork(), both parent and child continue execution

In multithreaded programs:

- There are two possible behaviors depending on the system:
 - **Duplicate all threads** (older/strict UNIX behavior)
 - **Duplicate only the calling thread** (modern/common behavior)

Most modern UNIX/Linux systems:

- Only the **thread that called fork() is copied**
- Other threads are not replicated in the child process

Why only one thread is copied?

- Copying all threads is complex and unsafe
- The child usually calls exec() immediately after fork()

Some UNIX systems provide two variants:

- One that forks the whole process (all threads)
- One that forks only the calling thread





Semantics of fork() and exec()

exec()

- Replaces the current process with a new program
- Entire process memory is overwritten

Important in multithreading:

- All threads are destroyed
- The new program starts as a **single-threaded process**

SO,

- ▶ **fork()**: Create a child process
- ▶ **exec()**: Replace it with a new program





Thread Cancellation

- Thread cancellation means stopping a thread before it finishes execution.

Why needed?

- **Example:** In a search task, if one thread finds the result, other threads can be stopped to save resources.

Types of Cancellation

1. Asynchronous Cancellation

- Thread is stopped immediately by another thread
- Unsafe because it may:
 - Leave shared data inconsistent
 - Skip releasing locks or memory
- So, is Fast but dangerous





Thread Cancellation

Types of Cancellation

2. Deferred Cancellation

- Thread is not killed directly
- It checks periodically if it should stop
- Happens at **cancellation points** (e.g., `sleep()`, `wait()`, `pthread_testcancel()`)
- Safe and widely used





Signal Handling in Multithreading (UNIX)

- A signal is a notification sent to a process when an event occurs (like an interrupt).

Signal Lifecycle

- **Generated** by an event
- **Delivered** to a process/thread
- **Handled** by a signal handler

1. Generated by an event

- A signal is created when something important happens in the system.

Examples:

- Division by zero → error occurs
- Illegal memory access
- So first: **an event triggers a signal**





Signal Handling in Multithreading (UNIX)

- **2. Delivered to a process/thread**
- The operating system then decides where to send that signal.

Simply:

- The signal is **sent to the running program (process)**
- In multithreading, it may go to a **specific thread**
- So, **OS delivers the signal**

3. Handled by a signal handler

- Once received, the signal must be processed.
 - This is done by a **signal handler**, which is just a function that reacts to the signal.
- So, **the program responds to the signal**





Signal Handling in Multithreading (UNIX)

Types of Signal Handlers

1. Default Handler (OS-controlled)

- Automatically provided by the operating system
- Runs if the programmer does nothing
- **Example:**
- Ctrl + C gives SIGINT (Signal Interrupt) that means terminates the program
- That behavior is the default handler

2. User-defined Handler (programmer-controlled)

- Programmer writes custom code to handle the signal
- Overrides the default behavior
- **Example:**

Instead of crashing on Ctrl + C, the program might:

- Save work
- Show a message
- Exit safely

For example:

- Ignore Ctrl + C
 - Ask "Are you sure?"
 - Save work before exiting
 - Clean up resources before stopping
- That's done using signal handling
`signal(SIGINT, handler)` in
C/C++





Signal Handling in Multithreading (UNIX)

Signal Handling Problem in Threads

- In multithreading, a key question is:
Which thread should receive the signal?

Signal Delivery Options

- To a **specific thread**
- To **all threads in the process**
- To **some selected threads**
- To a **dedicated signal-handling thread**

Common practice:

- Block signals in most threads
- Assign one thread to handle all signals





Signal Handling in Multithreading (UNIX)

Signal Types

1. Synchronous Signals

- Caused by the thread itself (e.g., divide by zero)
- Delivered to the same thread

2. Asynchronous Signals

- Come from external events (e.g., Ctrl+C)
- Can be delivered to:
 - Any thread
 - Specific thread
 - Dedicated handler thread

Key POSIX / Pthreads Insight

- Cancellation request does NOT stop thread immediately
- Thread may ignore it if cancellation is disabled
- Default mode: **Deferred cancellation**
- Actual cancellation happens at **safe points**
- Cleanup handlers run before termination





Thread-Local Storage

- Thread-Local Storage (TLS) is a mechanism that allows each thread in a process to have its own **private copy of a variable**, even though threads normally share memory.

Problem

- Since threads share the same memory space, using shared variables can lead to:
 - ▶ Race conditions, Data corruption & Need for synchronization

Solution: TLS

- TLS gives **each thread its own independent copy of data**
- So:
- One thread's data changes do not affect others
- No need for locks or synchronization
- Even though threads belong to the same process, TLS ensures:
“Same variable name, but separate storage per thread”





Scheduler Activations

- Scheduler Activations is an advanced mechanism used to improve coordination between **user-level threads** and **kernel-level threads**.

Problem

- In multithreading models (**M:M** and **two-level models**), there is a mismatch between:
 - User-level threads (managed by the thread library)
 - Kernel-level threads (managed by the OS)

This causes:

- Inefficient scheduling
- Threads blocking unnecessarily
- Poor CPU utilization





Scheduler Activations

Why Mismatch Happens?

- **Two schedulers:** User threads are managed by a thread library, while kernel threads are managed by the OS → they act independently.
- **Limited visibility:** The kernel only sees kernel threads, not user-level threads inside a process.
- **Blocking issue:** If one kernel thread blocks, other ready user threads may still be stuck.
- **Resource imbalance:** User threads may be ready, but there may not be enough kernel threads to run them.

The kernel and user thread library manage threads separately, so their scheduling is **not always in sync**.





Scheduler Activations

Key Idea (Solution)

- Scheduler Activations enable **cooperation between the kernel and the user-level thread library**.
- **The kernel informs the user-level scheduler about important events so it can adjust thread execution.**

How It Works

- Kernel sends **upcalls (activations)** to the thread library when:
 - A thread blocks
 - A thread finishes
 - CPU becomes available
- The user-level thread scheduler reacts and:
 - Reschedules threads
 - Adjusts execution dynamically





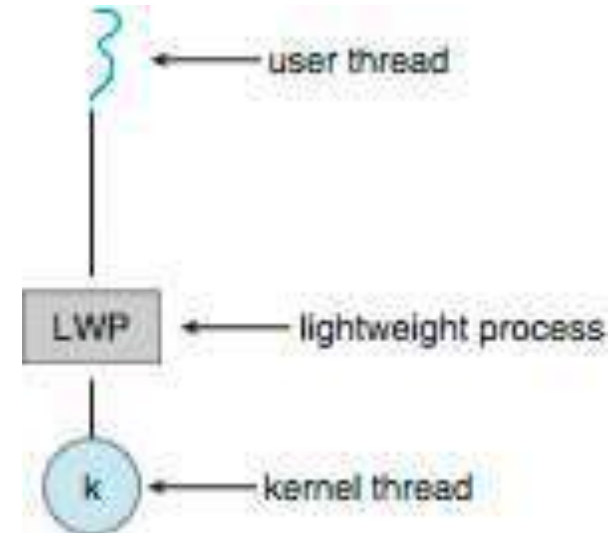
Scheduler Activations

To support this system, an intermediate structure is used:
LWP (Lightweight Process)

- Acts as a **virtual processor**
- Each LWP is attached to a kernel thread
- User-level threads are scheduled onto LWPs

So:

- User threads → run on LWPs
- LWPs → map to kernel threads



Key Design Question

- How many LWPs should be created?
- Too few → poor parallelism
- Too many → overhead

Benefit of Scheduler Activations

- Better CPU utilization
- Avoids unnecessary blocking
- Efficient mapping of user threads to kernel threads
- Dynamic adaptation to system load



Operating System Examples

- Windows Threads
- Linux Threads





Windows Threads

- Windows uses the **Windows API** and follows a **one-to-one model**

Each user thread = one kernel thread

What each thread contains (Thread Context)

- **Thread ID**: A unique identifier for the thread.
 - **CPU register set**: Values currently in CPU registers (e.g., program counter, stack pointer). This is the exact point where the thread is executing.
 - **User-mode stack**: Used for function calls, local variables, and normal program execution.
 - **Kernel-mode stack**: Used when the thread enters the OS kernel (e.g., system calls, interrupts).
 - **Private data (libraries/DLLs)**: Thread-specific data used by runtime libraries or dynamic libraries.
-
- All of this together is called the **thread context**
 - A thread context is everything needed so the OS can pause a thread and later resume it exactly where it stopped.





Windows Threads

Main Data Structures

ETHREAD (Executive Thread Block)

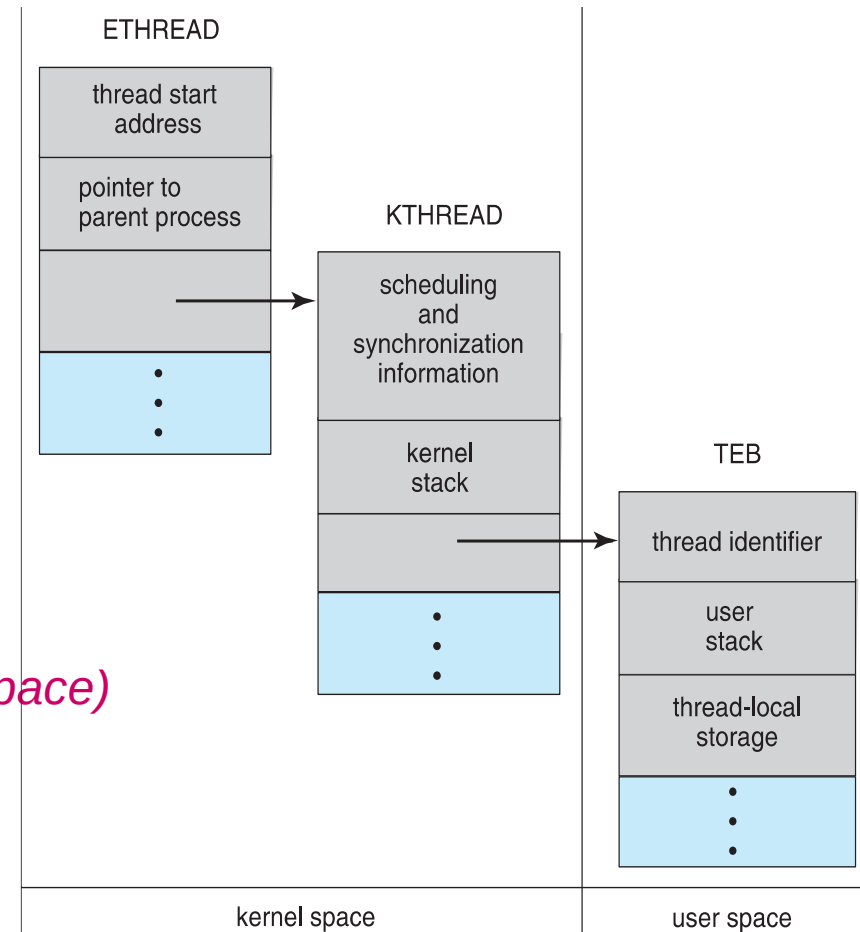
- Links thread to its process
- Points to KTHREAD

KTHREAD (Kernel Thread Block)

- Contains scheduling info
- Kernel stack
- Sync information

TEB (Thread Environment Block) (*user space*)

- Thread ID
- User stack
- Thread-local storage (TLS)
- **Simply, Windows keeps thread info in both user space (TEB) and kernel space (ETHREAD/KTHREAD).**





Linux Threads

- In Linux:
There is no strict difference between *process* and *thread*
Both are called **tasks**

1. Thread creation: clone()

- Linux creates threads using the system call: **clone()**
- This is different from fork() because:
 - **fork()** → creates a separate process
 - **clone()** → can create a thread that **shares resources**

2. What clone() does

- clone() decides what is shared between parent and child:
- A child task can share:
 - Memory
 - Files
 - File system info
 - Signal handlers
- OR it can be separate (like a process)





Linux Threads

- These flags control whether it behaves like a **thread or process**

flag	meaning
CLONE_FS	File-system information is shared.
CLONE_VM	The same memory space is shared.
CLONE_SIGHAND	Signal handlers are shared.
CLONE_FILES	The set of open files is shared.

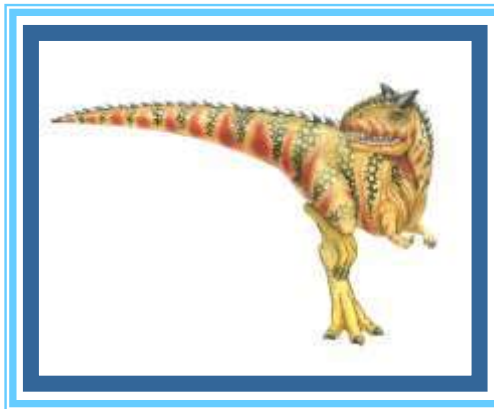
3. task_struct

- Main data structure in Linux for a process/thread
- Every task (thread/process) has a task_struct
- It stores:
 - Scheduling info
 - Memory info
 - File info
 - Signal info
- If resources are shared → points to same data
If not shared → has its own copy

In Linux, threads are called tasks. They are created using `clone()`, which decides what resources are shared using flags like `CLONE_VM` and `CLONE_FILES`. All tasks are represented using `task_struct`.



Chapter 5: Process Synchronization





Background

Process Synchronization

- In a computer system, multiple processes can run at the same time (concurrently).
 - A process may be stopped in the middle of its work and resumed later.
 - When processes share data, they can accidentally overwrite or corrupt it.
- This leads to **data inconsistency**.

Why synchronization is needed?

- We need rules or mechanisms to make sure processes work in an organized way when sharing data.





Producer-Consumer Problem

- **Example: Producer–Consumer Problem**
- There is a buffer (storage area) shared by two processes:
 - **Producer:** adds items to the buffer
 - **Consumer:** removes items from the buffer
- We use a variable called **counter** to track how many items are in the buffer:
 - Starts at **0**
 - Producer **adds 1** after producing an item
 - Consumer **subtracts 1** after consuming an item





If producer and consumer access buffer without control?

Situation	What Goes Wrong
Produce when buffer is full	Old data overwritten
Consume when buffer is empty	Invalid or garbage data

Key idea

No proper coordination → unpredictable result.





Race Condition

Example

- **counter = 5**
- **Producer increases it → should make 6**
- **Consumer decreases it → should make 4**
- **But because both run together without control:**
 - They both read 5
 - Producer writes 6
 - Consumer writes 4

Final problem

- The final value becomes incorrect because **updates overlap**.

This is why **synchronization** is required, to ensure

- Producer waits when buffer is full
- Consumer waits when buffer is empty





Critical Section Problem

- In a system with multiple processes, each process has a **critical section**, which is the **part of code where it accesses or modifies shared resources** (e.g., variables, files, tables).
- If multiple processes execute their critical sections at the same time, it can lead to **data inconsistency**. **Therefore, a proper protocol** is needed to control access.





Critical Section

Process Structure

- Each process is divided into:
 - **Entry Section** → requests permission to enter critical section
 - **Critical Section** → accesses shared resources
 - **Exit Section** → releases the critical section
 - **Remainder Section** → other work

Goal

- Design a solution so that **only one process executes in the critical section at a time** while ensuring fairness and efficiency.

do {

entry section

critical section

exit section

remainder section

} while (true);

General structure of
process P_i





Solution to Critical-Section Problem

1. **Mutual Exclusion** - If process P_i is executing in its critical section, then no other processes can be executing in their critical sections
2. **Progress** - If no process is in the critical section and some want to enter, the selection of the next process cannot be delayed indefinitely.
3. **Bounded Waiting** - No process waits forever; everyone gets a fair turn within a limit.

Why this is important? It prevents starvation (where a process keeps waiting forever).

Key Idea

- The critical section problem ensures **safe and controlled access to shared resources in concurrent systems.**





Critical-Section Handling in OS

- There are two ways an operating system kernel handles running processes:

1. Preemptive Kernel

- The OS can **stop (interrupt)** a running process even in kernel mode.
- CPU can be taken away anytime.
- **Simply, The OS can interrupt a process anytime.**

Example:

- Process A is updating a file.
- OS stops A in the middle.
- Process B starts updating the same file.
- Both may work on the same data at the same time → **race condition can happen**





Critical-Section Handling in OS

- There are two ways an operating system kernel handles running processes:

2. Non-preemptive Kernel

- Once a process enters kernel mode, it **runs fully until it finishes, or gives up CPU.**
- It cannot be interrupted in between.

Example:

- Process A is updating a file in kernel mode.
- It must finish the update fully.
- Process B waits outside until A is done.
- Since only one process is inside at a time, **no shared data gets corrupted → no race condition.**





Peterson's Solution

- Good algorithmic description of solving the problem
- Works only for **2 processes**
- **Shared variables:**
 - $\text{flag}[i] \rightarrow$ “I want to enter”
 - $\text{turn} \rightarrow$ “whose turn is it?”
- **How it works (intuition):**
 - Process says: “*I want to enter*” $\rightarrow \text{flag}[i] = \text{true}$
 - Gives turn to other $\rightarrow \text{turn} = j$
- Waits if:
 - other also wants ($\text{flag}[j] = \text{true}$)
 - AND it's other's turn
- So only one enters.





Algorithm for Process P_i

```
do {  
    flag[i] = true;  
    turn = j;  
    while (flag[j] && turn == j);  
        critical section  
    flag[i] = false;  
        remainder section  
} while (true);
```





Mutex Locks

- Peterson is theoretical. Real systems use **mutex locks**.
- Mutex Lock Protect a critical section by first **acquire()** a lock then **release()** the lock
 - Boolean variable indicating if lock is available or not

Idea:

- A lock = key to a room
- **acquire(lock)**
→ enter critical section
- **release(lock)**
 - If locked → wait
 - If free → enter
- **Problem: Busy waiting (spinlock)**
Process keeps checking again and again → wastes CPU





Semaphore

- Synchronization tool that provides more sophisticated ways (than Mutex locks) for process to synchronize their activities.
- Semaphore **S** – integer variable
- Can only be accessed via two indivisible (atomic) operations
 - wait(S) → decrease
 - signal(S) → increase

wait(S)- “Wait until resource available”

- while (S <= 0) wait
S--
- **signal(S)**- “I’m done, someone else can go”
- S++

Type of Semaphore	Values	Use
Binary	0 or 1	Same as mutex lock
Counting	0,1,2,3...	Multiple resources





Deadlock and Starvation

Deadlock – Deadlock happens when **two or more processes are stuck forever**, each waiting for something that only the other can release.

Example:

- Process P0 locks S, then waits for Q
- Process P1 locks Q, then waits for S
- Both are waiting forever → **deadlock**

Starvation – Starvation happens when a process **keeps waiting for a very long time (maybe forever)** because it is never given a chance to run.

- It stays stuck in the waiting queue and never gets CPU/resources.

Priority Inversion – Scheduling problem happens when:

- A **low-priority process holds a resource (lock)**
- A **high-priority process needs that resource but must wait**

Fix: Priority Inheritance Protocol

- The low-priority process temporarily gets higher priority so it can finish faster and release the lock.





Readers-Writers Problem

■ This is a classic Operating Systems problem.

■ **There is:**

- A **shared resource** (like a file, database, memory)
- Two types of processes:
 - **Readers**
 - **Writers**

■ **Rules of the Problem**

1. **Many readers can read at the same time.**
2. **Only ONE writer can write at a time.**
3. **When a writer is writing:**
 1. No readers allowed.
 2. No other writers allowed.
4. **A writer must wait if even 1 reader is reading.**



■ **Example Scenario**

1. Reader1 enters → locks writer access
2. Reader2 enters → allowed
3. Reader3 enters → allowed
4. Writer tries → waits
5. All readers leave
6. Writer enters





■ How Do We Solve It?

■ We use a synchronization tool called: **Semaphore**

■ A semaphore (Locking mechanism) - variable used to control access to shared resources.

■ Variables Used:

1. $x \rightarrow$ protects readercount
2. $y \rightarrow$ protects shared resource
3. **readercount** $\rightarrow$ number of readers inside





What the Program Does?

Program Starts

$x = 1$
 $y = 1$
 $readercount = 0$

If Reader Comes

Step 1:

Lock x
Increase readercount

Step 2:

If $readercount == 1$
→ Lock y
→ Block writers

Step 3:

Unlock x

Step 4:

Reader reads

When Reader Leaves

Step 1:

Lock x
Decrease readercount

Step 2:

If $readercount == 0$
→ Unlock y
→ Allow writers

Step 3:

Unlock x

If Writer Comes

Step 1:

Try to lock y
If $y == 0$ → WAIT
If $y == 1$ → Lock y

Step 2:

Writer writes

Step 3:

Unlock y





Important Observations

- **x becomes 0** only when a reader is updating readercount
- **y becomes 0**:
 - When first reader enters
 - When writer is writing
- Multiple readers can read when $y = 0$
- Writer can write only when $y = 1$ and $\text{readercount} = 0$





Condition Summary

1. Resource Busy Condition

$y == 0$

Means: Someone is using resource

2. Readers Active Condition

$readercount > 0$

Means: Readers inside & Writer must wait

3. Resource Free Condition

$y == 1$ AND $readercount == 0$

Means: Anyone can enter (Reader or Writer)

4. Reader Waiting Condition

Reader waits only when:

Writer is writing ($y == 0$ AND $readercount == 0$)

How It Repeats

Program creates multiple reader and writer threads.

For each:

Reader: Check, Enter, Read & Exit

Writer: Check, Enter, Write & Exit

This continues until all threads finish.





Dining-Philosophers Problem

Dining Philosophers Problem

- 5 philosophers sit around a table.
- Each philosopher **alternates between thinking and eating**.
- There is **one bowl of rice (shared resource)**.
- Between each pair of philosophers, there is **one chopstick**.

Rules

- To eat, a philosopher needs **two chopsticks**:
 - one on the left
 - one on the right
- They pick them up **one at a time**
- After eating, they **put both chopsticks back**



Readers – Writer Problem

Initial values:

- $x = 1$
- $y = 1$
- $\text{readercount} = 0$

Example Scenario

R1 enters → R2 enters → R1 leaves → R2 leaves → W1 writes

Event	x	y	readercount	Explanation
1. Initial State	1	1	0	System start
2. R1 locks x	0	1	0	R1 updating readercount
3. R1 increments readercount	0	1	1	First reader
4. R1 locks y	0	0	1	Blocks writers
5. R1 unlocks x	1	0	1	Other readers can update
6. R1 reading	1	0	1	Reading in progress
7. R2 locks x	0	0	1	R2 updating count
8. R2 increments readercount	0	0	2	Two readers active
9. R2 unlocks x	1	0	2	y not changed (already 0)
10. R2 reading	1	0	2	Both R1 & R2 reading
11. R1 locks x (leaving)	0	0	2	Updating count
12. R1 decrements readercount	0	0	1	One reader left
13. R1 unlocks x	1	0	1	y still 0 (since one reader remains)
14. R2 locks x (leaving)	0	0	1	Updating count
15. R2 decrements readercount	0	0	0	Last reader leaving
16. R2 unlocks y	0	1	0	Writers allowed now
17. R2 unlocks x	1	1	0	Reader section complete
18. W1 locks y	1	0	0	Writer enters
19. W1 writing	1	0	0	Exclusive access
20. W1 unlocks y	1	1	0	Resource free



Banker's Algorithm

Banker's Algorithm is a method used by an operating system to avoid deadlocks by **only giving resources when it is sure that all processes can finish safely.**

It checks before allocating: *"Will everyone still be able to finish?"*

What data it uses:

For each process:

- **Allocation** → resources currently given
- **Max** → maximum resources it may need
- **Need** → what it still needs
 - **Need = Max – Allocation**
- **Available** → free resources in system



Key Concepts in Banker's Algorithm

1. Safe State:

There is **some order** in which all processes can finish.

2. Unsafe State:

No matter what you try, **someone might get stuck later**.

Current Available Resources = 3

Process	Max	Allocation	Need
P0	7	1	6
P1	5	2	3
P2	8	3	5

Step	Available	Process Selected	Condition (Need ≤ Available)	New Available
1	3	P1	$3 \leq 3 \checkmark$	$3 + 2 = 5$
2	5	P2	$5 \leq 5 \checkmark$	$5 + 3 = 8$
3	8	P0	$6 \leq 8 \checkmark$	$8 + 1 = 9$

STEPS:

1. Calculate → NEED
2. Compare → Need ≤ Available
3. If yes → run process
4. Add allocation back
5. Repeat

Safe Sequence

P1 → P2 → P0

Total Available Resources = 3

Process	Allocated	Max	Need (Max – Alloc)
P1	1	7	6
P2	2	5	3
P3	3	8	5



Total Available Resources = 3

Total Allocated = 1 + 2 + 3 = 6

Current Available Resources= Total A.R – Allocated= 3 – 6 = -3

Process	Need	Available	Can Execute?
P1	6	-3	No
P2	3	-3	No
P3	5	-3	No



Unsafe State

Example:

Considering a system with five processes P_0 through P_4 and three resources of type A, B, C., with **current availability**

Process	Allocation	Max	Available
	A B C	A B C	A B C
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

Need Matrix

$\text{Need}[i, j] = \text{Max}[i, j] - \text{Allocation}[i, j]$

So, the content of Need Matrix is:

Process	Need		
	A	B	C
P ₀	7	4	3
P ₁	1	2	2
P ₂	6	0	0
P ₃	0	1	1
P ₄	4	3	1

Safe State and Safe Sequence Analysis

Applying the Safety algorithm on the given system,

Step 1 of Safety Algo

$m = 3, n = 5$

Work = Available

Work =

3	3	2
---	---	---

Finish =

0	1	2	3	4
false	false	false	false	false

Step 2

For $i = 0$

$Need_0 = 7, 4, 3$

Finish [0] is false and $Need_0 > Work$

So P_0 must wait

But $Need \leq Work$



Step 2

For $i = 1$

$Need_1 = 1, 2, 2$

Finish [1] is false and $Need_1 < Work$

So P_1 must be kept in safe sequence



Step 3

$3, 3, 2$ $2, 0, 0$
 $Work = Work + Allocation_1$

Work =

A	B	C
5	3	2

Finish =

0	1	2	3	4
false	true	false	false	false

Step 2

For i = 2

$Need_2 = 6, 0, 0$

Finish [2] is false and $Need_2 > Work$
So P_2 must wait



Step 2

For i = 3

$Need_3 = 0, 1, 1$

Finish [3] is false and $Need_3 < Work$
So P_3 must be kept in safe sequence



Step 3

$5, 3, 2$ $2, 1, 1$
 $\text{Work} = \text{Work} + \text{Allocation}_3$

Work =

A	B	C
7	4	3

Finish =

0	1	2	3	4
false	true	false	true	false

Step 2

For $i = 4$

$\text{Need}_4 = 4, 3, 1$

Finish [4] is false and $\text{Need}_4 < \text{Work}$
So P_4 must be in safe sequence



Step 3

$7, 4, 3$ $0, 0, 2$
 $\text{Work} = \text{Work} + \text{Allocation}_4$

Work =

A	B	C
7	4	5

Finish =

0	1	2	3	4
false	true	false	true	true

Step 2

For $i = 0$

$\text{Need}_0 = 7, 4, 3$

Finish [0] is false and $\text{Need}_0 < \text{Work}$
So P_0 must be in safe sequence



Step 3

$$\text{Work} = \overset{7,4,5}{\text{Work}} + \overset{0,1,0}{\text{Allocation}_0}$$

Work =

A	B	C
7	5	5

Finish =

0	1	2	3	4
true	true	false	true	true

Step 2

For i = 2

$$\text{Need}_2 = 6, 0, 0$$

Finish [2] is false and $\text{Need}_2 < \text{Work}$
So P_2 must be in safe sequence



Step 3

$7, 5, 5$ $3, 0, 2$
Work = Work + Allocation₂

Work =

A	B	C
10	5	7

Finish =

0	1	2	3	4
true	true	true	true	true

Step 4

Finish [i] = true for $0 \leq i \leq n$

Hence the system is in Safe State

The Safe Sequence is P_1, P_3, P_4, P_0, P_2

Problem-01

A single processor system has three resource types X, Y and Z, which are shared by three processes. **There are 5 units of each resource type.** Consider the following scenario, where the column alloc denotes the number of units of each resource type allocated to each process, and the column request denotes the number of units of each resource type requested by a process in order to complete execution. Which of these processes will finish LAST?

- 1.P0
- 2.P1
- 3.P2
- 4.None of the above since the system is in a deadlock

	Alloc			Request		
	X	Y	Z	X	Y	Z
P0	1	2	1	1	0	3
P1	2	0	1	0	1	2
P2	2	2	1	1	2	0

Process	Allocation (X Y Z)	Max (X Y Z)	Need (X Y Z)
P0	(1, 2, 1)	(2, 2, 4)	(1, 0, 3)
P1	(2, 0, 1)	(2, 1, 3)	(0, 1, 2)
P2	(2, 2, 1)	(3, 4, 1)	(1, 2, 0)

Step 1: Compute Available

Sum of allocations:

- $X = 1 + 2 + 2 = 5$
- $Y = 2 + 0 + 2 = 4$
- $Z = 1 + 1 + 1 = 3$

(Current Available = Total Available – Sum of Allocated)

so, $(5,5,5) - (5,4,3) = (0,1,2)$

Step 2: Find Who Can Finish

Available = (0,1,2)

- P0 needs (1,0,3) → not possible
- P1 needs (0,1,2) → possible
- P2 needs (1,2,0) → not possible

So P1 finishes first

Step 3: Release P1 Resources

New Available = $(0,1,2) + (2,0,1) = (2,1,3)$

Available = (2,1,3)

- P2 needs (1,2,0) → not possible
- P0 needs (1,0,3) → possible

P0 finishes next

Step 4: Release P0 Resources

New Available = $(2,1,3) + (1,2,1) = (3,3,4)$

Available = (3,3,4)

P2 needs (1,2,0) → possible

P2 finishes last

Safe Sequence

P1 → P0 → P2

Problem-02

	Allocation			Max		
	X	Y	Z	X	Y	Z
P0	0	0	1	8	4	3
P1	3	2	0	6	2	0
P2	2	1	1	3	3	3

An operating system uses the banker's algorithm for deadlock avoidance when managing the allocation of three resource types X, Y and Z to three processes P0, P1 and P2. The table given below presents the current system state. Here, the Allocation matrix shows the current number of resources of each type allocated to each process and the Max matrix shows the maximum number of resources of each type required by each process during its execution.

There are 3 units of type X, 2 units of type Y and 2 units of type Z still available. The system is currently in safe state.

Current Available = [X Y Z] = [3 2 2]

Process	Allocation (X Y Z)	Max (X Y Z)	Need (X Y Z)
P0	(0, 0, 1)	(8, 4, 3)	(8, 4, 2)
P1	(3, 2, 0)	(6, 2, 0)	(3, 0, 0)
P2	(2, 1, 1)	(3, 3, 3)	(1, 2, 2)

Check Safe Sequence

Current Available = (3, 2, 2)

•P1 can run $\rightarrow (3,0,0) \leq (3,2,2)$ ✓

New Available = (6,4,2)

•P2 can run $\rightarrow (1,2,2) \leq (6,4,2)$ ✓

New Available = (8,5,3)

•P0 can run $\rightarrow (8,4,2) \leq (8,5,3)$ ✓

Safe Sequence

P1 $\rightarrow$ P2 $\rightarrow$ P0

Example:

Considering a system with five processes P_0 through P_4 and three resources of type A, B, C. Resource type A has 10 instances, B has 5 instances and type C has 7 instances. Suppose at time t_0 following snapshot of the system has been taken:

Process	Allocation	Max	Available
	A B C	A B C	A B C
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

Question1. What will be the content of the Need matrix?

$\text{Need}[i, j] = \text{Max}[i, j] - \text{Allocation}[i, j]$

So, the content of Need Matrix is:

Process	Need		
	A	B	C
P_0	7	4	3
P_1	1	2	2
P_2	6	0	0
P_3	0	1	1
P_4	4	3	1

Question1. What will be the content of the Need matrix?

$$\text{Need}[i, j] = \text{Max}[i, j] - \text{Allocation}[i, j]$$

So, the content of Need Matrix is:

Process	Need		
	A	B	C
P ₀	7	4	3
P ₁	1	2	2
P ₂	6	0	0
P ₃	0	1	1
P ₄	4	3	1

Question2. Is the system in a safe state? If Yes, then what is the safe sequence?

Applying the Safety algorithm on the given system,

Step 1 of Safety Algo

m=3, n=5

Work = Available

Work =

3	3	2
---	---	---

Finish =

false	false	false	false	false
-------	-------	-------	-------	-------

Step 2

For i=3

Need₃ = 0, 1, 1

Finish [3] = false and Need₃ < Work

So P₃ must be kept in safe sequence

Step 3

Work = Work + Allocation₃

Work =

7	5	5
---	---	---

Finish =

false	true	false	true	false
-------	------	-------	------	-------

Step 2

For i=2

Need₂ = 6, 0, 0

Finish [2] is false and Need₂ < Work

So P₂ must be kept in safe sequence

Step 3

Work = Work + Allocation₂

Work =

10	5	7
----	---	---

Finish =

true	true	true	true	true
------	------	------	------	------

Step 2

For i=4

Need₄ = 4, 3, 1

Finish [4] = false and Need₄ < Work

So P₄ must be kept in safe sequence

Step 3

Work = Work + Allocation₄

Work =

7	4	5
---	---	---

Finish =

false	true	false	true	true
-------	------	-------	------	------

Step 2

For i=0

Need₀ = 7, 4, 3

Finish [0] is false and Need₀ > Work

So P₀ must wait

Step 2

For i=1

Need₁ = 1, 2, 2

Finish [1] is false and Need₁ < Work

So P₁ must be kept in safe sequence

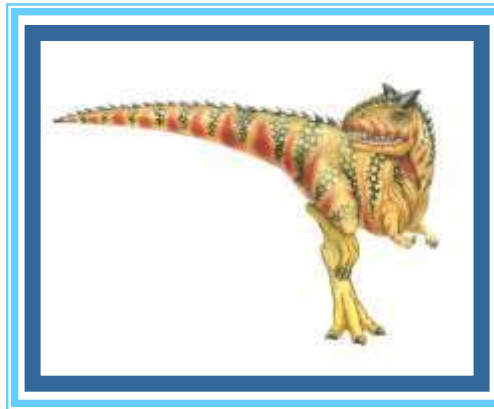
Step 4

Finish [i] = true for 0 ≤ i ≤ n

Hence the system is in Safe state

The safe sequence is P₁, P₃, P₄, P₀, P₂

Chapter 6: CPU Scheduling





CPU Scheduling in Operating Systems

- CPU scheduling is how the operating system chooses which program runs on the CPU next.
- It's needed because many programs want to run at the same time, but the CPU can only handle one at a time.

It helps:

- Keep the CPU working all the time
- Reduce waiting time for programs
- Make sure programs get a fair chance to run





Terminologies Used in CPU Scheduling

Terminologies Used in CPU Scheduling

- **Arrival Time:** The time at which the process arrives in the ready queue.
- **Completion Time:** The time at which the process completes its execution.
- **Burst Time:** Time required by a process for CPU execution.
- **Turn Around Time:** Time Difference between completion time and arrival time.

$$\text{Turn Around Time} = \text{Completion Time} - \text{Arrival Time}$$

- **Waiting Time(W.T):** Time Difference between turn around time and burst time.

$$\text{Waiting Time} = \text{Turn Around Time} - \text{Burst Time}$$





Things to Consider for Designing a CPU Scheduling Algorithm

Different **CPU scheduling algorithms** have different designs, and the choice depends on several performance factors:

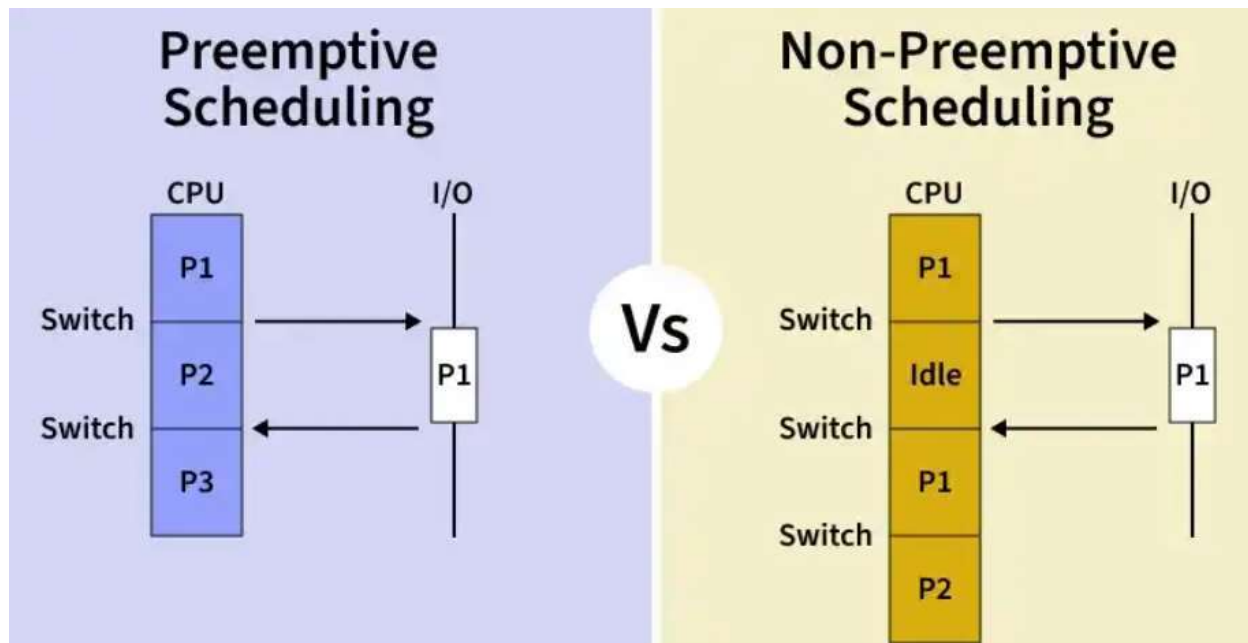
- **CPU Utilization:** How much the CPU is being used. Higher is better (closer to 100%).
- **Throughput:** How many processes finish in a given time. Higher throughput means more work is finished in less time.
[No. of Process/Total Time]
- **Turnaround Time:** The total time taken for a process from submission to completion. It includes waiting time, CPU execution time, and I/O time. **[Completion Time- Arrival Time]**
- **Waiting Time:** The total time a process spends in the **ready queue** waiting for CPU access. (How long a process waits before getting CPU time, Less is better).





Different Types of CPU Scheduling Algorithms

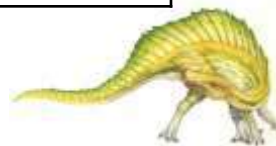
- There are two main types of CPU scheduling:
- **Preemptive Scheduling:** The CPU can take control away from a running process. This happens when a process is moved from **running** → **ready state** or **waiting** → **ready state**. In this method, a process can be interrupted.
- **Non-Preemptive Scheduling:** Once a process starts using the CPU, it keeps it until it finishes or moves to the **waiting state** (for I/O). The CPU is not taken away in the middle of execution.





Differences Between Preemptive and Non-Preemptive Scheduling

Parameter	PREEMPTIVE SCHEDULING	NON-PREEMPTIVE SCHEDULING
Basic	In this resources(CPU Cycle) are allocated to a process for a limited time.	Once resources(CPU Cycle) are allocated to a process, the process holds it till it completes its burst time or switches to waiting state
Interrupt	Process can be interrupted in between.	Process can not be interrupted until it terminates itself or its time is up
Starvation	If a process having high priority frequently arrives in the ready queue, a low priority process may starve	If a process with a long burst time is running CPU, then later coming process with less CPU burst time may starve
Flexibility	flexible	Rigid
Response Time	Preemptive scheduling response time is less	Non-preemptive scheduling response time is high
Process control	The OS has greater control over the scheduling of processes	The OS has less control over the scheduling of processes
Examples	Examples of preemptive scheduling are Round Robin and Shortest Remaining Time First	Examples of non-preemptive scheduling are First Come First Serve and Shortest Job First





CPU Scheduling Algorithms

Let us now learn about these few CPU scheduling algorithms in operating systems one by one:

- FCFS - First Come, First Serve
- SJF - Shortest Job First
- Round Robin
- Priority Scheduling





FCFS - First Come First Serve CPU Scheduling

First Come, First Serve (FCFS) is a simple CPU scheduling algorithm where processes are handled in the order they arrive in the **ready queue**, just like people standing in a line.

- It is a **non-preemptive algorithm**, meaning once a process starts running, it cannot be stopped in the middle.
- The process keeps using the CPU until it finishes or goes for **I/O (input/output)**.
- It does not consider priority or process size, only arrival time matters.





How Does FCFS Work?

How FCFS works:

- **Arrival:** Processes are placed in a queue in the order they arrive.
- **Execution:** The CPU picks the first process in the queue and runs it completely until it finishes.
- **Repeat:** Then it moves to the next process in the queue and does the same.
- This continues until all processes are completed.

FCFS examples:

To understand FCFS better, we use two cases:

- When all processes arrive at the same time
- When processes arrive at different times

In both cases, we can draw a **Gantt chart** and calculate:

- **Turnaround time**
- **Waiting time**





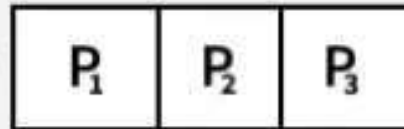
Example

Given:

Processes with their Arrival Time and Burst Time:

Process No	Arrival Time	Burst Time
P1	0	5
P2	0	3
P3	0	8

Ready Queue :
at $t = 0$



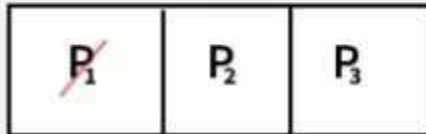


Solution

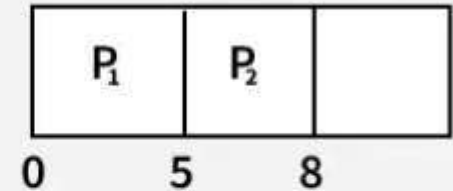
Gantt chart at t = 5



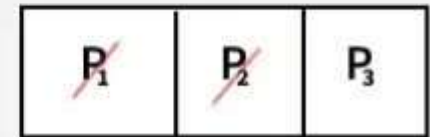
Ready Queue :
at t = 5



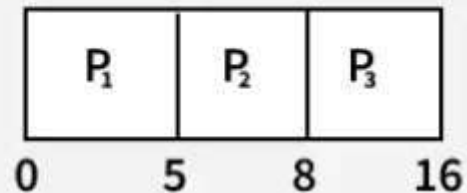
Gantt chart at t = 8



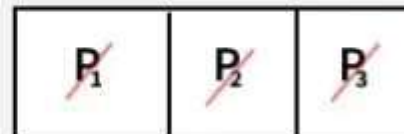
Ready Queue :
at t = 8



Gantt chart at t = 16



Ready Queue :
at t = 16





Solution

Now, let's calculate average waiting time and turn around time:

Turnaround Time = Completion Time - Arrival Time

Waiting Time = Turnaround Time - Burst Time

AT : Arrival Time

BT : Burst Time or CPU Time

TAT : Turn Around Time

WT : Waiting Time

Processes	AT	BT	CT	TAT	WT
P1	0	5	5	$5 - 0 = 5$	$5 - 5 = 0$
P2	0	3	8	$8 - 0 = 8$	$8 - 3 = 5$
P3	0	8	16	$16 - 0 = 16$	$16 - 8 = 8$

•Average Turn around time = 9.67

•Average waiting time = 4.33





Example2

Given:

Processes with their Arrival Time and Burst Time:

Process	Arrival Time (AT)	Burst Time (BT)
P2	0	3
P1	2	5
P3	4	4





Solution

Process	Arrival Time (AT)	Burst Time (BT)	Completion Time (CT)	Turnaround Time (TAT = CT - AT)	Waiting Time (WT = TAT - BT)
P2	0	3	3	$3 - 0 = 3$	$3 - 3 = 0$
P1	2	5	8	$8 - 2 = 6$	$6 - 5 = 1$
P3	4	4	12	$12 - 4 = 8$	$8 - 4 = 4$

Average Turnaround Time = $(3 + 6 + 8) / 3 = 5.67$

Average Waiting Time = $(0 + 1 + 4) / 3 = 1.67$





Shortest Job First or SJF CPU Scheduling

Shortest Job First (SJF) is a CPU scheduling method where the process with the **smallest burst time (execution time)** is selected first.

- It can be **preemptive or non-preemptive**.
- It helps reduce the **average waiting time** for processes.

How SJF works:

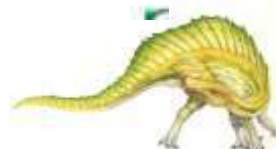
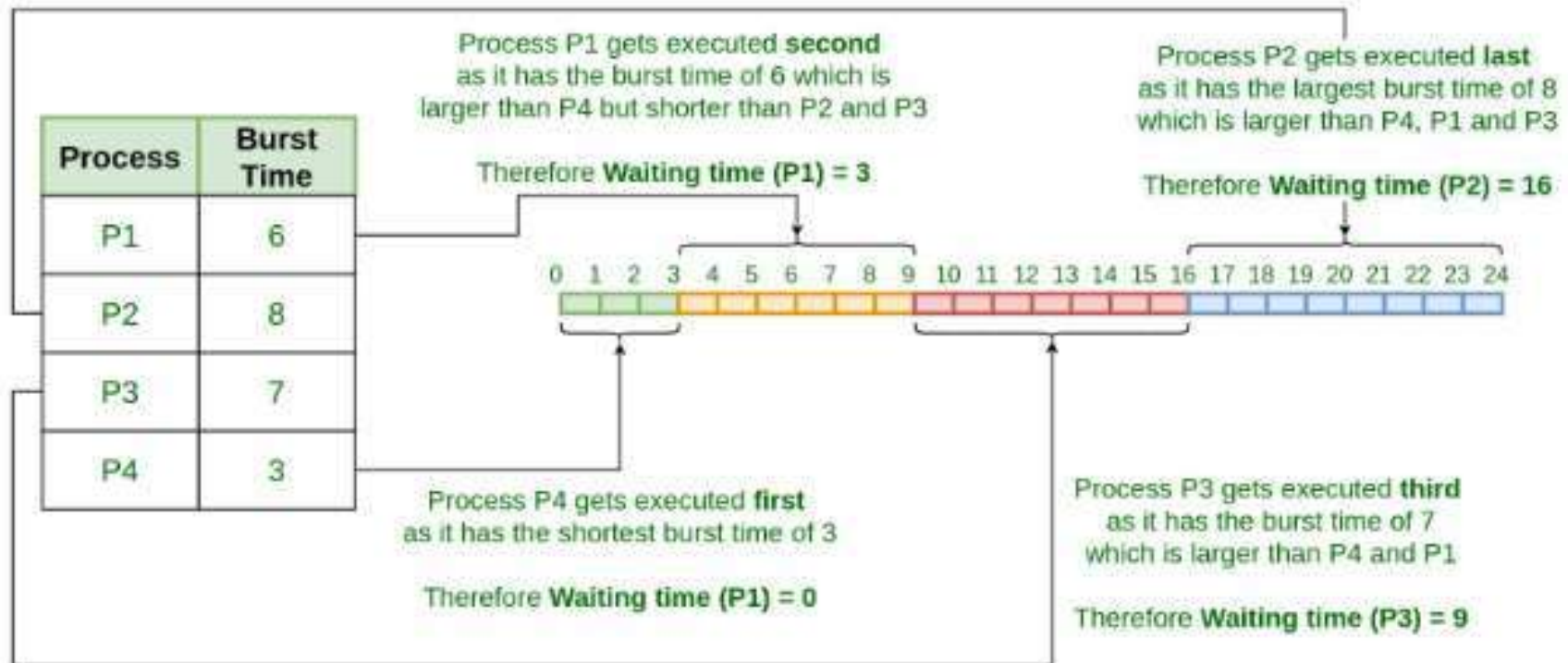
- Arrange processes based on **arrival time**.
- From the available processes, pick the one with the **shortest burst time**.
- Run that process first.
- As new processes arrive, add them to the **ready queue**.
- Again select the process with the **smallest burst time** from the queue.
- Repeat until all processes are finished.
- **In short:**
SJF always chooses the process that takes the **least time to complete**, to improve efficiency and reduce waiting time.





Shortest Job First or SJF CPU Scheduling

Shortest Job First (SJF) Scheduling Algorithm





SJF Gantt chart

01
Step

at $t = 0$

Process	Arrival Time	Burst Time
P_1	0 ms	6 ms
P_2	2 ms	8 ms
P_3	4 ms	3 ms

Ready Queue :

P_1	
-------	--

Gantt chart :

P_1	
-------	--

0 6



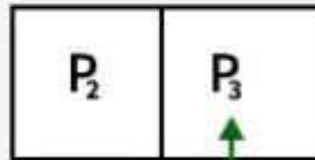


SJF Gantt chart

02
Step

at $t = 6$

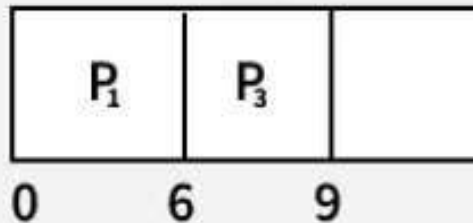
Ready Queue :



↑
shortest burst
time

Remaining Time (P_1) : 0 ms

Gantt chart :



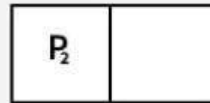


SJF Gantt chart

03
Step

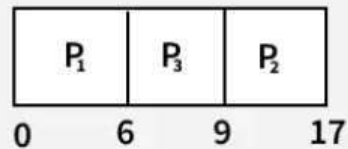
at $t = 9$

Ready Queue :



Remaining Time (P_3) : 0 ms

Gantt chart :



Example of Non-Pre-emptive Shortest Job First CPU Scheduling Algorithm



As we know,

- Turn Around time = Completion time - arrival time
- Waiting Time = Turn around time - burst time

Now, let's calculate average waiting time and turn around time:

- **Average Turn around time** = $(6 + 15 + 5)/3 = 8.6 \text{ ms}$
- **Average waiting time** = $(2 + 0 + 7)/3 = 9/3 = 3 \text{ ms}$

Process	Arrival Time (AT)	Burst Time (BT)	Completion Time (CT)	Turn Around Time (TAT)	Waiting Time (WT)
P1	0	6	6	$6 - 0 = 6$	$6 - 6 = 0$
P2	2	8	17	$17 - 2 = 15$	$15 - 8 = 7$
P3	4	3	9	$9 - 4 = 5$	$5 - 3 = 2$





Round Robin Scheduling in OS

Round Robin is a CPU scheduling method where each process gets a small fixed amount of time called a **time quantum**.

The CPU works like a circle:

- It gives each process a turn one by one.
- Each process runs only for its fixed time slice.
- If a process is not finished, it goes to the back of the queue.
- This continues until all processes are done.

Main idea:

Every process gets a fair chance to use the CPU, so no process is ignored.
(Each process gets an equal share of the CPU)





Example of Round Robin Scheduling Algorithm:

- To understand the Round Robin Scheduling algorithm, let's consider the following two scenarios:
- **Scenario 1: Processes with Same Arrival Time**
- Consider the following table of arrival time and burst time for three processes P1, P2 and P3 and given **Time Quantum = 2 ms**

Process	Burst Time	Arrival Time
P1	4 ms	0 ms
P2	5 ms	0 ms
P3	3 ms	0 ms





Example of Round Robin Scheduling Algorithm:

- **Step-by-Step Execution:**
- **Time 0-2 (P1):** P1 runs for 2 ms (total time left: 2 ms).
- **Time 2-4 (P2):** P2 runs for 2 ms (total time left: 3 ms).
- **Time 4-6 (P3):** P3 runs for 2 ms (total time left: 1 ms).
- **Time 6-8 (P1):** P1 finishes its last 2 ms.
- **Time 8-10 (P2):** P2 runs for another 2 ms (total time left: 1 ms).
- **Time 10-11 (P3):** P3 finishes its last 1 ms.
- **Time 11-12 (P2):** P2 finishes its last 1 ms.





Gantt chart for Round Robin Scheduling Algorithm:

01
Step

Time quantum = 2ms
at $t = 0$

Process	Arrival Time	Burst Time
P_1	0 ms	4 ms
P_2	0 ms	5 ms
P_3	0 ms	3 ms

Ready Queue :

P_1	P_2	P_3		
-------	-------	-------	--	--

Round Robin Scheduling Algorithm with same arrival time



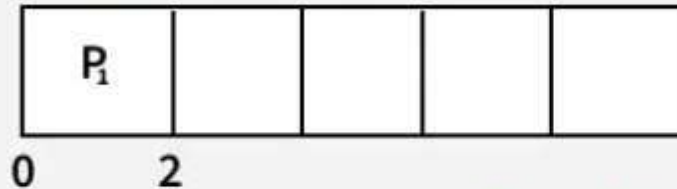


Gantt chart for Round Robin Scheduling Algorithm:

02
Step

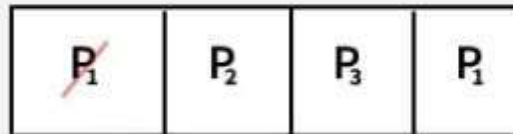
at $t = 2$

Gantt chart



Remaining time left(P_1) : 2 ms

Ready Queue :



Round Robin Scheduling Algorithm with same arrival time



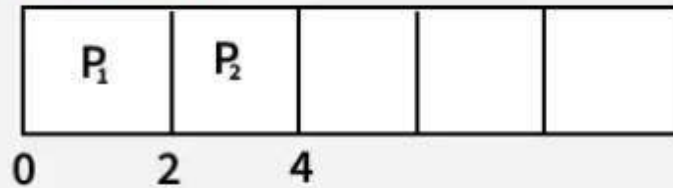


Gantt chart for Round Robin Scheduling Algorithm:

03
Step

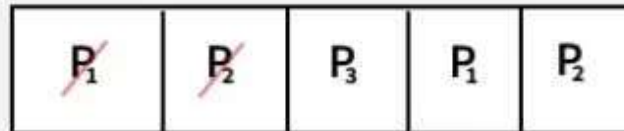
at $t = 4$

Gantt chart



remaining time left(P_2) : 3 ms

Ready Queue :



Round Robin Scheduling Algorithm with same arrival time



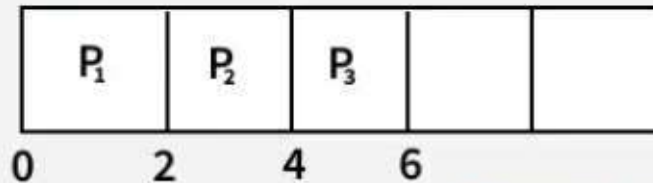


Gantt chart for Round Robin Scheduling Algorithm:

04
Step

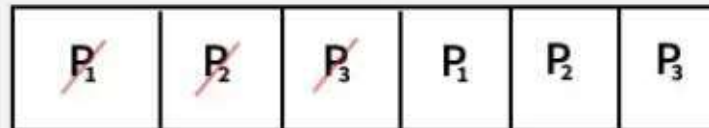
at $t = 6$

Gantt chart



Remaining time left(P_3) : 1 ms

Ready Queue :



Round Robin Scheduling Algorithm with same arrival time



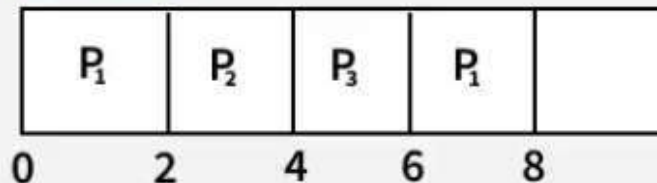


Gantt chart for Round Robin Scheduling Algorithm:

05
Step

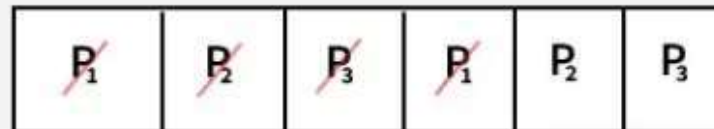
at $t = 8$

Gantt chart



Remaining time left(P_j) : 0 ms

Ready Queue :



Round Robin Scheduling Algorithm with same arrival time





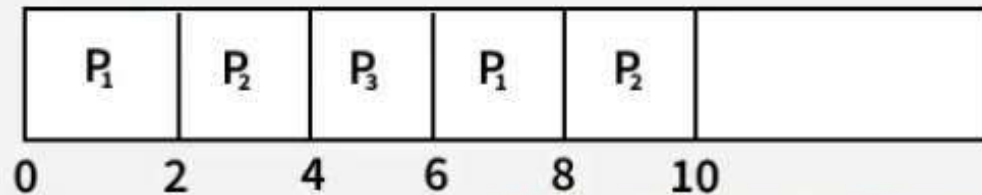
Gantt chart for Round Robin Scheduling Algorithm:

06

Step

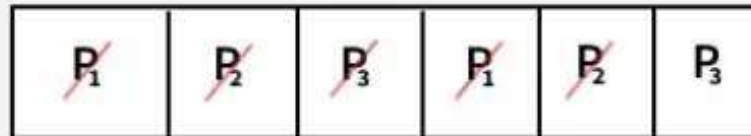
at $t = 10$

Gantt chart



Remaining time left(P_2) : 1 ms

Ready Queue :



Round Robin Scheduling Algorithm with same arrival time



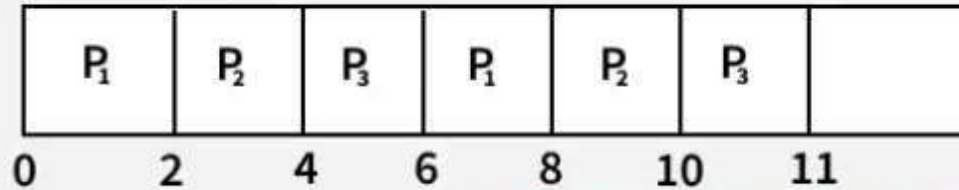


Gantt chart for Round Robin Scheduling Algorithm:

07
Step

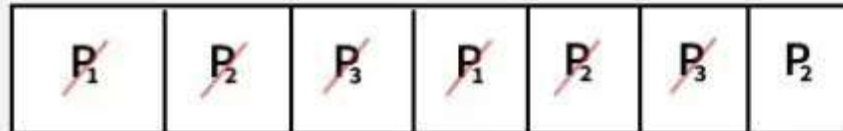
at $t = 11$

Gantt chart



Remaining time left(P_3) : 0 ms

Ready Queue :



Round Robin Scheduling Algorithm with same arrival time



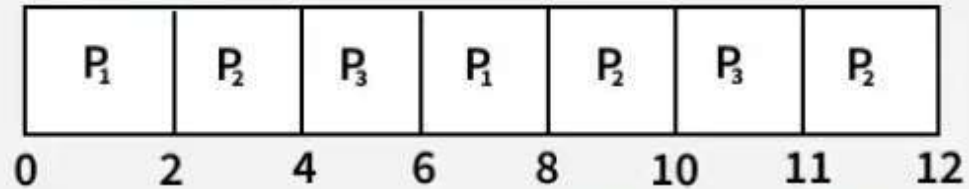


Gantt chart for Round Robin Scheduling Algorithm:

08
Step

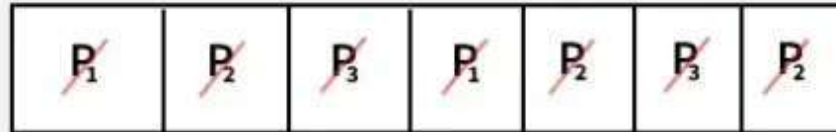
at $t = 12$

Gantt chart



Remaining time left(P_2) : 0 ms

Ready Queue :



Round Robin Scheduling Algorithm with same arrival time





Now, lets calculate average waiting time and turn around time:

- Turnaround Time = Completion Time - Arrival Time
- Waiting Time = Turnaround Time - Burst Time
- **Average Turn around time** = $(8 + 12 + 11)/3 = 31/3 = 10.33$ ms
- **Average waiting time** = $(4 + 7 + 8)/3 = 19/3 = 6.33$ ms

Processes	AT	BT	CT	TAT	WT
P1	0	4	8	$8 - 0 = 8$	$8 - 4 = 4$
P2	0	5	12	$12 - 0 = 12$	$12 - 5 = 7$
P3	0	3	11	$11 - 0 = 11$	$11 - 3 = 8$



Scenario 2: Processes with Different Arrival Times



- Consider the following table of arrival time and burst time for three processes P1, P2 and P3 and given **Time Quantum = 2**

Process	Burst Time (BT)	Arrival Time (AT)
P1	5 ms	0 ms
P2	2 ms	4 ms
P3	4 ms	5 ms





Step-by-Step Execution:

- **Time 0-2 (P1 Executes):**
 - P1 starts execution as it arrives at 0 ms.
 - Runs for 2 ms; remaining burst time = $5 - 2 = 3$ ms.
 - **Ready Queue:** [P1].
- **Time 2-4 (P1 Executes Again):**
 - P1 continues execution since no other process has arrived yet.
 - Runs for 2 ms; remaining burst time = $3 - 2 = 1$ ms.
 - P2 arrive at 4 ms.
 - **Ready Queue:** [P2, P1].
- **Time 4-6 (P2 Executes):**
 - P2 starts execution as it arrives at 4 ms.
 - Runs for 2 ms; remaining burst time = $2 - 2 = 0$ ms.
 - P3 arrive at 5ms
 - **Ready Queue:** [P1, P3].
- **Time 6-7 (P1 Executes):**
 - P1 starts execution.
 - Runs for 1 ms; remaining burst time = $1 - 1 = 0$ ms.
 - **Ready Queue:** [P3].
- **Time 7-9 (P3 Executes):**
 - P3 starts execution.
 - Remaining burst time = $4 - 2 = 2$ ms.
 - **Ready Queue:** [P3].
- **Time 9-11 (P3 Executes Again):**
 - P3 resumes execution and runs for 2 ms and complete its execution
 - Remaining burst time = $2 - 2 = 0$ ms.
 - **Ready Queue:** [].





Gantt Chart:

01
Step

Time quantum = 2ms

at $t = 0$

Process	Arrival Time	Burst Time
P_1	0 ms	5 ms
P_2	4 ms	2 ms
P_3	5 ms	4 ms

Ready Queue :

P_1					
-------	--	--	--	--	--

Round Robin Scheduling Algorithm with different Arrival Time



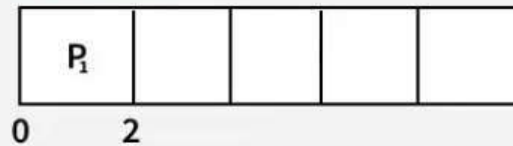


Gantt Chart:

02
Step

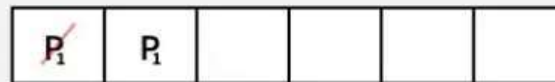
at $t = 2$

Gantt chart



Remaining time left (P_1) : 3ms

Ready Queue :



Round Robin Scheduling Algorithm with different Arrival Time



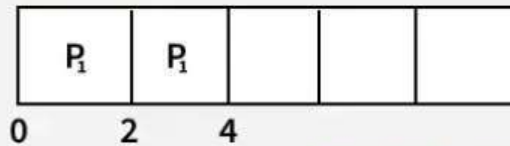


Gantt Chart:

03
Step

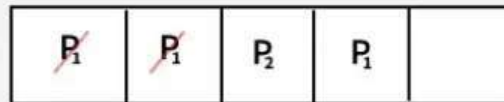
at $t = 4$

Gantt chart



Remaining time left (P_1) : 1ms

Ready Queue :



Round Robin Scheduling Algorithm with different Arrival Time



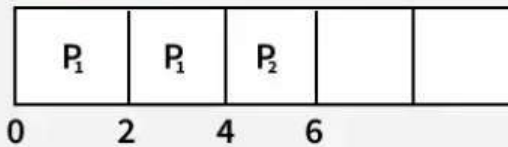


Gantt Chart:

04
Step

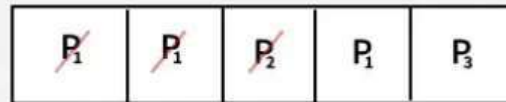
at $t = 6$

Gantt chart



Remaining time left (P_2) : 0ms

Ready Queue :



Round Robin Scheduling Algorithm with different Arrival Time



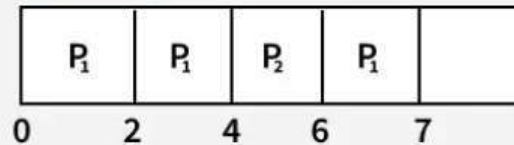


Gantt Chart:

05
Step

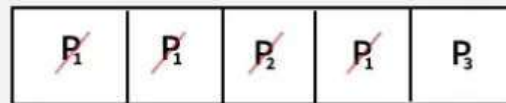
at $t = 7$

Gantt chart



Remaining time left (P_1) : 0ms

Ready Queue :



Round Robin Scheduling Algorithm with different Arrival Time



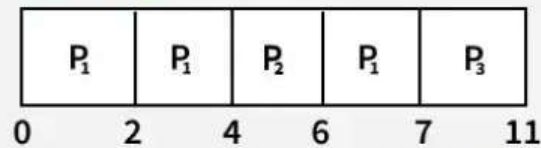


Gantt Chart:

06
Step

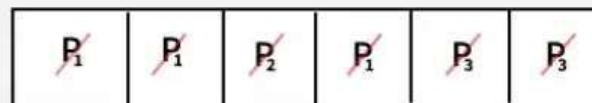
at $t = 11$

Gantt chart



Remaining time left (P_1) : 0ms

Ready Queue :



Round Robin Scheduling Algorithm with different Arrival Time





Now, lets calculate average waiting time and turn around time:

- Average Turn around time $= 7 + 2 + 6 / 3 = 15 / 3 = 5\text{ms}$
- Average waiting time $= 2 + 0 + 2 / 3 = 1.33\text{ms}$

Process	Completion Time (CT)	Turnaround Time (TAT = CT - AT)	Waiting Time (WT = TAT - BT)
P1	7 ms	7 ms	2 ms
P2	6 ms	2 ms	0 ms
P3	11 ms	6 ms	2 ms





Now, lets calculate average waiting time and turn around time:

- Average Turn around time $= 7 + 2 + 6 / 3 = 15 / 3 = 5\text{ms}$
- Average waiting time $= 2 + 0 + 2 / 3 = 1.33\text{ms}$

Process	Completion Time (CT)	Turnaround Time (TAT = CT - AT)	Waiting Time (WT = TAT - BT)
P1	7 ms	7 ms	2 ms
P2	6 ms	2 ms	0 ms
P3	11 ms	6 ms	2 ms





Preemptive Priority Scheduling

In this method, the CPU is always given to the process with the **highest priority**.

Step-by-Step Execution:

- 1. Time 0–4:** P2 runs (burst = 4)
- 2. Time 4–11:** P1 runs (burst = 7)
- 3. Time 11–17:** P3 runs (burst = 6).

Process	Arrival Time	Burst Time	Priority
P1	0	7	2
P2	0	4	1
P3	0	6	3





Preemptive Priority Scheduling

Process	AT	BT	Priority	CT	TAT = CT-AT	WT = TAT-BT
P2	0	4	1	4	4	0
P1	0	7	2	11	11	4
P3	0	6	3	17	17	11





Non-Preemptive Priority Scheduling

Process	AT	BT	Priority
P1	0	7	2
P2	2	4	1
P3	4	6	3

Time 0:

- Only P1 is available → runs first

P1 runs:

- 0 → 7

Time 7:

- P2 (priority 1)
 - P3 (priority 3)
- So, P2 runs next

P2 runs:

- 7 → 11

P3 runs:

- 11 → 17

Execution Order:
P1 → P2 → P3





Non-Preemptive Priority Scheduling

Process	AT	BT	Priority	CT	TAT = CT-AT	WT = TAT-BT
P1	0	7	2	7	7	0
P2	2	4	1	11	9	5
P3	4	6	3	17	13	7



CPU Scheduling Exercises

Problem 1

Solutions

Process	Burst	Priority
P ₁	8	4
P ₂	6	1
P ₃	1	2
P ₄	9	2
P ₅	3	3

First Come First Served

0	8	14	15	24	27
P ₁	P ₂	P ₃	P ₄	P ₅	

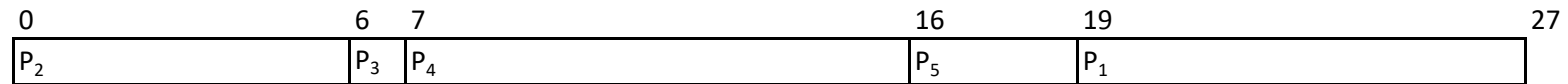
Avg. Wait = $0+8+14+15+24 = 61/5 = 12.2\text{ms}$ Avg. TAT = $8+14+15+24+27 = 88/5 = 17.6\text{ms}$

Shortest Job First

0	1	4	10	18	27
P ₃	P ₅	P ₂	P ₁	P ₄	

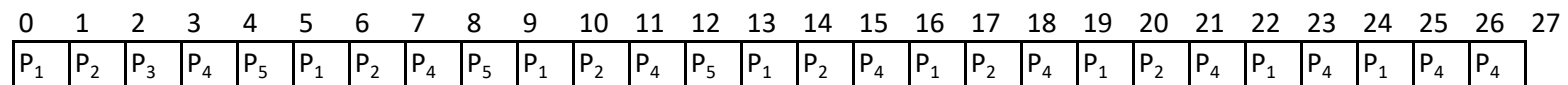
Avg. Wait = $0+1+4+10+18 = 33/5 = 6.6\text{ms}$ Avg. TAT = $1+4+10+18+27 = 60/5 = 12\text{ms}$

Non-Preemptive Priority



Avg. Wait Time = $0+6+7+16+19 = 48/5 = 9.6\text{ms}$ Avg TAT = $6+7+16+19+27 = 75/5 = 15\text{ms}$

Round Robin (1ms Quantum)



Wait Time P₁ = $0+5-1+9-6+13-10+16-14+19-17+22-20+24-23 = 0+4+3+3+2+2+2+1 = 17$

Wait Time P₂ = $1+6-2+10-7+14-11+17-15+20-18 = 1+4+3+3+2+2 = 15$

Wait Time P₃ = 2

Wait Time P₄ = $3+7-4+11-8+15-12+18-16+21-19+23-22+25-24 = 3+3+3+3+2+2+1+1 = 18$

Wait Time P₅ = $4+8-5+12-9 = 4+3+3 = 10$

Avg Wait Time = $62/5 = 12.4\text{ms}$

Avg TAT = $25+21+3+27+13 = 89/5 = 17.8\text{ms}$

Algorithm	Avg Wait	Avg TAT
FCFS	12.2	17.6
SJF	6.6	12
NonP Priority	9.6	15
RR	12.4	17.8

SJF has shortest wait and shortest TAT